



Operační systémy

IOS

Studijní opora

Tomáš Kašpárek, Radek Kočí, Petr Peringer, Tomáš Vojnar

30. května 2007

Tento učební text vznikl za podpory projektu „Zvýšení konkurenceschopnosti IT odborníků – absolventů pro Evropský trh práce“, reg. č. CZ.04.1.03/3.2.15.1/0003. Tento projekt je spolufinancován Evropským sociálním fondem a státním rozpočtem České republiky.

Obsah

1 Úvod	4
1.1 Metodické informace	4
1.2 Další zdroje informací	4
1.3 Značky použité v textu	5
1.4 Autoři	5
2 Správa souborů	6
2.1 Základní principy	6
2.1.1 Atributy souboru	6
2.1.2 Operace nad souborem	7
2.1.3 Typy souborů	8
2.1.4 Řízení přístupu k souborům	10
2.1.5 Adresáře	12
2.1.6 Připojování adresářů – operace <i>mount</i>	14
2.2 Uložení dat na pevných discích	17
2.2.1 Hardware pevného disku	17
2.2.2 Struktura rozdělení pevného disku	18
2.2.3 Pokročilé diskové systémy (RAID, LVM)	19
2.3 Algoritmy správy volného a obsazeného místa	22
2.3.1 Souvislá alokace	22
2.3.2 Alokace pomocí extentů	23
2.3.3 Alokace pomocí seznamů	23
2.3.4 Alokace pomocí indexů a stromů	23
2.3.5 Správa volných alokačních bloků	24
2.4 Výkon souborového systému	25
2.4.1 Paměť cache	25
2.4.2 Block Read Ahead	26
2.4.3 Omezení pohybu diskových hlaviček	26
2.4.4 Plánování sekvence V/V operací	27
2.5 VFS – Virtual Filesystem	28
2.5.1 Aplikační rozhraní pro vstup/výstup	28
2.6 Implementace souborových systémů	31
2.6.1 Klasické souborové systémy	31
2.6.2 Moderní (žurnálovací) souborové systémy	33
2.6.3 Síťové a distribuované souborové systémy	33

2.6.4	Virtuální souborové systémy	34
2.7	Shrnutí	35
2.8	Kontrolní otázky	37
2.9	Řešení vybraných kontrolních otázek	38
3	Správa procesů	39
3.1	Koncept procesů	39
3.1.1	Proces	39
3.1.2	Stavy procesu	40
3.1.3	Process Control Block	41
3.1.4	Přepnutí kontextu procesu	42
3.1.5	Vlákna	42
3.2	Plánování procesů	44
3.2.1	Plánovací fronta	44
3.2.2	Plánovač	45
3.2.3	Plánovací kritéria	46
3.3	Plánovací algoritmy	46
3.3.1	First-Come, First-Served	47
3.3.2	Shortest-Job-First	47
3.3.3	Priority Scheduling	49
3.3.4	Round-Robin	49
3.3.5	Víceúrovňové prioritní plánování	50
3.3.6	Problémy spojené s prioritním plánováním	51
3.4	Procesy v systému Unix	51
3.4.1	Hierarchie procesů	51
3.4.2	Vytvoření nového procesu	52
3.5	Komunikace procesů	53
3.5.1	Signály	54
3.5.2	Obsluha signálů	54
3.5.3	Zpracování signálů	55
3.5.4	Roury	57
3.6	Shrnutí	58
3.7	Kontrolní otázky	59
4	Virtuální paměť	60
4.1	Adresové prostory	60
4.2	Princip virtualizace paměti	61
4.3	Princip stránkování na žádost	64
4.4	Tabulky stránek	67
4.5	Překlad adres, MMU	71
4.5.1	TLB	72
4.5.2	Efektivita překladu adres	73
4.6	Odkládání stránek	74
4.6.1	Lokalita odkazů	74
4.6.2	Výběr stránky pro uvolnění	75
4.6.3	Swap	75

4.6.4	Souvislosti a problémy	75
4.7	Obsluha výpadku stránky	76
4.8	Výkonnost stránkování na žádost	78
4.8.1	Thrashing	79
4.8.2	Poznámky	79
4.9	Další vlastnosti systému stránkování na žádost	80
4.9.1	Sdílení stránek	80
4.9.2	Sdílená paměť	81
4.9.3	Paměťově mapované soubory	81
4.9.4	Sdílené knihovny	82
4.9.5	Zamykání stránek ve FAP	83
4.10	Shrnutí	83
4.11	Otázky	83
5	Synchronizace procesů	85
5.1	Časově závislé chyby	85
5.2	Problém kritické sekce	86
5.2.1	Specializované algoritmy pro vstup do kritické sekce	88
5.2.2	Využití hardware pro synchronizaci	90
5.2.3	Semaforey	91
5.2.4	Monitory	93
5.3	Některé klasické synchronizační problémy	95
5.3.1	Producent a konzument	95
5.3.2	Čtenáři a písaři	96
5.3.3	Večeřící filozofové	97
5.4	Uváznutí (deadlock)	99
5.4.1	Prevence uváznutí	100
5.4.2	Vyhýbání se uváznutí	101
5.4.3	Detekce uváznutí a zotavení	102
5.5	Formální verifikace	102
5.6	Shrnutí	104
5.7	Kontrolní otázky	105
5.8	Řešení vybraných kontrolních otázek	106

Kapitola 1

Úvod

Operační systém je součástí každého běžného výpočetního systému. Jedná se o komplexní programové vybavení vytvářející mezivrstvu mezi hardware počítače a jeho aplikačním vybavením a uživateli. Operační systémy měly, mají a zřejmě budou mít značný vliv na vývoj jak hardware tak software. Správné pochopení jejich principů má tedy značný význam jak pro vývojáře hardware tak software. Operační systémy navíc zahrnují řadu velmi sofistikovaných principů a řešení, které mohou být i samy o sobě inspirací pro řešení problémů vznikajících v nejrůznějších oblastech software a hardware.

1.1 Metodické informace

Tento *podpůrný* učební text je určen pro předmět „Operační systémy“ (IOS) v rámci bakalářského studia na FIT VUT v Brně. Jedná se o vybrané kapitoly z operačních systémů, a to ty, které patří dle zkušenosti autorů v rámci kursu IOS k nejnáročnějším. Konkrétně se jedná o následující:

- souborové systémy,
- procesy a plánování,
- správa paměti, virtuální paměť a
- synchronizace procesů.

Styl textu je relativně stručný – *cílem je usnadnit čtenáři orientaci v dané oblasti a pochopení jejich základních konceptů*. Předpokládá se ovšem, že čtenář využije také některé z níže uvedených dalších informačních zdrojů.

1.2 Další zdroje informací

Další podrobnosti k probírané látce je možné najít v literatuře odkazované z textu. Čtenáři je také doporučeno konfrontovat zde uvedené informace s informacemi volně dostupnými na Internetu (a snadno dosažitelnými např. prostřednictvím encyklopedií jako je www.wikipedia.org nebo pomocí vyhledávačů jako je www.google.com; řada průběžně obnovovaných odkazů na relevantní dokumenty je také uvedena na stránkách kursu IOS na FIT VUT, jež jsou dostupné na adrese <http://www.fit.vutbr.cz/study/courses/IOS>.

Dalším možným a velmi cenným zdrojem podrobnějších informací pro zvědavé studenty jsou pak také zdrojové kódy systémů jako Linux či FreeBSD, kde

je možné si do nejmenších detailů ověřit, jak zde abstraktně presentované principy jsou skutečně implementovány.

1.3 Značky použité v textu

Některé části textu jsou označeny piktogramy, které čtenáři usnadňují orientaci v textu. Jejich význam je popsán v tabulce 1.1.

	Čas potřebný pro studium		Otázka, příklad k řešení
	Cíl		Počítačové cvičení, příklad
	Definice		Příklad
	Důležitá část		Reference
	Rozšiřující látka		Správné řešení
	Obtížná část		Souhrn
			Slovo tutora, komentář
			Zajímavé místo

Tabulka 1.1: Význam používaných piktogramů

1.4 Autoři

Na tvorbě textu této studijní opory se podíleli následující autoři:

- T. Kašpárek – kapitola souborových systémů,
- R. Kočí – kapitola zaměřená na procesy a plánování,
- P. Peringer – správa paměti, virtuální paměť a
- T. Vojnar – kapitola zaměřená na synchronizaci procesů a koordinace a finální sestavení opory.

Kapitola 2

Správa souborů



2:00



Cílem této kapitoly je diskuse problematiky správy souborů a ukládání dat z hlediska operačního systému. V první polovině jsou popsány principy organizace dat do souborů, základní informace o fyzickém uložení dat na discích, algoritmy pro správu datových bloků a některé aspekty ovlivňující výkon souborových systémů. V druhé části je pak přiblížena implementace souborových systémů spolu s několika konkrétními ukázkami.

2.1 Základní principy

Při potřebě ukládat v rámci aplikací data perzistentně – tedy aby byla trvale k dispozici i po skončení běhu aplikace která je vytvořila, vytvořily již první operační systémy koncept souborů a jejich uložení na konkrétní médium – tedy souborového systému.

Soubor je základní organizační jednotka pro uchovávání dat na discích či jiných médiích.



Souborový systém je souhrn pravidel definujících chování a vlastnosti jednotlivých souborů, možnosti jejich další logické organizace (do adresářů) a následně konkrétní způsob uložení požadovaných dat spolu s dalšími potřebnými informacemi na fyzickém médiu.

Alokační blok je nejmenší jednotka, s kterou souborový systém schopen pracovat. Velikost bloku bývá zespodu limitována použitým zařízením – například pevný disk má velikost sektoru 512B, jinou velikost bloku může mít například pásková nebo ramdisk (1B nebo 1 paměťová stránka dle implementace). Většinou však z důvodu efektivity čtení a zápisu na médium používá souborový systém alokační blok větší, z důvodu cachování často velikost stránky paměti (na architektuře x86 4KB) nebo její násobek (např. 8KB nebo 16KB).

2.1.1 Atributy souboru

Při ukládání dat nestačí ukládat jen samotná uživatelská data, ale požadujeme také uchování dalších důležitých informací:

jméno souboru je jeho základní charakteristikou, která nám umožňuje k souboru opětovně přistupovat. Pravidla pro pojmenování souboru a význam jednotlivých částí se liší v každém operačním systému, základem však bývá pojmenování pomocí řetězce alfanumerických znaků dlouhého alespoň 8

znaků. Většina dnešních operačních (a souborových!) systémů podporuje jména až 255 znaků dlouhá. Některé rozlišují malá a velká písmena (Unix) pro jiné je *data.txt* a *DATA.TXT* stejný soubor. Většina operačních systémů podporuje dvě složky pro pojmenování souboru – název a příponu. Přípona je od názvu oddělena tečkou a většinou může být kumulativní (např. *main.c.gz*). Některé OS používají příponu k jednoznačnému určení typu souboru (např. MS Windows), jiné k tomuto účelu využívají více charakteristických prvků (přípona, prvních N bajtů souboru atd.) (např. Unix). V některých systémech může mít jeden soubor více jmen (např. Unix)

velikost souboru je počet bajtů, uložených v souboru. Je vždy menší než skutečný prostor, který je pro uložení souboru potřebný na disku (ukládáme také atributy souboru, dochází k fragmentaci atd.).

časové informace souboru většinou obsahují čas a datum vytvoření a poslední modifikace souboru, někdy se též odlišuje čas modifikace dat souboru od času modifikace ostatních ukládaných informací o souboru.

identifikace vlastníka souboru, případně skupina souboru je údaj důležitý v multiuživatelských systémech, kde je nutné provádět jednak kontrolu obsazení dostupného místa na discích a pak také kontrolu oprávnění přístupu k souborům.

oprávnění přístupu k souboru definují seznam osob (identifikátorů osob) s oprávněním číst, zapisovat či provádět (spouštět) daný soubor.

umístění souboru je určeno jeho příslušností ke konkrétnímu adresáři.

Metadata je souhrn informací které požadujeme nebo z hlediska implementace musíme uložit na disk spolu s vlastním obsahem souboru. Jedná se tedy jak o výše uvedené atributy, které požaduje uložit uživatel, tak například o informace pro log žurnálovacích souborových systémů.

DEF

2.1.2 Operace nad souborem

Z uživatelského hlediska je soubor definován seznamem operací, které lze nad ním provádět. Přestože každý operační (a někdy i souborový) systém má vlastní množinu podporovaných operací, některé jsou pro všechny soubory a systémy společné:

vytvoření souboru – vytvoření souboru se skládá ze dvou kroků. Jednak je nutné alokovat prostor pro nově vytvářený soubor (i pokud zatím neobsahuje žádná uživatelská data, je nutno uložit alespoň jeho atributy) a aby bylo možné se souborem opětovně pracovat, musíme vytvořit položku s jeho jménem v adresáři, do kterého soubor patří. Někdy využíváme toho, že potřebujeme vytvořit soubor s exkluzivním přístupem pouze pro aktuální běžící program (např. dočasné mezivýsledky). V takovém případě se využívá triku, kdy se soubor vytvoří otevře pro zápis/čtení a ihned smaže.

Tím, že je otevřen, zůstává nadále fyzicky na disku a dostupný pro běžící program, ale vzhledem k neexistenci záznamu v adresáři nikdo jiný není schopen soubor pojmenovat a tudíž s ním jakkoliv pracovat.

čtení dat – při čtení dat ze souboru mohou nastat dva základní případy. Pokud data již nějaký (i jiný) proces požadoval, mohou již být obsažena ve vyrovnávací paměti operačního systému. V tom případě jsou zkopírována do oblasti paměti požadované procesem při čtení. Pokud požadovaná část souboru není ve vyrovnávací paměti nalezena, proběhne vlastní fyzické čtení z disku a data jsou nejprve umístěna do vyrovnávací paměti a odtud opět zkopírována jako v prvním případě.

zápis dat – při zápisu dat se opět využije vyrovnávací paměti. To ale znamená, že data nemusí být fyzicky zapsána na disk. Zápis na disk se provádí jednak periodicky po určeném časovém intervalu (řádově jednotky sekund), v případě nedostatku paměti po jiné účely (část vyrovnávací paměti s daty zapsanými na disk se v tom případě uvolní pro jiné potřeby) nebo pokud o to uživatel výslovně požádá při zápisu. Důležité je si uvědomit, že operace zápisu je již z principu výrazně pomalejší než operace čtení. To je dáno tím, že při zápisu se modifikují data již na disku obsažená a proto je potřeba je nejprve z disku načíst, provést modifikaci oblasti, kterou požadoval uživatel a poté teprve na disk opět zapsat. Vyrovnávací paměť dokáže tento jev sice značně eliminovat, při jejím nedostatku se však může projevit velmi výrazně.

změna pozice v souboru – vzhledem k tomu, že velmi často se se souborem pracuje sekvenčně (tj. postupně se čte/zapisuje od začátku do konce), bylo by neefektivní udávat pozici pro čtení/zápis při každé této operaci. Proto většinou existuje samostatná funkce pro změnu pozice v souboru. Tato operace pouze nastavuje interní datové struktury operačního systému a neprovádí žádnou operaci nad fyzickým médiem.

smazání souboru je opět jako vytvoření operace skládající se ze dvou kroků. Nejprve je nutné zrušit záznam v adresáři, kde byl soubor umístěn a až poté se mohou uvolnit datové bloky patřící k souboru pro pozdější využití.

Zkuste se zamyslet, co by se mohlo stát, kdyby se při mazání souboru (nebo jeho zkracování) nejprve uvolnily datové bloky a potom teprve provedla úprava informací v adresáři.



2.1.3 Typy souborů

V některých operačních systémech mohou existovat soubory více typů s odlišným chováním. U běžných uživatelských souborů můžeme rozlišovat více druhů přístupu k souboru. Ty mohou být dány operačním systémem nebo typem fyzického zařízení s kterým se pracuje.

Sekvenční přístup – tento přístup je charakteristický například pro páskové mechaniky. Zde je možné zapisovat/číst pouze následující blok dat. Pokud dojdeme na konec pásky, provede se převinutí zcela na začátek. Druhá možnost, kde se využívá sekvenční přístup je zařízení, které pracuje s bloky nestejně velikosti. Za takových podmínek není možné určit začátek libovolně vybraného bloku a záznamy se mohou procházet pouze sekvenčně.

Přímý přístup – pokud jsou velikosti zpracovávaných záznamů stejné, je možné vybrat v každém okamžiku libovolný blok. Běžně pracujeme se soubory s velikostí záznamu 1byte. Při přístupu k pevnému disku se pak používá jednotka 1 sektor (tj. 512B). Za pomoci přímého přístupu je možné jednoduše emulovat sekvenční přístup, naopak je to sice možné také, ale je to enormně neefektivní.

Indexový přístup – je efektivní kombinací obou zmíněných přístupů, používá se především u databází. Vlastní záznamy jsou uloženy v blocích s proměnnou délkou. Aby bylo možné k nim efektivně přistupovat, je jako pomocná struktura vybudován index, tedy seznam dvojic klíč a adresa začátku bloku s vlastními daty. Velikost klíče je pevná a v rámci indexu může tedy fungovat přímý přístup. Problémem je pak změna velikosti bloku s daty a tím vznikající fragmentace úložného prostoru.

Dále některé systémy (například MS DOS) rozeznávají soubory textové a binární. U textových souborů interpretujeme jednotlivé bajty (nebo skupiny bajtů u vícebajtových kódování jako UTF16) jako znaky v určitém kódování (příkladem je ASCII, ISO 8859-2, Windows 1250 nebo již zmíněné UTF16). Navíc zde rozeznáváme jednotlivé řádky textu což jsou sekvence znaků oddělené speciálně pro to určeným znakem například 0x0A (CR – carriage return), 0x0D (LF – line feed) nebo více znaky (jako MS DOS kde se používá sekvence dvou znaků – CR LF).

Kromě výše uvedeného například Unix rozumí pod typem souboru ještě něco úplně jiného. Je to dáno především tím, že jedna ze základních myšlenek, na kterých je Unix postaven, je že téměř vše v operačním systému se tváří jako soubor. Pak rozeznáváme (kromě uživatelských souborů) následující typy souborů:

blokový speciální soubor – zpřístupňuje ovladač blokového zařízení Tyto zařízení jsou schopna komunikovat po blocích o velikosti větší než 1B. Typickým příkladem je pevný disk s velikostí bloku 1 sektor (512B). Blokové zařízení bývají většinou vybavena vyrovnávací pamětí.

znakový speciální soubor – zpřístupňuje ovladač znakového zařízení. Tyto zařízení jsou schopna komunikovat po jednotlivých bajtech. Typickým příkladem je ovladač sériového portu nebo ovladač zvukové karty.

unixový soket – tento speciální soubor umožňuje lokální komunikaci pomocí rozhraní pro síťové sokety. Bývá tak používáno například pro serverové aplikace, kde požadujeme zaručený přístup pouze z lokálního počítače (např. databáze používaná pouze pro lokální web server).

pojmenovaná roura – perzistentní obdoba běžné roury (pipe), která umožňuje komunikaci více procesům. V běžném použití jeden proces data do roury zapisuje a jeden čte, obecně může být čtecích i zapisovacích procesů více.

symbolický odkaz – jedná se o speciální soubor podobně jako například adresář, který ve své oblasti určené pro uživatelská data nese jméno libovolného jiného souboru (i neexistujícího, obecně tedy libovolný text). Pokud symbolický odkaz odkazuje na existující soubor, je při většině operací tato operace provedena na souboru, na který odkaz ukazuje. Výjimkou jsou operace smazání a konkrétní verze operace zjištění informací o souboru – *lsstat*. Na rozdíl od pevného odkazu (hard link) může ukazovat i mimo hranice souborového systému, kde leží.

2.1.4 Řízení přístupu k souborům

Pokud k datovým či speciálním souborům na daném počítači může přistupovat více osob, je nutné nějakým způsobem řešit ochranu přístupu, tedy možnost určit, kdo má právo s daným souborem pracovat a jakým způsobem. Tyto oprávnění potom hlídá operační systém a při libovolné operaci se souborem jsou nejprve vyhodnocena všechna přístupová práva a teprve pokud uživatel má oprávnění danou akci provést, je akce vykonána.

Unixové systémy používají dva způsoby řízení přístupu. Klasický používá 3 sady oprávnění. Pro vlastníka souboru, pro skupinu souboru a pro ostatní nespádající do obou kategorií. Každá sada oprávnění se skládá ze tří základních práv a případně několika rozšiřujících:

- r** – právo čtení – pro běžný soubor znamená povolení operace otevření souboru v režimu čtení. Pro adresář pak umožní přístup k položkám adresáře (soubory a podadresáře) pokud známe jejich název.
- w** – právo zápisu – pro běžný soubor znamená povolení operace otevření souboru pro zápis. Pro adresář právo modifikovat jeho obsah, tedy vytvářet, přejmenovávat a rušit položky v adresáři.
- x** – právo spustit daný soubor. Pro programy jde o vytvoření procesu, který vykonává kód v programu definovaný, nicméně systémy Unix umožňují definovat operaci spuštění pro libovolný typ souboru (například programy systému MS DOS *.com* a *.exe* pomocí emulátoru). Kromě specializovaných postupů pro spouštění existuje speciální konvence pro programy v interpretovaných jazycích. Pokud soubor začíná na prvním řádku sekvencí

`#!/absolutní/cesta/interpret parametry`

Je zavolán zde uvedený program a daný skript je mu předložen k provedení. Typickým příkladem jsou například skripty příkazového interpretu *bash* nebo jazyků jako *perl* či *python*.

Pro adresáře znamená oprávnění pro spuštění operaci procházení celého obsahu adresáře – tedy výpis jeho obsahu (tj. např *ls /directory*).

- t** – tzv. sticky bit – pokud je nastaven u souboru je ignorován, zde měl význam u starších systémů, stále se však používá u adresářů, kde zajišťuje, že smazat soubor může pouze jeho vlastník. To se využívá nejčastěji u adresářů */tmp*, kde může zapisovat každý (právo *w* pro vlastníka, skupinu i ostatní), ale nechceme aby kdokoliv mohl smazat cizí soubory a adresáře.
- s** – tzv. SUID respektive SGID (set user ID / group ID) právo, v tomto případě spíše příznak, určuje, že proces, který vznikne spuštěním souboru s tímto právem bude mít UID respektive GID nastaveno nikoliv podle uživatele/skupiny, která proces vytvořila ale podle vlastníka/skupiny dané tímto spustitelným souborem. Příkladem využití jsou programy pro přístup k citlivým informacím jako je databáze uživatelů. Pro změnu hesla je nutné aby měl i běžný uživatel možnost modifikovat tyto soubory, tím že je nucen k tomu použít konkrétní program, děje se tak kontrolovanou cestou.

Ve výpisu se práva většinou označují jako tři trojice písmen, první tři pozice jsou pro práva vlastníka, druhé tři pozice pro skupinu a třetí tři pro ostatní. Pokud je dané oprávnění nastaveno, je na dané pozici příslušné písmenko, jinak je použita pomlčka:

```
$ ls -l ~joe
drwxr-x--- 47 joe users 4096 Oct 9 18:02 /home/joe
```

Mějme program *passwd* s následujícími právy *-r-s-x-x* a vlastníkem *root*. Soubor se záznamy uživatelů */etc/passwd* pak má oprávnění následující *-rw-r--* a vlastníka opět *root*. Pokud by programu *passwd* chybělo oprávnění *s*, bude moci běžný uživatel nad souborem se záznamy uživatelů provádět pouze operaci čtení (oprávnění *r* pro ostatní). Tím, že je přítomno i oprávnění *s*, je proces *passwd* spuštěn sice se skupinou uživatele, který program spustil ale pod uživatelem který vlastní program *passwd*, tedy *root*. Ten má pak potřebná oprávnění k manipulaci s potřebným souborem.

Kromě symbolického vyjádření přístupových práv se také někdy používá číselná hodnota, která je použita při implementaci uložení těchto hodnot v souborovém systému. V tom případě se jedná o 4 cifry. První je součtem speciálních příznaků: 4 pro SUID, 2 pro SGID a 1 pro sticky bit. Další tři cifry jsou opět pro vlastníka, skupinu a ostatní s následujícím významem. Součet hodnot je zde 4 pro čtení, 2 pro zápis a 1 pro spuštění.

Uvažujme soubor s následujícími právy:

```
d-----rw- joe users 275 Oct 8 2002 /tmp/README
```

Na první pohled to vypadá, že když jsou nastavené práva pro čtení a zápis pro ostatní, tak i Joe jako vlastník může se souborem dělat cokoliv, nicméně vzhledem k tomu, že pro danou osobu se uplatňují nejprve práva pro vlastníka pokud jím je, dále pro skupinu pokud do ní patří a teprve nakonec práva pro ostatní, bude Joovi přístup odepřen. Stejně tak komukoliv, kdo patří do skupiny *users*.

 $x + y$
 $x + y$

Nicméně Joe jako vlastník může změnit práva tohoto souboru (a je jediný kromě uživatele *root*, kdo to může udělat) a tím přístup získat.

Výše uvedené řízení přístupu je sice poměrně flexibilní, nicméně je navrženo s ohledem na běžné modelové situace. Nespornou výhodou je malá a navíc konstantní velikost dat nutných k jeho uložení v souborovém systému – jako číslo s maximální hodnotou 7777. Později se ve vývoji souborových systémů ovšem objevil požadavek na metodu zcela obecnou, která by umožňovala každému objektu v souborovém systému nastavit přístupová práva nezávisle pro každého uživatele. Tato metoda se nazývá seznam řízení přístupu – ACL (Access control list). Tento přístup je možné implementovat pomocí dvourozměrné matice, která na jedné ose obsahuje objekty souborového systému a na druhé všechny uživatele. Hodnotou jsou přístupová práva pro daný objekt a uživatele. To by ovšem bylo nesmírně neefektivní, větší část matice by byla zřejmě buď prázdná nebo duplicitní, proto se tato vlastnost implementuje jako seznam atributů souboru nebo adresáře. Moderní souborové systémy umožňují ke každému objektu ukládat libovolné množství metadat – dvojic název atributu, hodnota atribut. Pro ACL je název atributu jméno uživatele a hodnota určuje práva pro tohoto uživatele.

2.1.5 Adresáře

Aby bylo možné spravovat větší množství souborů je nutné mít k dispozici nástroj na jejich sdružování a hierarchickou organizaci. Tímto nástrojem jsou adresáře (directory), v některých systémech nazývaných složky (folder).

Adresář je základní organizační jednotka pro hierarchickou organizaci souborů. Na jednotlivé soubory se odkazujeme pomocí jejich názvu a názvu jim nadřazených adresářů. K oddělení jednotlivých názvů se používá k tomu vymezený znak, nejčastěji lomítko (/) nebo zpětné lomítko (\).

DEF

Adresář, který nemá žádný další nadřazený adresář a je tedy v hierarchii na nejvyšším místě se nazývá **kořenový adresář**. V unixových systémech je takový vždy pouze jeden v rámci celkové hierarchie, pak můžeme ještě rozlišovat kořenový adresář konkrétního souborového systému, kterých je být opět více – jeden pro každý souborový systém viz. níže operace *mount*. V systémech jako například MS DOS, které dělí dostupný prostor na jednotlivé logické disky je jeden na každém tomto disku. Dále rozlišujeme pojmy **aktuální adresář** označovaný speciálním názvem *.* (tečka) a **nadřazený adresář** (tj. obsahující daný adresář) označovaný pomocí *..* (dvě tečky). Tato zpětná vazba nám umožňuje procházet stromovou strukturu adresářů jednoduše pomocí relativních cest.

Sekvence názvů nadřazených adresářů a oddělovačů pro daný soubor se nazývá **cesta**. Rozlišujeme cestu relativní, která určuje pozici souboru vzhledem k adresáři kde se právě nacházíme (aktuálnímu adresáři – current working directory). Ta začíná názvem adresáře. Absolutní cesta začíná vždy kořenovým adresářem tedy v unixových systémech lomítkem (např. */tmp*), v systémech s logickými disky označením disku (např. MS DOS: *C:\TEMP*). Přístup na úrovni souborového systému je již stejný v obou případech, pro relativní cestu pouze nadřazená část operačního systému zřetězí hodnoty aktuálního adresáře a relativní cesty.

Výhoda je tak pouze v možnosti kratšího zápisu označení souboru. Některé systémy dále zobecňují koncept aktuálního adresáře na seznam adresářů, které jsou prohledávány při zadání relativní cesty.

Příklad relativní cesty je s využitím nadřazených adresářů:

```
../../lib/../../share/../../pixmaps/no.xpm
```

$x + y$

Jak vidíte aktuální i nadřazené adresáře jsou běžné adresářové položky, lze je tedy využívat nejen na začátku ale kdekoliv v rámci cesty (i když to jako zde většinou nemá velký praktický smysl)

Důležité je, že pro adresáře používáme jiné rozhraní než pro soubory. O některých jsme se již zmínili při povídání o přístupových právech, kompletní souhrn je pak následující:

vyhledání souboru – (file lookup) – pokud požadujeme práci se souborem zadaným absolutní cestou, je pro zpřístupnění souboru (vyhledání patřičných dat na disku) nutné provést vyhledání první položky v cestě v kořenovém adresáři, v tomto adresáři další položku cesty atd. až po poslední komponentu, kterou je již název souboru. Operace otevření souboru tedy není nikterak triviální, proto se k za účelem zrychlení používají různé hierarchie cache. Jednak cache disku aby se omezilo čtení z disku protože data, která potřebujeme budou pravděpodobně rozmístěna po celé ploše disku a tím bude jejich získání hodně pomalé. Dále existují ve většině systému také přímo cache pro vyhledávání komponent cest, aby nebylo nutné procházení neustále opakovat.

vytvoření souboru – spočívá v alokaci potřebných datových struktur na disku a následném vytvoření položky adresáře pro nový soubor. Z hlediska konzistence souborového systému je nezbytné aby tato operace proběhla atomicky – tedy buď celá úspěšně nebo vůbec (a to i v případě výpadku elektřiny a podobně). Tento požadavek na atomičnost je pak jedním z hlavních důvodů odlišného rozhraní pro adresářové operace. Vytvoření podadresáře funguje stejně, jen typ vytvářených dat je jiný (adresářová položka může být zcela identická).

zrušení souboru – zrušení souboru je inverzní operace k jeho vytvoření, postup je tedy opačný – nejprve zrušení položky v adresáři a následně zrušení struktur pro vlastní soubor. Opět je důležitá atomičnost této operace. Na některých systémech jsou tyto operace ovšem z pohledu operačního systému (ne jednoho konkrétního uživatele) oddělené. To umožňuje zrušit i soubor, který má nějaký proces právě otevřen. Po zrušení je soubor pro všechny ostatní procesy (tedy ty, které jej neměly před zrušením otevřen) nedostupný, nicméně proces který soubor otevřený měl s ním může dál normálně pracovat. V okamžiku zrušení se totiž zruší adresářová položka, ale ne data souboru. Ty jsou zrušena automaticky až všechny procesy daný soubor uzavřou. Tato technika se používá pro jednoznačné dočasné soubory. Proces vytvoří soubor, otevře ho a následně okamžitě smaže. Tím má zaručeno, že nikdo jiný tento soubor nemůže omylem ani vědomě využívat.

přejmenování souboru – operace spočívá ve zrušení adresářové položky a vytvoření nové, opět je ovšem nutno zajistit správnost operace její atomicitou.

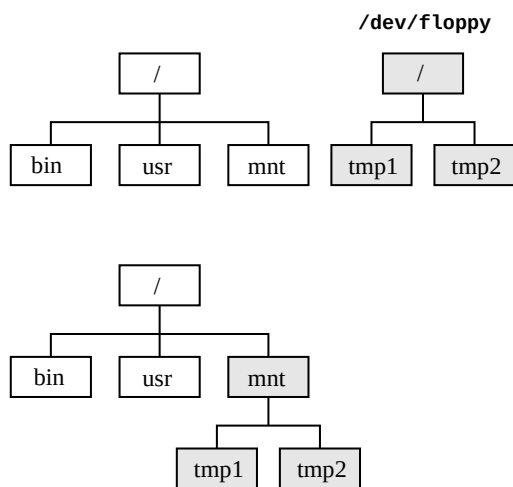
výpis obsahu adresáře – je operace doplňující vyhledání souboru. Toto rozhraní nám umožňuje otevřít daný adresář a načítat postupně všechny jeho položky – jména obsažených souborů a podadresářů. Jakmile máme k dispozici jména pomocí operace vyhledání souboru jsme schopni získat zbývající atributy souboru. Někdy bývá k dispozici i rekurzivní podoba této operace, tedy procházení celého adresářového stromu s vyvoláním uživatelsky definované funkce pro každý nalezený soubor.

Při implementaci adresářů se na některých systémech používá speciálních datových struktur na úrovni souborového systému, většinou se však jedná o speciální soubor, s nastaveným typem na adresář, díky kterému operační systém ví, že má použít jiné zpracování než u uživatelských souborů (tedy výše popsané operace místo souborových). Při implementaci pomocí souboru takový soubor poté obsahuje dvojice název souboru a odkaz na data souboru – v unixových systémech číslo tzv. *i-uzlu* viz. 2.6.

2.1.6 Připojování adresářů – operace *mount*

Systémy jako Unix jsou založeny na konceptu jednoho stromu souborů s jedním kořenovým adresářem (root directory). V těchto systémech i přesto potřebujeme s pomocí tohoto jednoho stromu přistupovat na více disků, diskových oddílů (partition) nebo síťových disků. Tento rozpor se řeší zavedením operace *mount*. Ta umožňuje kořenový adresář určitého souborového systému umístit namísto obsahu zvoleného adresáře. Původní obsah je dočasně skryt a nepřístupný – až do provedení odpojení nového souborového systému inverzní operací *umount*.

Příklad rozvržení adresářů před a po operaci *mount*:



Obrázek 2.1: Příklad připojení adresáře

Při připojení souborového systému musíme zadat nejprve zařízení (oblast disku, disketu atd.) nebo název síťového serveru a jeho sdílený svazek a poté lokální adresář, do kterého má být požadovaný obsah namapován. Adresář do kterého připojujeme jiný souborový systém se nazývá **přípojný bod (mount point)**. V některých systémech je nutno nové systémy připojovat ručně nebo s pomocí speciálního systémového programu (Unix), v jiných jsou zařízení připojována automaticky (MacOS, MS Windows). K určení zařízení, které má být připojeno se využívají právě speciální soubory o nichž byla řeč dříve. V Linuxu tak například první diskový oddíl SATA disku připojíme pomocí:

```
mount /dev/sda1 /mnt/sata-disk
```

Je důležité si uvědomit, že umístění speciálních souborů v adresáři */dev* je pouze konvence, a speciální soubor je možné vytvořit na libovolném místě adresářové hierarchie pokud to konkrétní konfigurace a souborový systém umožňuje.



Typická struktura adresářů v Unixu

V následujícím přehledu jsou uvedeny nejdůležitější adresáře, které se nacházejí na většině OS Unixového typu.

- /bin** – základní nesystémové programy nutné pro start systému jako shell *bash*, *mount*, *grep*, *cat*, *cp*, *echo*. Bývá vždy přítomen na kořenovém svazku (tj. část disku připojená jako */*).
- /boot** – zde (nebo v obdobně nazvaném adresáři) jsou obrazy jádra systému, případně další (konfigurační) soubory související se zavedením jádra.
- /dev** – obsahuje speciální soubory pro znakové a blokové zařízení, je implementován buď v rámci souborového systému kořenového svazku, nebo jako pseudo-souborový systém (tj. s obsahem generovaným na základě požadavku bez patřičného úložného zařízení jako disk).
- /etc** – konfigurační soubory jak systému samotného tak jednotlivých aplikací, dále startovací a systémové skripty někdy i binární nástroje (především ve starších a komerčních verzích Unixu)
- /home** – domovské adresáře uživatelů. Na větších systémech bývá umístěn na samostatném disku nebo připojen pomocí síťového souborového systému (CIFS, NFS)
- /lib** – základní systémové knihovny používané programy v */bin* a */sbin*. Dále například moduly jádra operačního systému nebo binární obrazy firmware, který při startu vyžaduje nahrát některý HW (jako WiFi adaptéry apod.)
- /lost+found** – používá se při obnově souborového systému při havárii. Má význam především u starších nežurnálovaných souborových systémů. Pokud je při kontrole souborového systému objeven soubor u něhož je ztracena informace o adresáři kam patří a s tím většinou i jeho jméno, je umístěn do tohoto adresáře. Jako nové jméno je použito číslo i-uzlu tohoto souboru.

/mnt – zde jsou připojeny ostatní souborové systémy například se síťových souborových serverů. Původně se sem připojovaly i vyměnitelná média, dnes často tuto funkci zastává adresář */media*.

/proc – další virtuální souborový systém. Data v něm obsažená (tedy soubory v něm obsažené i obsahy jeho podadresářů) jsou generovány při pokusu o přístup k nim. Pokud tedy necháme například zobrazit obsah tohoto adresáře pomocí příkazu *ls -l /proc*, budou jednotlivé položky které příkaz *ls* obdrží generovány v daném okamžiku jádrem operačního systému. Tento adresář obsahuje informace o běžících procesech. Každý proces představuje adresář pojmenovaný podle PID daného procesu. V tomto adresáři jsou pak informace o otevřených souborech, obsazení paměti a jiné statistiky. Na některých systémech jsou v */proc* dostupné také další obecné informace o systému */proc/cpuinfo* – informace o procesoru či je možné dokonce zápisem do některých souborů nastavovat jeho chování (například nastavení správy paměti, plánovače procesů či sítě). Více např. [10].

/root – domovský adresář superuživatele *root*. Není jako ostatní umístěn v */home*, protože i na systému, kde jsou ostatní domovské adresáře připojeny ze síťového serveru je často domovský adresář superuživatele pouze lokální.

/sbin – podobně jako */bin* obsahuje programy nutné pro spuštění a základní běh systému. Zde jsou ovšem programy, které vyžadují speciální privilegia, nejčastěji je tedy spouští pouze superuživatel. Příkladem jsou nástroje pro práci s moduly jádra, programy na vytváření, rekonstrukci a správu souborových systému či nástroje pro nastavení sítě.

/tmp – slouží pro dočasné odkládání souborů. Je to klasický příklad využití sticky bitu. Pokud mají soubory přetrvávat delší dobu, je vhodné použít adresář */var/tmp*, u obou dochází většinou k automatickému promazávání starých souborů, zde mají objekty ovšem typicky menší trvanlivost (kolem 10 dnů).

/usr – obsahuje vlastní uživatelské programy a jejich data. Z důležitých podadresářů jmenujme alespoň */usr/bin*, */usr/sbin* a */usr/lib* – obdoba již zmíněných, obsahuje však uživatelské programy nutné až za běhu systému nikoliv při startu (*/usr* opět bývá často na jiném diskovém oddíle než kořenový adresář a není tedy v době startu k dispozici), */usr/X11* – grafické rozhraní pro systémy Unix, */usr/include* – hlavičkové soubory pro programování v jazyce C, */usr/man*, */usr/info* a */usr/doc* – dokumentace v různých formátech, */usr/share* – datové soubory pro aplikace jako bitmapy, zvuky a podobně, */usr/local* – aplikace doplněné správcem do základní instalace operačního systému.

/var – zde jsou ukládány často se měnící informace. Například záznamy systémového logu */var/log*, zámky pro aplikace či servery */var/lock*, informace o běžících službách */var/run*, příchozí pošta */var/mail* atd. sto bývá kvůli požadavkům na místo a především výkon umístěn opět na samostatném disku.

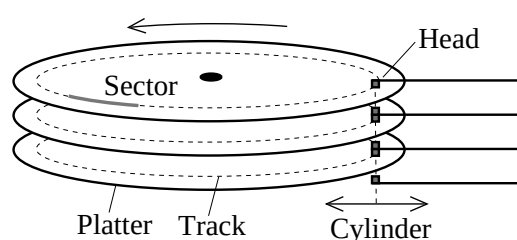


2.2 Uložení dat na pevných discích

Pokud chceme porozumět tomu jak pracují souborové systémy a proč jsou prováděny některé techniky, je vhodné vědět jak vypadá organizace dat na pevných discích i hardware pevných disků.

2.2.1 Hardware pevného disku

Hardware pevného disku pro naše potřeby znázorňuje následující obrázek:



Obrázek 2.2: Schéma hardware pevného disku

Základními prvky pevného disku jsou čtecí a zápisové hlavy (head) a dále vlastní datové plotny (platter). U ploten rozlišujeme stopu (track), což je kruhová oblast, přes kterou se pohybuje záznamová hlava při otáčení ploten. Soubor nad sebou ležících stop se nazývá cylindr (cylinder). Stopa je tvořena jednotlivými sektory což jsou nejmenší oblasti, s kterými je disk schopen pracovat, jejich velikost je dnes 512 bajtů (začínají se ozývat hlasy po jejím zvětšení, což je důsledek narůstající kapacity dnešních pevných disků).

Z těchto pojmů vychází i způsob adresování jednotlivých sektorů na disku. Starší přístup (zavedený v důsledku neschopnosti prvních řadičů disků pracovat s většími čísly) spočíval v udání čísla cylindru, hlavy (tím tedy plotny) a sektoru v rámci stopy. Vzhledem k limitům na velikost jednotlivých položek začalo být toto adresování velmi brzo limitem (jeden z prvních limitů byl pro disky 540MB). Řešením se na čas stalo přepočítávání na tzv. logické souřadnice. U disků se pak udávala charakteristika logická (jak se má k disku přistupovat) a fyzická (jak je disk skutečně konstruován). Překlad prováděl buď operační systém nebo řadič disku. Moderní disky pak již nejsou limitovány jednoduchým HW a proto se přešlo na adresování lineární – tj. disk adresujeme pouze číslem sektoru a nezajímá nás vnitřní uspořádání. Toto adresování se nazývá LBA (linear block address). Výhodou je, že disk může takto v případě potřeby změnit umístění sektoru s konkrétní adresou (např. se původní sektor stane chybovým a nečitelným).

Pro připojení disků dnes používáme řadu rozhraní jak navržených speciálně pro tyto účely (ATA/IDE, SerialATA, SCSI/SAS, FibreChannel), tak rozhraní určená pro připojení periferních zařízení obecně (USB, FireWire).

Pro ukládání dat v počítači vzniká hierarchie pamětí, některé trvalé (perzistentní), jiné pouze dočasné (volatile), které se liší především rychlostí přenosu dat, přístupovou dobou, dostupnou kapacitou a cenou za 1GB. Obecně platí, že

čím rychlejší zařízení, tím je dostupnější v menší kapacitě a tato kapacita je dražší. Hierarchie začíná na registrech procesoru a pokračuje přes procesorové cache (L1,L2,L3), dále paměť RAM až po perzistentní zařízení typu flash, pevný disk, páska.

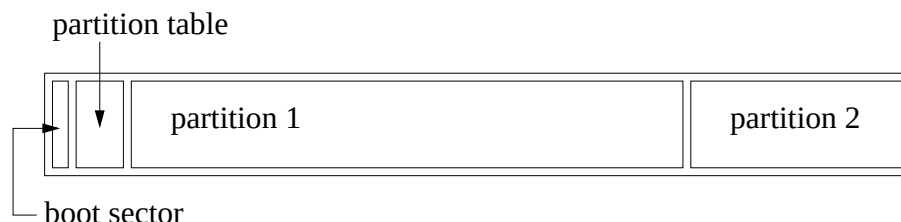
	Registry	L1 cache	L2 cache	RAM	disk
Cena cca	10Kč/B	50Kč/kB	2Kč/kB	2Kč/MB	15Kč/GB
Velikost	10-100B	8-32kB	128k-2MB	1-4GB	~300GB
Příst. doba	<1ns	1ns	5ns	0.1 μ s	5-10ms
Přenos. rychl.	100GB/s	64GB/s	48GB/s	2-8GB/s	50-90MB/s

Tabulka 2.1: Tabulka hierarchie používaných pamětí

Pro výkon souborových systémů je tedy vhodné jednak pokud možno eliminovat přístupy na řádově pomalejší média (disk) vůbec (využití cache), a pokud je přístup nezbytný, tak alespoň provádět čtení/zápis pokud možno v co největších blocích čímž se eliminuje vliv přístupové doby.

2.2.2 Struktura rozdělení pevného disku

Následující obrázek zachycuje schéma rozdělení pevného disku. Důležitými prvky jsou tzv. *boot sector*, který obsahuje část (max. 512B – velikost 1 sektoru) zavaděče operačních systémů jako ntldr, grub, lilo atd.

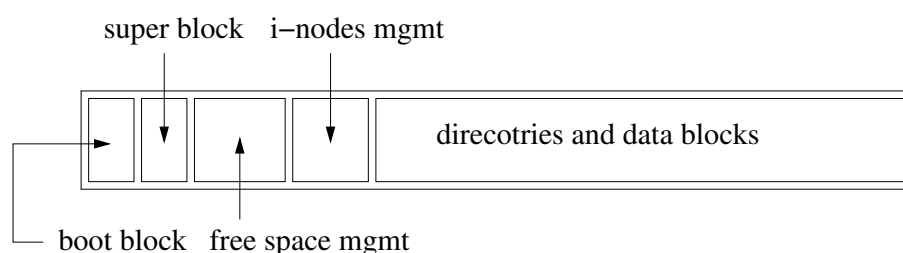


Obrázek 2.3: Schéma rozdělení pevného disku

Další důležitou částí je **tabulka rozdělení disku** (partition table). Na platformě PC může obsahovat 4 položky. Každý záznam je určen pro jeden diskový oddíl. Obsahuje vždy začátek a konec diskového oddílu, typ diskového oddílu a příznaky – především příznak zda je daný oddíl startovací (boot flag), tedy zda je určen k zavedení operačního systému. Pokud je nutné mít diskových oddílů více, musí jeden ze 4 primárních oddílů být typu rozšířený (extended), rozšířený oddíl obsahuje na svém začátku další tabulku rozdělení opět pro 4 položky.

Vlastní diskový oddíl, jenž obsahuje souborový systém, potom může vypadat přibližně tak, jak je popsáno níže.

Tak jako u celého disku, i zde se nám opakuje boot sektor. To umožňuje pro každý operační systém (tj. každý diskový oddíl) použít jiný zavaděč (používá se termín sekundární zavaděč). Dále každý souborový systém obsahuje oblast s základními informacemi o daném souborovém systému tzv. **superblok**. Ten



Obrázek 2.4: Schéma rozdělení diskového oddílu

v sobě uchovává především informaci o typu souborového systému (aby při operaci *mount* bylo možné určit jaký ovladač souborového systému se má použít), dále velikost daného souborového systému, počet obsazených a volných datových bloků, počet volných a obsazených i-node atd. Dále následuje část pro uložení informací o použitých i-uzlech, část pro uložení informace o volném místě a nakonec část, kde jsou uloženy vlastní data (i-uzly a datové bloky). Z důvodu výkonu (menší pohyb diskových hlaviček) jsou poslední 3 uvedené položky rozděleny na více částí a rozprostřeny po celém disku. Zároveň se spolu s nimi většinou ukládá i kopie superbloku, protože při jeho ztrátě je celý souborový systém zničen (respektive chybí základní informace pro práci s ním tedy jej nelze připojit a tudíž použít).

2.2.3 Pokročilé diskové systémy (RAID, LVM)

U serverů a pracovních stanic si často nevystačíme s prostým připojením disků jako u kancelářského počítače. Narazíme na problém nízkého výkonu, nedostatečné spolehlivosti nebo špatné přizpůsobitelnosti novým podmínkám. Pro tyto účely se nasazují pokročilé diskové subsystémy jako RAID nebo LVM.

RAID

Zkratka znamená **Redundant Array of Independent** (nebo **Inexpensive**) **Discs**, tedy redundantní pole nezávislých (nebo levných) disků. Jedná se nástroj jak z více fyzických disků vytvořit jeden logický, který je v některých ohledech lepší. U polí RAID se předpokládá použití disků stejných kapacit (pro maximální výkon stejných modelů stejného výrobce, z hlediska spolehlivosti nejlépe z různých výrobních sérií a zásilek místnímu dodavateli). Získaná vlastnost závisí na typu (v angličtině se používá termín *level* – úroveň) RAID pole:

RAID0 (stripping) – bloky dat jsou rozděleny mezi jednotlivé disky tak že blok 0 je na disku 0, blok 1 na disku 1 až blok N na disku N, blok N+1 je opět na disku 0 atd. Minimální počet disků je tedy 2. Zvyšuje poměrně rapidně výkon subsystému – maximální datový tok (sustained) je většinou téměř N-násobek datového toku jednoho disku. Kapacita je N krát kapacita jednoho disku (nejmenšího v případě rozdílných velikostí). Velikost bloku dat je

možné většinou určit pohybuje se v řádech desítek KB (typicky 16KB – 64KB – 128KB).

RAID1 (mirror) – bloky dat jsou zopakovány na všech discích. Každý blok je tedy uložen N-krát. Tím dostáváme redundanci N-1 disků a tedy výrazně zvyšujeme spolehlivost. Zápis zůstává zhruba stejně rychlý (o něco pomalejší kvůli režii), čtení je naopak rychlejší než jeden disk, protože je možné číst najednou ze všech (funkčních a tedy synchronizovaných) disků podobně jako u RAID0. Používá se většinou v konfiguraci se dvěma disky, výjimečně se třemi. Velikost bloku dat je možné většinou určit pohybuje se v řádech desítek KB (typicky 16KB – 64KB – 128KB).

RAID2 každý blok dat je rozdělen po bitech mezi více disků. Z dat je počítán ECC kontrolní součet, systém je odolný pro případ výpadku více disku. V praxi se příliš nepoužívá, implementace jsou pomalé a potřebuje více disků navíc pro redundanci.

RAID3 Podobné jako RAID2, využívá však informaci od řadiče disků o tom, které sektory byly přečteny správně a které jsou vadné. Proto umí kromě detekce chyby i opravu. Navíc na rozdíl od RAID3 potřebuje navíc jen jeden disk, z tohoto důvodu se RAID2 nepoužívá a v případě potřeby se využije přímo RAID3. Vzhledem k tomu, že v každé operaci je zapojeno více disků, přináší větší maximální datový tok (téměř jako RAID0), ale maximální počet operací za jednotku času se nezvýší, je stejné jako pro jeden disk.

RAID4 Podobné jako RAID0, používají se větší kusy bloků, k nim se počítá parita, která je ukládána na jeden z disků parita na jednom z disků, to představuje úzké hrdlo, proto se v praxi opět příliš nepoužívá.

RAID5 pro zajištění odolnosti vůči výpadku jednoho disku používá paritu, ta je však stejně jako data rozdělena na více disků. Při N discích je jeden blok dat (stripe) rozdělen na N-1 stejných částí, z nich je spočítána parita (o stejné velikosti jako každý z bloků a následně jsou bloky uloženy postupně na všech N disků. Paritní data se ukládají střídavě na všechny disky, tudíž odpadá úzké hrdlo. Proto je systém kromě zvýšené dostupnosti také rychlejší – především pro čtení, kde funguje podobně jako RAID0 s N-+ disky (parita se nečte, nebo zahazuje). Při zápisu je nutné načíst blok dat (stripe), provést jeho modifikaci v části kterou uživatel změnil, spočítat novou paritu a zapsat zpátky na disky. Proto je zápisová operace výrazně pomalejší než u RAID0 i přes případnou podporu v HW. Minimální počet disků je 3, u většího množství disků (nad cca 6-8) narůstá riziko poškození více než jednoho disku nad únosnou mez). Kapacita je N-1 krát velikost disku.

RAID6 je vylepšením RAID5 přidáním dalšího paritního disku. Operace čtení má výkon obdobný jako N-2 velký RAID0, zápis je opět o něco málo pomalejší než RAID5 (ještě složitější počítání parity), rozdíl však již není

zásadní. Minimální konfigurace jsou 4 disky, únosné maximum se oproti RAID5 zvedne na cca 12-16 disků.

JBOD v tomto režimu (Just a Bunch Of Discs) jsou do operačního systému podstoupeny jednotlivé disky tak jak jsou připojeny k adaptéru. Systém pak nad nimi provádí další abstrakci pomocí SW. Příkladem využití je buď dále zmiňované LVM nebo pomocí CPU počítaný a ovladač RAID (tzv. SW RAID).

linear bývá nabízen většinou jen pro SW RAID (nebo LVM), nikoliv HW adaptéry. Opět jsou jednotlivé bloky dat rozděleny na všechny dostupné disky, ovšem bez prokládání, tedy nejprve se obsadí celý první disk bloky 0 až X, dále druhý disk bloky X+1 až Y atd. Nepřináší tak téměř žádný nárůst výkonu (pokud neuvažujeme současné čtení více procesy z různých částí logického disku) ani zabezpečení, ale v případě výpadku jednoho z disků je jednodušší zbývajících data obnovit.

U výkonu diskových polí (kde se můžeme pohybovat až v jednotkách GB/s je nutné zajistit dostatečnou propustnost cest až do místa zpracování dat – tedy PCI-X/PCIE sběrnice do jižního můstku chipsetu, propojení mezi chipsety, rozhraní (FSB, HT) procesoru, napojení pamětí. S nástupem PCIE sběrnice je možné tuto podmínku do jisté míry zajistit i na běžných kancelářských počítačích.



LVM

Zkratka znamená Logical volume manager a jedná se o softwarovou vrstvu, vytvářející logické disky z dostupných fyzických. U LVM jsou dostupné disky rozděleny na alokační bloky tzv. PE (physical extents) o velikosti řádově jednotky MB. Tyto PE jsou zpřístupněny do skupiny svazků tzv. VG (volume group). Ta je zásobníkem dostupného fyzického prostoru právě v podobě PE. Zařízení pro souborový systém vytvoříme v podobě logického svazku LV (logical volume). V rámci jeho konfigurace určíme velikost (kolik PE se pro něj má vyhradit) a způsob alokace alokačních bloků PE. Ta bývá jedním ze způsobů popsaných u technologie RAID. Pokud nám jde jen o flexibilitu správy diskového prostoru volíme často typ *linear*, jinak se používají režimy RAID0, 1 a 5.

Hlavní výhoda systému ale spočívá v tom, že díky abstraktní vrstvě, kterou jsme do systému vložili, máme možnost manipulace s fyzickými disky. Je možné do systému (za chodu) další disky přidat, přiřadit je do VG a LVM je začne automaticky využívat dle zvolené politiky, nebo můžeme existující využívaný disk odebrat z LVM (využité PE jsou přesunuty na jiné dostupné disky) a poté odebrat ze systému (a např. nahradit novějším rychlejším nebo větším). Systém LVM se často používá u velkých serverů, kde je nutné zajistit možnost zásahů do diskového systému bez výpadku (restartu) serveru.

2.3 Algoritmy správy volného a obsazeného místa

Pokud chceme ukládat data o rozličné velikosti navíc s tím, že jejich velikost se může dále měnit, potřebujeme nástroj pro efektivní správu alokovaného (případně volného nebo obojí) místa na disku. Kromě toho je většinou nutné sledovat jak obsazené místo datovými bloky tak místo obsazené metadaty. V případě žurnálovacích souborových systémů může ještě přibýt požadavek na správu využitých oblastí žurnálu (ta bývá ovšem většinou výrazně jednodušší).

V praxi se používají většinou tři základní metody alokace: souvislá, pomocí seznamu, pomocí indexu případně pomocí různých variant stromů. Většina systémů používá pouze jeden druh, některé umožňují typ alokace vybrat nebo alespoň pomocí parametrů upravit.

2.3.1 Souvislá alokace

Souvislá alokace předpokládá, že každý soubor na disku je uložen v po sobě následujících blocích. Výhodou souvislé alokace je relativně velký výkon, protože proces, který čte sekvenčně soubor čte i sekvenční oblast na disku a minimalizuje se tak režie pohybu diskových hlav. Pro uložení informací o souboru nám stačí offset začátku souboru a jeho velikost, což je také velmi efektivní. I přímý přístup je jednoduchý. K offsetu začátku souboru o pouze přičteme požadovaný offset v rámci souboru i . Hlavním problémem u sekvenční alokace je vyhledávání volného místa pro nové soubory nebo prodloužení stávajících. V případě zrušení souboru nebo jeho zkrácení vznikají potenciálně volné úseky, ale postupně dochází k zhruba rovnoměrnému rozložení souboru a volného místa rovnoměrně po celém disku. Nastává tzv. **externí fragmentace**.

Představme si že na disk ukládáme databázi CD titulů. Každé CD je v samostatném souboru v libovolné adresářové struktuře např. podle interpreta nebo podle roku vydání (pro nás není důležité). Pro jednoduchost předpokládejme že záznamy a tudíž soubory jsou všechny stejně velké. Průběžně budeme některé záznamy mazat nebo upravovat, nové vždy alokovat na konci obsazeného místa na disku. Takto postupně rozdělíme celý disk na části o velikosti jednoho záznamu které budou obsahovat buď některý ze souborů nebo volné místo. Nyní se dostaneme na konec disku a zjistíme, že do nově ukládaných záznamů chceme přidat jako další informaci název vydavatelství. Při pokusu o zapsání souboru dostaneme ovšem chybové hlášení že disk je plný. Smažeme několik nepotřebných souborů ale hlášení zůstává, i přesto, že nástroj na zjištění obsazení disku indikuje řekněme 60% obsazeno. Kde je problém?

Odpovědí je, že máme sice dostatek volných bloků, ale žádný z nich není dostatečně velký pro náš nový větší záznam. V praxi se tento problém řeší setřásáním záznamů k sobě, tak aby dostupné oblasti volného místa byly co největší.

Jediným místem, kde se dnes více používá sekvenční alokace jsou souborové systémy určené pro read-only média jako CD-ROM (CD-R, RW atd), kde jsou dopředu vždy známy velikosti všech souborů a po prvním zápise se již nemění.

$$x + y$$

2.3.2 Alokace pomocí extentů

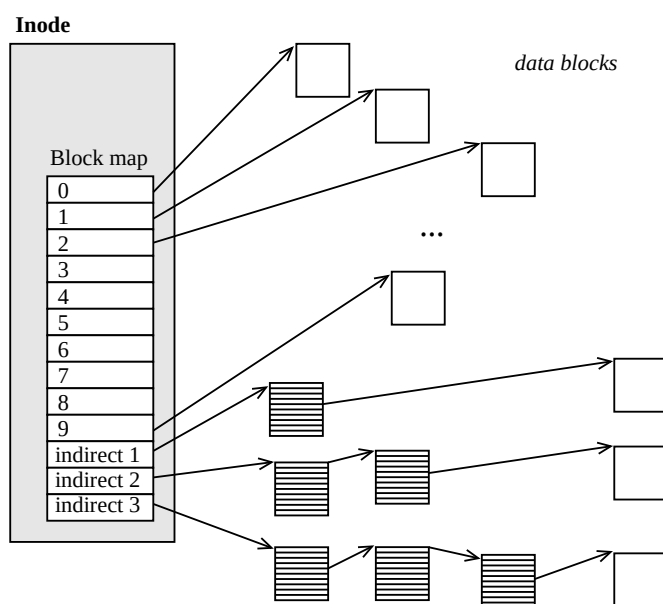
Je rozšířením souvislé alokace. Při vytvoření nového souboru je alokována souvislá sekvence bloků zvaná *extent*. Jakmile soubor překročí velikost jednoho extentu, je na jiném vhodném místě (ideálně samozřejmě hned za předcházejícím) alokován extent další opět jako souvislá sekvence bloků. Pro jeden soubor je tedy nutno udržovat seznam extentů spolu s informací o jejich uložení, ale dosáhneme vysokého výkonu při práci se soubory díky alespoň částečně sekvenční alokaci.

2.3.3 Alokace pomocí seznamů

Alokace pomocí seznamů se snaží řešit problém s externí (resp. interní při prealokaci) fragmentací u sekvenční alokace. Zde je každý soubor ukládán do bloků umístěných libovolně na disku a k nim je uložena informace o posloupnosti bloků ve formě odkazu na další blok v každém bloku. Pro uložení informací o souboru pak potřebujeme odkaz na první a poslední blok (kde začít a přestat číst). Nevýhodou je zde výkon. U sekvenčního přístupu je možné ještě efekt pohybu diskových hlaviček z bloku na blok eliminovat jejich umístěním vedle sebe (defragmentace ze systému MS DOS či MS Windows pro systém souborů FAT16/32). Větší problém ovšem nastává při požadavku na náhodný přístup. Zde jsme nuceni načítat všechny bloky od začátku disku abychom získali odkaz na následující blok i když data nevyužijeme. Přechtení jediného bajtu z konce souboru tak trvá stejně dlouho jako načtení celého souboru. Částečným řešením problému s výkonem je informace o následnících umístit mimo datové bloky tak aby bylo možné tyto oblast držet přímo v paměti RAM. Tato technika se nazývá tabulka pro alokace souborů – FAT (File Allocation Table). Pokud je tato tabulka uložena v několika kopiích na různých místech disku, značně to zvyšuje spolehlivost, neboť ztráta FAT tabulky znamená ztrátu všech souborů, neboť se k nim nejsme schopni dostat.

2.3.4 Alokace pomocí indexů a stromů

Poslední metoda pro každý soubor vytváří datovou strukturu zvanou i-uzel (i-node) (i jako indexovací). tato struktura obsahuje jak všechny atributy souboru, tak adresy všech diskových bloků souboru. Na rozdíl od FAT tabulky, která pro efektivní provoz musí být v paměti RAM celá, zde stačí mít v paměti i-uzly otevřených souborů. Tím dostáváme limit současně otevřených souborů místo limitu počtu souborů v souborovém systému, což je jistě mnohem přijatelnější. Většinou bývá velikost i-uzlu zvolena tak aby její násobek byl roven velikosti alokačního bloku souborového systému nebo použitého zařízení. Od toho se odvíjí počet odkazů na datové bloky, které je možné umístit do i-uzlu. Většinou se jedná o počet cca 8-16. Tím by byla ovšem silně limitována maximální velikost souboru (na 16*4KB tj. 64KB pro 4KB alokační blok). Proto se posledních několik bloků používá pro nepřímé odkazy. Nepřímý odkaz v i-uzlu ukazuje na speciální datový blok, který již obsahuje přímé odkazy na datové bloky souboru. Nepřímé odkazy druhé úrovně obsahují odkazy na datové bloky, které obsahují odkazy na datové bloky, které obsahují teprve odkazy na vlastní datové bloky souboru. Situaci znázorňuje obrázek 2.5.



Obrázek 2.5: Schéma nepřímých odkazů v i-uzlu

Tímto způsobem jsme schopni efektivně zvětšovat maximální velikost souboru v podstatě na libovolnou hodnotu. Pro efektivnost se někdy pro extrémně malé soubory používá druhý extrém, data souboru uložena přímo v i-uzlu.

Systém i-uzlů navíc umožňuje odkazovat se na jeden soubor z více adresářů. Pokud v adresářové položce použijeme stejné číslo i-uzlu, pracujeme automaticky se stejným souborem – všechna další data kromě jména jsou totiž přímo v i-uzlu. Tímto způsobem jsou v Unixu implementovány tzv. hard-linky. V některých systémech je počet i-uzlů určen při vytváření souborového systému (Ext2) a je pro ně alokováno místo, které pokud nepotřebujeme daný počet souborů zůstává nevyužito. V jiných systémech (XFS, JFS) je prostor a tím počet i-uzlů alokovan dynamicky podle potřeby.

2.3.5 Správa volných alokačních bloků

Pro správu volných alokačních bloků můžeme použít další odlišné techniky. To vychází z toho, že volné bloky jsou všechny stejné velikosti a je možné do nich umístit libovolnou informaci (nevyužívá je uživatel, my pro naše potřeby můžeme dokud si o ně uživatel neřekne).

Bitový vektor

Jedna z možností je použít bitový vektor a každý bit nastavit jako příznak zda blok na adrese dané indexem bitu je volný nebo obsazen. Nalezení prvního volného bloku je pak poměrně efektivní neboť dnešní procesory většinou podporují instrukci index prvního (ne)nulového bitu ve vektoru zadaném v registru (až

128bitů na jednu instrukci). Nevýhodou je, že pro větší disky potřebujeme relativně hodně prostoru k uložení vektoru a to jak na disku ale také v paměti RAM.

Vázaný seznam

Druhou metodou je jednosměrně vázaný seznam podobně jako při alokaci datových bloků pomocí seznamu. Nyní můžeme pro uložení odkazu využít přímo volné bloky (tedy nepotřebujeme žádný prostor navíc) a při požadavku na volné místo potřebujeme vždy pouze začátek seznamu, tedy sekvenční přístup. U FAT tabulky je tato struktura využita jak pro seznamy datových bloků tak pro seznamy volných bloků. Každý blok je tak právě v jednom seznamu (buď je volný nebo obsazen nějakým souborem).

2.4 Výkon souborového systému

V této části probereme několik technik, které mají zásadní vliv na výkon souborových systémů. Obecně platí, že většinou můžeme dosáhnout efektivitu uložení dat (tedy malé velikosti režijních dat, která jsou navíc) nebo většího výkonu a je nutno najít mezi těmito dvěma možnostmi kompromis.

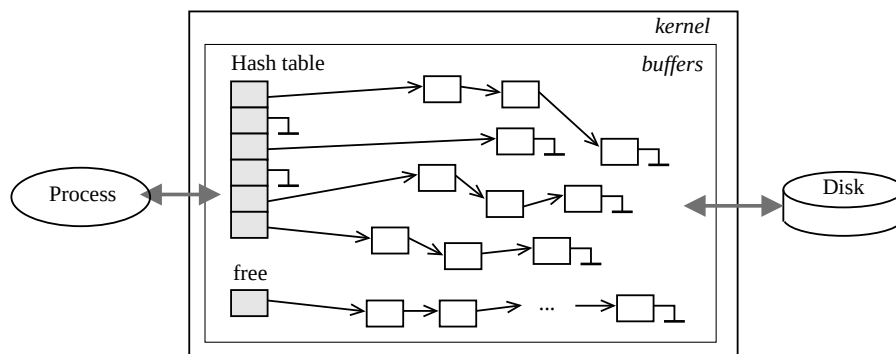
Příkladem může být udržování času posledního přístupu a poslední modifikace souboru. Čas posledního přístupu znamená provádět zápisy na disk i pokud jsou data jen čtena, pokud tuto informaci nepotřebujeme, je možné většinu souborových systémů připojit s parametrem *noatime*, který tuto aktualizaci zakáže a dosáhneme tak lepšího výkonu.

Kromě omezení funkčnosti ovšem existuje několik obecných technik, které pomohou zlepšit výkon

2.4.1 Paměť cache

Základní problém s výkonem diskových subsystémů vyplývá z tabulky 2.1. Pevný disk je o několik řádu pomalejší než další úroveň paměťové hierarchie. Zvýšení výkonu musí tedy směřovat k omezení počtu operací s diskem. Jedním z nejúčinnějších mechanismů je zavedení paměti cache. Ta pracuje tak, že při požadavku na čtení z pomalejšího zařízení (disku) načtou data poprvé z tohoto zařízení a uloží se do rychlejšího (paměť RAM). Při opakovaném požadavku na čtení se již data předají rovnou z rychlejšího zařízení (RAM) bez jakéhokoliv přístupu na disk. Při zápisu se data zapíší opět do bufferu v rychlejším zařízení (RAM) a teprve jednou za delší dobu se provede zápis na pomalejší zařízení (disk). Tím jednak omezíme počet zápisů na disk (změna v RAM mohla za tu dobu proběhnout několikrát) a navíc budeme schopni většinou zapisovat větší blok dat najednou, což dále zlepšuje rychlost zápisu. Paměť cache pro disky bývá organizována a zpracovávána po oblastech odpovídajících alokačním blokům na disku, většinou tato hodnota bývá vybrána jako násobek stránky v paměti. Pro správu paměti cache je zapotřebí mechanismus podobný jako pro alokaci dat, většinou se zde využívá několika seznamů zpřístupněných přes hash tabulku (např. dle adresy požadovaného bloku). Dále musíme být schopni uvolnit vhodné části paměti

cache při nedostatku paměti v systému nebo požadavku na uložení nových bloků z disku do cache.



Obrázek 2.6: Schéma organizace paměti cache

2.4.2 Block Read Ahead

Další technikou je načítání dat dopředu. Pokud proces požádá o data z disku na adrese x , provedeme načtení dat (do paměti cache) i z bloků $x+1$ až $x+n$. Vzhledem k tomu, že pro čtení malého bloku dat, je hlavním parametrem určujícím výkon vystavení hlavy a rotační zpoždění (seek), bude načtení jednoho bloku dat trvat téměř stejně dlouho jako načtení n bloků. Pokud proces bude nadále požadovat další data z adres $x+i$, bude možné jeho požadavky uspokojit z paměti cache a ušetříme několik celých diskových operací. Na druhou stranu, pokud proces provádí náhodné čtení dat, budeme načítat některá data zbytečně. To sice nebude mít velký vliv na výkon disku, ale budeme zaplňovat cache nepoužitými daty a v nejhorším případě zrušíme položky, které by mohly být v budoucnu využity. Proto je velikost bloku načítaného dopředu jedním ze základních parametrů, kterými může uživatel přizpůsobit výkon systému konkrétnímu typu zátěže.

2.4.3 Omezení pohybu diskových hlaviček

Jak jsme si řekli dříve při práci s malými bloky dat je nejdůležitějším parametrem udávajícím rychlost čtení nebo zápisu zpoždění hlaviček disků a případně rotační zpoždění ploten. Pokud se nám tedy podaří tento pohyb omezit, dojde zřejmě k výraznému nárůstu výkonu.

První technikou pro minimalizaci pohybu diskových hlaviček je rozvržení souborového systému na diskovém oddílu, které se snaží umístit části dat a metadat, které budou pravděpodobně potřebné najednou blízko sebe. Základem je tedy sdružovat diskové bloky s informacemi pro správu volného prostoru (pokud tyto informace nejsou stabilně v RAM – např. u bitmapy), bloky pro uložení i-uzlů, bloky pro adresáře a konečně vlastní datové bloky souborů. Rozdělme tedy disk na několik logických částí a do každé umístíme informace dle obrázku 2.3, tedy díl informací pro správu volného místa (spadajícího do této logické části),

díl i-uzlů pro tuto část, díl bloků pro adresáře a díl bloků pro data souborů. Pokud dosáhneme toho, že uživatel bude pracovat se soubory které budou všechny ležet ve stejné logické části, omezíme pohyb hlavičky disku. V praxi se ukazuje, že stačí umístit obsah jednoho adresáře do téže logické části. Pro tuto techniku se používá název skupiny cylindrů (cylinder groups).

2.4.4 Plánování sekvence V/V operací

Pokud požaduje proces vykonat vstupně/výstupní (V/V angl. I/O) operaci, zadá požadavek operačnímu systému. Ten zkontroluje jestli je dané zařízení připraveno a pokud ano zašle požadavek k vyřízení. Pokud je ovšem požadavků více, zařízení může být vytíženo a další požadavky již nepřijímá. V tom případě jsou požadavky řazeny do fronty a z ní jsou vyzvedávány jakmile je zařízení opět připraveno. Nic nám ovšem nebrání vybírat požadavky z fronty místo FIFO v jiném pořadí. Naším cílem bude seřadit požadavky tak, aby celkový pohyb, který budou muset hlavičky disku udělat byl minimální.

FCFS – klasické FIFO

Nejjednodušší plánování požadavků je označováno First Come First Serve – první příchozí požadavek je první obsloužen, jedná se o klasickou frontu. Její nevýhodou je obecně nižší výkon (špatná optimalizace), nicméně výhodu je na druhou stranu spravedlivé vyřizování všech požadavků.

SSTF – nejrychlejší požadavky nejdřív

Tento plánovač (Shortest Seek Time First) vybere z fronty vždy požadavek který je nejbližší předchozímu vyřizovanému (tj. aktuální pozici diskové hlavy, minimální seek). Algoritmu není optimální, oproti FCFS představuje ovšem radikální vylepšení, často až na 30% času vystavování. Má však jednu zásadní nevýhodu. Pokud bude do fronty neustále přicházet dostatek požadavků nacházejících se v jedné oblasti disku, všechny ostatní požadavky zůstanou nevyřízeny. Bude docházet k hladovění (starvation), což je v praxi většinou nepřijatelné.

SCAN algoritmus

Algoritmus který se nazývá SCAN (scanuje povrch disku) postupně posunuje diskovou hlavu od začátku disku ke konci a během cesty je nad každým dostupným cylindrem, z fronty tedy vybírá postupně požadavky adresami tvořícími neklesající sekvenci a postupně je vyřizuje. Jakmile dojde na konec disku změni směr a pokračuje stejně směrem k jeho začátku cestou opět vyřizujíc všechny dostupné požadavky. Existuje ve variantě nazývané C-SCAN (C od circular), která se liší v tom, že jakmile dorazí na konec disku přesune hlavu bez vyřizování požadavků ihned na začátek a začne další kolo. Tím se ještě dále zlepší rovnoměrnost rozložení doby čekání jednotlivých požadavků na vyřízení. V praxi se tyto algoritmy implementují tak, že neprovádí posun hlavy až na konec-záčátek disku ale jen na požadavek s největší/nejmenší adresou (nazýváno někdy LOOP resp. C-LOOP).

2.5 VFS – Virtual Filesystem

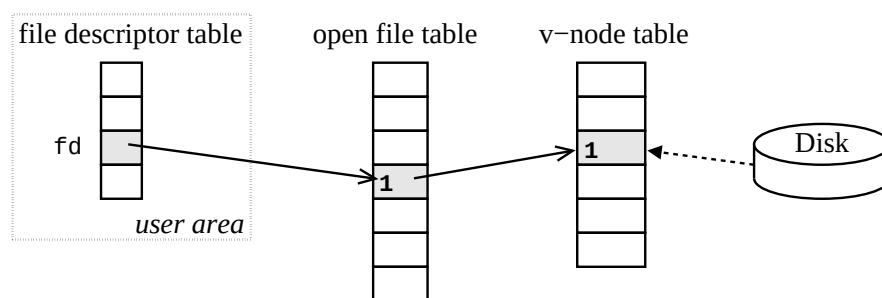
Pokud programujeme aplikaci, potřebujeme stabilní rozhraní pro přístup k souborovým systémům. Naopak programátor souborového systému musí vědět jaké je rozhraní operačního systému na které se má napojit. V unixových operačních systémech je tímto rozhraním tzv. **virtual filesystem**. Ten definuje jednotlivé operace a jejich parametry pro uživatelské aplikace a zároveň určuje, jaké operace a sémantiku musí implementovat tvůrce souborového systému. Zároveň jednotné rozhraní a abstraktní vrstva, která ho tvoří umožňuje na jednom systému koexistovat najednou více souborovým systémům. I pokud pomineme vhodnost rozdělení pevného disku a použití různých souborových systémů na různých diskových oblastech pro různé účely, budeme potřebovat kromě základního souborového systému pro pevný disk ještě minimálně souborový systém pro CD-ROM a DVD (ISO9660, UDF), systém VFAT pro disketové mechaniky a případně nějaký síťový souborový systém.

2.5.1 Aplikační rozhraní pro vstup/výstup

Otevření souboru

```
int open(char *path, int flags, int mode)
```

Otevření souboru spočívá ve vyhledání jednotlivých komponent cesty souboru (*lookup* až v posledním adresáři cesty najdeme číslo i-uzlu souboru. Pokud ještě neexistuje, načte se i-uzel z disku do paměti a připojí se záznam do systémové tabulky otevřených souborů, kde je doplněna o další informace (např. kolik procesů/deskriptorů má soubor momentálně otevřen, režim otevření, pozici v souboru) – nazývá se pak v-uzel (v-node). Následně je vytvořen nový popisovač (descriptor) souboru v tabulce otevřených souborů procesu. Index do této tabulky je poté vrácen procesu jako výsledek volání *open()*. Situaci demonstruje následující obrázek:



Obrázek 2.7: Použité struktury při otevření souboru

Čtení a zápis do souboru

```
int read(int fd, char *buf, size_t count)
```

```
int write(char *buf, size_t count)
```

Při čtení dat se provede nejprve určení adresy na disku, ze které je požadováno čtení. Offset v souboru je uložen v systémové (globální) tabulce otevřených souborů. Po zjištění adresy bloku na disku se zkontroluje, zda daný blok již není načten ve vyrovnávací paměti (cache). Pokud ne, je blok do paměti cache načten. V každém případě je poté potřebná část vyrovnávací paměti zkopírována do uživatelského bufferu.

Změna pozice v souboru

```
int lseek(int fs, off_t offset, int whence)
```

Volání lseek umožňuje nastavit novou pozici v souboru a tím provádět přímý přístup jen k částem souboru které nás zajímají. Při volání se předává jednak požadovaný offset (kladný i záporný) a poté příznak odkud má být offset aplikován. Možnosti jsou aktuální pozice, začátek a konec souboru. Pokud provedeme nastavení pozice před začátek souboru, volání se provede, ale pozice bude na začátku. Naproti tomu, když provedeme nastavení pozice za konec souboru a na tomto místě provedeme zápis vznikne nám tzv. **řídký (sparse) soubor**. Místo, které jsme v souboru přeskočili nebude uloženo na disku, při čtení se bude tato oblast díky operačnímu systému chovat jako by v ní byly uloženy samé nuly. Tímto způsobem se někdy vytvářejí odkládací soubory (swap file) (pokud jsou použity místo odkládacích diskových oblastí – swap partitions). Vytvoříme řídký soubor o velikosti například 512MB, který ale na disku zabírá 1B. Při alokaci tohoto odkládacího souboru se na disku alokují pouze skutečně využité bloky. Pokud tedy využijeme ze swapu nejvíce 128MB, bude odkládací soubor na disku zabírat 128MB. Řídké soubory musí podporovat daný systém souborů.

DATA	000000000000	DATA	0000000000000000	DATA
------	--------------	------	------------------	------

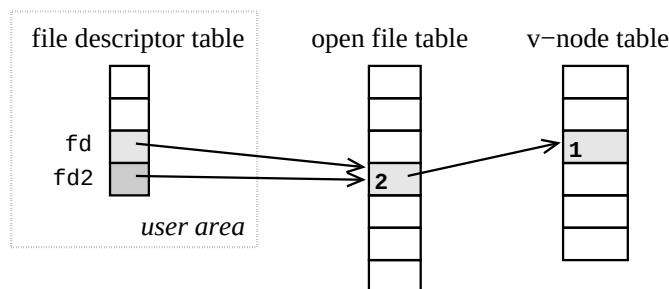
Obrázek 2.8: Schéma řídkého (sparse) souboru

Duplikace deskriptoru

```
int dup(int oldfd)
int dup2(int oldfd, int newfd)
```

Duplikace je nástroj pro provádění přesměrování vstupu a výstupu. Při jeho použití se zadaný deskriptor z tabulky procesu zkopíruje na nejmenší volný (volání `dup()`) nebo na zadaný `dup2()` deskriptor. Zároveň dojde k aktualizaci systémové tabulky otevřených souborů. Situaci shrnuje obrázek 2.9. Pro přesměrování standardního vstupu a výstupu se využívá faktu, že pod pojmem **standardní vstup** proces vždy rozumí deskriptor s hodnotou 0. Pokud do této pozice v tabulce umístíme jiný deskriptor (souboru, roury, soketu atd.) bude z něj proces

číst při požadavku na použití standardního vstupu. Stejně to platí pro **standardní výstup** (deskriptor 1) a **standardní chybový výstup** (deskriptor 2).



Obrázek 2.9: Výsledek duplikace deskriptoru procesu

Při přesměrování tedy otevřeme soubor, na který chceme přesměrování provést. Tím získáme jeho deskriptor. Dále uzavřeme voláním `close()` např. standardní vstup a následně voláním `dup()` zkopírujeme deskriptor souboru do uvolněné pozice pro standardní vstup. Další operace čtení ze standardního vstupu v programu již pracují s podstrčeným souborem. Vzhledem k tomu, že při volání `exec()` se deskriptory zachovávají, je možné nachystat v jednom programu přesměrování a poté s touto konfigurací spustit jiný program, který o tom nemusí nic vědět – pro něj je to stále pouze standardní vstup.

Zavření souboru

```
int close(int fd)
```

Při uzavření souboru (pokud nepoužijeme volání `close()`), provede se automaticky při ukončení procesu) dojde k zrušení položky pro daný deskriptor z tabulky deskriptorů procesu. Dále se sníží počítadlo v systémové tabulce otevřených souborů. Pokud dosáhne nuly, obsah v-uzlu je synchronizován zpátky na disk do i-uzlu a poté se v-uzel i záznam v tabulce uvolní. Pokud je při aktualizaci i-uzlu zjištěno, že počet jmen souboru je 0 (není veden v žádném adresáři), je v tomto okamžiku zrušen z disku (jeho i-uzel i datové bloky jsou uvolněny pro pozdější využití, některé systémy umožňují obojí před uvolněním přepsat nulami pro zamezení zneužití dat nebo informací o daném souboru). Při uzavření souboru se automaticky neukládá obsah jeho bloků ve vyrovnávací paměti na disk (lze vynutit explicitně voláním `fsync()`).

Informace o souboru

```
int stat(char *name, struct stat *buf)
int fstat(int fd, struct stat *buf)
```

Sada volání `stat()`, `fstat()` je určena ke zjišťování atributů souboru – buď z jeho deskriptoru nebo zadáním jména souboru. Informace, které jsou vráceny

ve struktuře *stat* byly popsány v části 2.1.1. Existuje ještě varianta *lstat()*, která pro symbolické odkazy vrací informace o odkazu místo informací o souboru na které je odkazováno jak to dělají základní uvedené varianty.

2.6 Implementace souborových systémů

V této části naleznete jednak několik odkazů na informační zdroje ohledně konkrétních souborových systémů respektive o jejich implementaci obecně. Dále zmíníme několik konkrétních souborových systémů jako příklady jednotlivých kategorií.

Velmi kvalitní knihou o implementaci souborových systémů je [17]. Podrobné informace o distribuovaných souborových systémech lze nalézt v [16]. Další odkazy jsou pak uvedeny u jednotlivých typů souborových systémů.



2.6.1 Klasické souborové systémy

CD-ROM filesystem ISO9660

Souborový systém používaný a CD-ROM (R,RW) discích je standardizován normou ISO číslo 9660, ta se často používá jako název tohoto souborového systému. CD disk nemá cylindry, ale celá stopa je lineární, spirálovitě navinutá od kraje do středu. Spirála je rozdělena na logické bloky o velikosti 2352 bajtů, z nichž 2048 je využitelných pro data. Na začátku (okraji) disku je 16 bloků, jejichž význam není upřesněn standardem, je zamýšlen na využití výrobcem nebo pro zaváděcí program v případě bootovacích disků. Následuje sekce popisných informací o disku jako identifikátor producenta disku, název disku nebo odkaz na kořenový adresář disku (tj. jeho počáteční blok). Položky v adresáři jsou řazeny abecedně s výjimkou první položky, která odkazuje na daný adresář a druhé která odkazuje na nadřazený adresář (tj. výsledné . a ..). maximální počet zanoření adresářů je stanoven na 8. Adresářová položka obsahuje 10-12 polí. Ty které jsou numerické jsou obsaženy dvakrát, jednou v big-endian kódování a jednou v little-endian aby se CD dalo zpracovávat jak na systémech jako Pentium tak například na stanicích s procesorem UltraSparc. Za adresářovými položkami kořenového adresáře leží datové bloky souborů a položky podadresářů. Je použita souvislá alokace, protože soubory se na disku nemohou měnit a tak se neprojeví její nevýhody. Pro určení, kde leží datové bloky souboru tak stačí počáteční blok a délka souboru, obě informace jsou v adresářové položce k dispozici. Na konci adresářové položky je případně doplněn nulový bajt tak aby celková velikost položky byla sudá, tedy aby číselné hodnoty ve všech adresářových položkách byly zarovnané stejně. Následuje ještě blok systémových informací, které může každý operační systém využít podle svého. Jak je vidět, je tento standard značně nehospodárný, což je dáno především tím, že v době jeho schvalování bylo hlavním zájmem prosazení standardu a tedy uspokojení všech případných uživatelů.

Vzhledem k limitacím, které iso9660 zavedlo se začaly brzy objevovat rozšíření pro doplnění některých vlastností. Jedním z nich je *Rock Ridge*, které definuje jak pomocí systémových dat v adresářové položce uložit unixová práva,

symbolické linky a další unixové speciality. Druhé nejrozšířenější vylepšení se jmenuje *Joliet*, navrhla ho firma Microsoft a přidává například dlouhé názvy souborů, kódování jmen v Unicode nebo zapouzdření adresářů větší než standardních 8.

FAT12,16,32 MS-DOS a Windows

Systém souborů fat se v různých verzích používá v systémech firmy Microsoft již řadu let. Systém je založen na principu FAT zmiňovaném v 2.3.3 – tabulka pro alokaci souborů. Číslo v názvu pak udává velikost hodnoty pro adresaci disku. Například FAT16 používá 16bitové adresy a je tedy schopna rozlišit 65536 různých adres. Při použití jednoho diskového bloku by maximální velikost disku byla pouze 264MB, proto se používají větší bloky dat, v systémech firmy Microsoft nazývané *clustery*. Jejich velikost je 512B až 4KB pro FAT12 (max. disk 16MB), 2KB až 32KB pro FAT16 (max disk 2GB) a 4KB až 32KB pro FAT32 (max disk 2TB). Adresářová položka má pevnou velikost – 32 bajtů. Jsou zde uloženy jméno souboru (8B), přípona (3B), atributy (1B), čas a datum vytvoření (2+2B), odkaz na první datový blok (2B) a velikost (4B). Zbytek místa je nevyužit. Pro FAT32 (Windows 98) byly některé položky rozšířeny (datum a čas vytvoření) a další přidány do předchozího volného místa (čas posledního přístupu změny a další 2B pro adresu začátku souboru). Dalším rozšířením FAT32 bylo přidání dlouhých názvů. Kvůli zachování kompatibility nebylo možné měnit základní strukturu a dlouhé názvy jsou tak uloženy na místě adresářové položky předcházející skutečnou položku pro daný soubor. Název je uložen pozpátku po položkách (ne po znacích), tedy začátek názvu v položce nejbližší vlastní adresářové položce souboru a může zabírat více bloků pro adresářové položky. Detaily o systému FAT16 a 32 najdete například v [22].

EXT2 – Standardní linuxový systém souborů

Původní souborový systém v Linuxu pochází ze systému MINIX. Ten byl později rozšířen a doplněn a nazván extfs – Extended filesystem. Po dalších úpravách vznikla verze ext2, která se používá v některých případech dodnes. Ext2 vychází do jisté míry ze systému souborů FastFilesystem, z projektu BSD. Ke správě datových bloků používá až trojitou indirekci v i-uzlu (viz. 2.3.4). Adresářové záznamy jsou uloženy na disku stejným způsobem jako normální soubor, používá se pouze odlišné přístupové API. Používá se zde technika logických skupin bloků (*cylinder groups*) pro zvýšení výkonu. Správa volného místa je prováděna pomocí bitové mapy. Ext2 také používá prealokaci pro rozšíření souboru a to až o velikosti 8 bloků (ty jsou o velikosti 1-4KB). Pokud je soubor uzavřen a prealokované bloky zůstanou nevyužity, jsou vráceny do seznamu volných bloků. Pokud je soubor uzavřen a prealokované bloky zůstanou nevyužity, jsou vráceny do seznamu volných bloků.

2.6.2 Moderní (žurnálovací) souborové systémy

Mezi moderní souborové žurnálovací systémy můžeme zařadit NTFS (MS Windows), ReiserFS (primárně Linux, práce na portu pro FreeBSD), JFS (IBM AIX, Linux) a dále dva z nich, kterým budeme věnovat větší pozornost:

ZFS

ZFS je asi dnes nejmodernějším souborovým systémem. Byl napsán zcela od začátku aby se oprostil od problémů úprav starého kódu a zachování kompatibility. Mezi jeho základní vlastnosti patří adresace pomocí 128bitů. Odtud pochází ostatně i název kde Z znamená *Zeta*. Systém byl navržen tak, aby neměl žádné reálné limity na počty souborů, adresářů jejich velikosti či velikost celého souborového systému. Dále tento systém spravuje pro každý blok dat kontrolní součet, takže je schopen detekovat jak chyby v metadatech (jako běžné žurnálovací souborové systémy), tak chyby v uživatelských datech. Zahrnuje dále vestavěnou podporu pro kompresi a pro softwarový RAID režim označovaný jako RAID-Z (viz. [13]), podobný režimu RAID5. ZFS podporuje systém stínových kopií svazků, což může být užitečné především pro zálohování velkých souborových systémů. A v neposlední řadě ZFS přichází s vestavěným LVM, díky čemuž je jeho použití velice rychlé a jednoduché. ZFS je primárně vyvíjen firmou Sun pro jejich OpenSolaris, existují porty pro Linux a FreeBSD.

XFS

Souborový systém XFS byl vyvinut firmou SGI pro jejich unixový operační systém IRIX, poslední dobou zažívá renesanci na platformě Linux. Jedná se o moderní žurnálovací souborový systém s podporou víceprocesorových systémů nebo real-time aplikací (garantovaná propustnost). Existuje port XFS do systému FreeBSD. Systém XFS má přímou podporu RAID polí či LVM kdy je možné při vytváření souborového systému popsat použité diskové řešení a souborový systém podle toho optimalizuje svou činnost – především alokační politiku pro maximální výkon. Další charakteristikou je proměnná velikost bloku založená na extentech. XFS podporuje on-line (za chodu, bez odpojení svazku či přepojení do režimu jen pro čtení) defragmentaci a změnu velikosti svazku. Jsou také podporovány stínové kopie (snapshots).

2.6.3 Síťové a distribuované souborové systémy

souborové systémy můžeme zařadit jak dnes již klasické – například NFS nebo SMB/CIFS, tak moderní distribuované a gridové souborové systémy jako Global Filesystem (Systina/RedHat), CXFS (distribuované rozšíření pro XFS od SGI) nebo OCFS (Oracle Cluster Filesystem – souborový systém specializovaný na využití pro clustery databázových serverů s DB Oracle).

NFS – Network Filesystem

Tento síťový souborový systém z dílny firmy Sun Microsystems je postaven na principu vzdáleného volání procedur (RPC). Jednotlivá volání funkcí souborového systému klienta jsou pomocí RPC přenesena na server a zde vykonána. Server zpřístupňuje části svého stromu adresářů tím že je tzv. exportuje. Exportované adresáře spolu s atributy a seznamem klientů, kteří mají povoleno připojení svazku je uveden v souboru */etc/exports*. Klient použije souborový systém serveru připojením stejně jako lokální systém souborů, jen místo názvu zařízení se použije definice serveru a svazku ve formátu *server:/exported/path*. NFS klient podporuje jednotlivá volání operačního systému překvapivě kromě volání *open()* a *close()*. To je z toho důvodu, že protokol NFS je *bezstavový*, to znamená, že k provedení operace nepotřebuje znát aktuální stav klienta či serveru nebo stav předchozích provedených operací. Každá z nich je naprosto samostatná a nezávislá. To má výhodu, že po restartu klienta nebo serveru není z pohledu uživatele nic znát. Komunikace se na chvíli zastaví a poté jede bez přerušení dál. Bezstavovost se ale projevuje v relativně nižším výkonu, server nepredikuje akce klienta (s výjimkou *read-ahead*) a jednotlivé operace probíhají v blocích o standardní velikosti 8KB, což přináší relativně velkou latenci. Proto se často jako první krok optimalizace výkonu doporučuje nastavit maximální podporovanou velikost požadavku, což bývá alespoň 16KB, někdy i 32KB. Existuje také systém souborů *cacheofs*, který provádí cachování požadavků na síťové soubory na lokálním disku, což může dále pomoci zvýšit výkon.

NFS dlouho nepodporovalo bezpečnost, poslední implementace verze protokolu 3 již umožňují použít rozšíření pro zabezpečení přenosu. Protokol nové verze 4 již bezpečnost podporuje nativně.

2.6.4 Virtuální souborové systémy

Vzhledem k využití souborových systémů jako prostředku abstrakce v unixových operačních systémech zde můžeme nalézt i tzv. virtuální souborové systémy, tedy takové, které nepoužívají žádné médium pro trvalé uložení dat, ale jsou generovány (jádreem) operačního systému při každém pokusu o přístup k nim.

ProcFS – souborový systém pro procesy

Tento souborový systém byl primárně určen pro zpřístupnění informací o jednotlivých procesech do uživatelského prostoru. Později byly tyto informace v některých operačních systémech doplněny o další obecné zjišťování informací (čtení souborů) ale i jejich nastavování (zápis do souboru) částmi jádra operačního systému. Implementace se nachází v jaderných modulech a spočívá především v implementaci funkce, která vygeneruje potřebná data pro čtení souboru a případně další, která zpracuje data předaná uživatelem (pro zápis do souboru). Typickými příklady jsou na Linuxu soubory */proc/uptime*, */proc/cpuinfo*, */proc/mounts* nebo */proc/meminfo*.

SysFS – export systémových informací

Jedná se o virtuální systém souborů, který byl primárně určen k přesunu informací o zařízeních v systému z *proc*. Informace zde dosažitelné jsou generovány poloautomaticky na základě údajů, které vyplní ovladače zařízení. Ovladač má pak možnost přidávat další soubory pro své potřeby. Typické pro SysFS je, že k jedné informaci (k jednomu adresáři) je možné se dostat více různými způsoby – informace jsou pomocí symbolických odkazů provázány a máme tak možnost přistupovat k nim z několika pohledů jako například přes seznam zavedených modulů, seznam zařízení na konkrétní sběrnici nebo typu sběrnice, zařízení dané třídy (sound) a podobně. Systém souborů SysFS bývá nejčastěji připojen do adresáře */sys*.

DevFS – dynamické vyvážení speciálních souborů

Speciální soubory pro jednotlivá zařízení byly v dřívějších verzích unixových operačních systémů statické, uložené na disku v obecném souborovém systému (např. Ext2/3), který je ovšem musel podporovat (nelze vytvořit speciální soubor na VFAT). Vzhledem k tomu, že zařízení připojených k počítači neustále přibývá a navíc je dnes velká část z nich tzv. hotplug – tedy připojitelná a odpojitelná za chodu systému, hledala se cesta, jak speciální soubory generovat dynamicky. Tím je zároveň možné snížit zmatek v adresáři */dev*, kde dříve byly spousty speciálních souborů pro všechny případy s tím, že většinou velké procento z nich nebylo na dané stanici nikdy využito a naopak při doplnění nového HW a jeho ovladače bylo nutno patřičný speciální soubor vytvořit ručně (program *mknod*). Proto byl vyvinut systém souborů DevFS a později jeho logický následovník *udev*. Oba fungují tak, že při registraci a inicializaci ovladače zařízení je dynamicky vytvořen (stále v adresáři */dev*) patřičný speciální soubor nebo soubory, které jsou pro dané zařízení potřebné. V některých případech jsou vedlejší čísla zařízení generována také dynamicky. Moderní *udev*, funguje na stejných principech, jen pro provádění akcí využívá systém *hotplug*, který je schopen část operací provádět v uživatelském režimu, což je pro jednoduchost a bezpečnost preferováno.

2.7 Shrnutí

V této kapitole jsme shrnuli nejdůležitější principy a techniky používané s souborovými systémy. Zjistili jsme, že pro pochopení funkce, nutných omezení a výsledného výkonu je důležité mít představu také o fungování fyzického zařízení, na které jsou data ukládána a pokročilých technologií, které nám umožňují zlepšovat spravovatelnost, výkon a spolehlivost těchto zařízení. Zmínili jsme různé speciální typy souborů, umožňujících, aby většina prostředků v operačním systému se tvářela jako soubor a využívala tak jednotné rozhraní uživatelského API. Pokud systém využívá více uživatelů, je nutné také řešit otázku řízení přístupu (díky abstrakci nejen k uživatelským souborům). Ukázali jsme si jak základní systém s trojicí práv, tak pokročilou techniku nazývanou seznamy pro řízení přístupu (ACL). V další části jsme zkoumali fungování implementace adresářové



struktury a důvody pro odlišný přístup od běžných souborů. Byl též shrnut význam základní adresářové struktury unixového operačního systému. Pro správu volného prostoru a datových bloků jsme popsali několik základních a běžně používaných technik s důrazem na zaměření na efektivnost ukládání nebo výkon. Nakonec jsme popsali základní operace uživatelského rozhraní pro práci se soubory a ukázali si několik příkladů reálných implementací souborových systémů.

2.8 Kontrolní otázky

1. Zkuste se zamyslet, které kombinace přístupových práv z části 2.1.4 nemají praktický smysl a mohly by se využít pro jiné účely.
2. Představte si systém, který neumí vytvářet adresáře a všechny soubory uchovává v jednom kořenovém adresáři. Navrhněte mechanismus, kterým by vytváření adresářů šlo emulovat.
3. Existuje zapojení diskového subsystému takové, že spojuje výhody vysokého výkonu a spolehlivosti dat? Dokážete seřadit úrovně RAID od nejvýkonnějšího k nejméně výkonnému a od nejspolehlivějšího k nejméně spolehlivému?
4. Jaká je nevýhoda LVM v případě totálního selhání diskového subsystému? Je stejný problém i u některého z polí RAID?
5. Zvažte rozdíly v nasazení alokačních politik dle typu nasazení souborového systému. Jako příklady uvažujte oddíl pro domácí adresáře uživatelů, pro uložení dat databázového serveru a oddíl pro ukládání logovacích záznamů systému.
6. Uvažujte 200GB disk s velikostí alokačního bloku 4KB. Kolik místa bude nutno vyhradit v paměti RAM pro FAT, jestliže každý ukazatel zabere 4B? Kolik otevřených souborů je možno uvažovat při stejné spotřebě paměti při použití indexové alokace a velikosti i-uzlu 128B. Jaká bude maximální velikost při použití přímých odkazů v i-uzlu. Jaká pro nepřímé odkazy první, druhé úrovně?
7. Předpokládejte, že ve frontě čekají na vyřízení požadavky na disk s adresami i86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. Disková hlava je nyní nad cylindrem 125. Celý disk má 5000 cylindrů. Pro algoritmy uvedené v sekci 2.4.4 spočítejte celkový počet přesunů hlavičky pro provedení všech požadavků.
8. Pokuste se navrhnout kostru programu, který jako argumenty přijme názvy dvou souborů a zbylé argumenty bude brát (bez kopírování!) jako název programu a jeho parametry. Provede přesměrování standardního vstupu a standardního výstupu a poté spustí s touto konfigurací zadaný program. Jako pomůcku využijte manuálové stránky volání *open()*, *close()*, *dup()*, *execvp()*.
9. Pokuste se najít příklady vhodnosti nasazení různých typů souborových systémů zmiňovaných v této kapitole nebo které sami znáte a diskutujte jejich výhody a nevýhody v daných situacích.



2.9 Řešení vybraných kontrolních otázek

8. Kostra programu může vypadat následovně (pro jednoduchost nejsou kontrolovány žádné návratové hodnoty ani provedena inicializace, což by ve skutečném programu bylo nemyslitelné!):



```
file_in = open(argv[1], O_RDONLY);
file_out = open(argv[2], O_WRONLY);
close(stdin);
dup2(file_in, stdin);
close(stdout);
dup2(file_out, stdout);
execvp(argv[3], &argv[4]);
```

Kapitola 3

Správa procesů



4:00

Cílem této kapitoly je seznámení se správou procesů v operačních systémech. Správa procesů (*process management*) zahrnuje *plánovač* (scheduler), který přiděluje procesor (CPU) procesům, *správu paměti* (memory management) a podporu *meziprocesové komunikace* (IPC). V této kapitole bude diskutován koncept procesů a vláken v operačních systémech, různé plánovací algoritmy a základní systémová volání spojená se správou procesů včetně meziprocesové komunikace.



3.1 Koncept procesů

První počítačové systémy umožňovaly běh pouze jednoho programu v čase. Tento program měl kompletní kontrolu nad celým systémem a přístup ke všem systémovým zdrojům. Současné počítačové systémy umožňují souběžný běh více programů, což mj. vedlo k zavedení pojmu proces. Proces je chápán jako program v běhu, jeho instance, která je nahrána do paměti počítačového systému a spuštěna. V jednom okamžiku tak mohou být spuštěny instance (procesy) různých, ale i stejných programů.

Hlavním úkolem operačního systému je zajištění běhu uživatelských procesů. Jelikož procesů může běžet současně více a systémové zdroje jsou omezené, musí procesy tyto zdroje sdílet. Operační systém musí také zajišťovat správu přístupů k těmto sdíleným zdrojům. Jedním z těchto zdrojů je i procesor (*CPU – Central Processing Unit*). Počet procesů obecně převyšuje počet dostupných procesorů v počítačovém systému a je tedy nutné plánovat přidělování procesoru (procesorů) jednotlivým procesům.

3.1.1 Proces

Jak už bylo řečeno, proces je chápán jako program v běhu. Proces je však víc než jen instance kódu, sekvence instrukcí, které se postupně provádějí. Součástí procesu jsou i informace o aktuální pozici (*program counter*), obsah registrů procesoru, zásobník obsahující dočasná data (parametry metod, lokální proměnné apod.) a další údaje v závislosti na hostitelském prostředí. Obecně je proces v operačním systému definován následujícími vlastnostmi:



- identifikátorem (*PID*),
- stavem jeho plánování,
- programem, kterým je řízen,
- obsahem registrů (včetně EIP a ESP apod.),

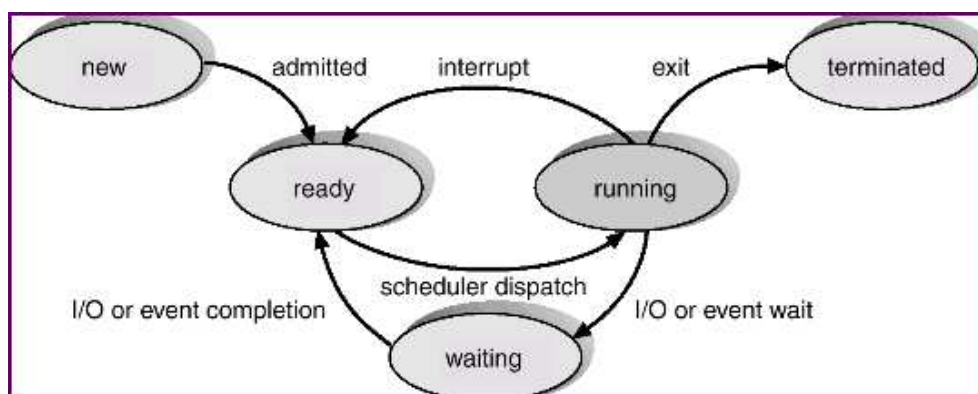
- daty a zásobníkem,
- využitím dalších zdrojů OS a vazbou na další objekty OS (otevřené soubory, signály atd.)

3.1.2 Stavy procesu

Proces může během své existence projít několika stavy. Stav procesu vyjadřuje aktuální aktivitu procesu a je důležitý pro plánování procesů. Počet a názvy stavů se mohou lišit v různých operačních systémech, v každém z nich však lze nalézt tyto základní stavy procesu (viz obrázek 3.1):

- *Nový (New)*. Proces je právě vytvořený.
- *Běžící (Running)*. Proces je přiřazen procesoru a běží, provádějí se jeho instrukce.
- *Čekající (Waiting)*. Proces čeká na výskyt nějaké události, např. dokončení vstupně výstupní operace.
- *Připravený (Ready)*. Proces čeká na své přiřazení procesoru.
- *Ukončený (Terminated)*. Proces ukončil svůj běh.

V každém stavu může být současně více procesů, ovšem v každém okamžiku může být procesoru přidělen pouze jeden proces.



Obrázek 3.1: Stavy procesů.

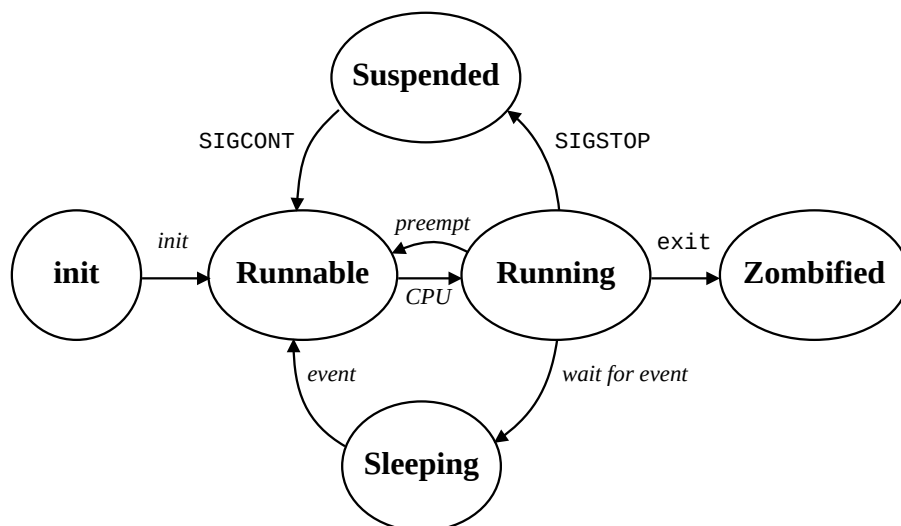
Stavy procesů v operačním systému Unix. Unix pracuje s trošku modifikovanou verzí stavové množiny a přechody mezi stavy. Tabulka 3.1 shrnuje základní stavy procesu v Unixu a jejich význam. Přechodový diagram stavů procesu v Unixu je ukázán na obrázku 3.2. Rozdíl oproti základnímu přechodovému



stav	význam
Vytvořený	ještě neinicializovaný
Připravený	mohl by běžet, ale nemá CPU
Běžící	používá CPU
Mátoha	po <code>exit</code> , rodič ještě nepřevzal exit-code
Čekající	čeká na událost (např. dokončení <code>read</code>)
Odložený	„zmrazený“ signálem <code>SIGSTOP</code>

Tabulka 3.1: Stavy plánování procesu v systému Unix.

diagramu spočívá v rozlišování dvou stavů, ve kterých proces čeká na výskyt události. Stav *sleeping* odpovídá stavu *čekající* (*waiting*), navíc je tu stav *suspended*, do kterého se proces dostane speciálním signálem `SIGSTOP` a čeká na signál `SIGCONT` – viz kapitola 3.5.1. Stav *terminated* je označován jako *zombified*. Proces se dostane do tohoto stavu v okamžiku, kdy ukončuje svůj běh a čeká, až si jeho rodičovský proces (viz kapitola 3.4.1) převezme jeho návratovou hodnotu. Každý proces končí s návratovou hodnotou, která identifikuje způsob jeho ukončení, tj. zda se ukončil v pořádku (hodnota 0) nebo zda došlo v průběhu běhu k chybě (hodnota různá od 0 pak identifikuje typ chyby).



Obrázek 3.2: Přechodový diagram stavů plánování procesu v Unixu.

3.1.3 Process Control Block

Každý proces je v operačním systému konceptuálně reprezentován speciální strukturou nazývanou *process control block* (PCB). PCB bývá realizována různými způsoby, podle potřeb jednotlivých operačních systémů. PCB obsahuje mnoho informací asociovaných s procesem, zejména:

DEF

- *Identifikátor procesu (PID)*. Jednoznačná identifikace procesu v rámci celého operačního systému.
- *Stav procesu*. Stav, ve kterém se proces aktuálně nachází.
- *Program counter*. Obsahuje adresu následující instrukce, která se má provést v tomto procesu.
- *Obsahy registrů CPU*. Počet a typ registrů se různí pro různé počítačové architektury. Jejich součástí jsou akumulátory, indexové registry, zásobníkový ukazatel, obecné registry atd. Tyto stavové informace (společně s adresou následující instrukce) musejí být uloženy kvůli přepínání kontextu, tj. situaci, kdy je nutné uložit komplexní stav procesu, obnovit stav jiného procesu a předat mu procesor pro pokračování v běhu.
- *Informace nutné pro plánování procesů*. Tyto informace obsahují prioritu procesu, ukazatel na plánovací frontu a další plánovací informace.
- *Informace nutné pro správu paměti*. Tyto informace mohou obsahovat např. tabulky stránek atd.
- *Informace spojené s plánováním (accounting information)*. Údaje o době, po kterou je přidělen procesor, informace o době života procesu atd.
- *Informace o stavu vstupně výstupních operací*. Informace obsahující seznam vstupně výstupních zařízení přidělených procesu, tabulku otevřených souborů atd.

3.1.4 Přepnutí kontextu procesu

Přepnutí procesoru (CPU) na jiný proces vyžaduje uložení stavu starého procesu, který dosud používal CPU, a nahrání uloženého stavu nového procesu. Tato úloha je označována jako *přepnutí kontextu*. Kontext procesu je reprezentován jeho PCB.

Princip přepnutí kontextu mezi procesy P_0 a P_1 je znázorněn na obrázku 3.3. Plánovač odebere procesor procesu P_0 , uloží jeho stav do PCB_0 , obnoví stav procesu P_1 z jeho PCB_1 a přidělí procesor procesu P_1 .

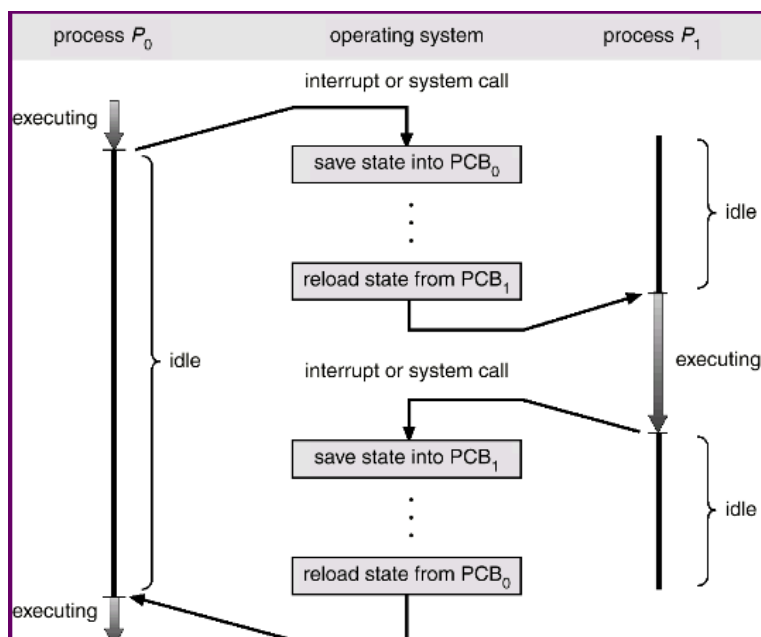
Přepnutí kontextu je spojeno s režii – přepnutí trvá stovky až tisíce instrukcí (1-1000 μs). Všechny informace a data, která jsou s během procesu spojena, není nutno ukládat a poté obnovovat, je tedy možné režii spojenou s přepnutím kontextu snížit.

3.1.5 Vlákna

Model procesů, se kterým jsme se až dosud zabývali, předpokládal, že proces je vykonávaný program s jedním vláknem řízení. Moderní operační systémy umožňují, aby proces obsahoval více vláken řízení. Každé vlákno je definováno svým identifikátorem (*thread ID*), aktuální pozicí v programu (*program counter*), množinou registrů a zásobníkem. Pokud má proces více vláken, je schopen vykonávat

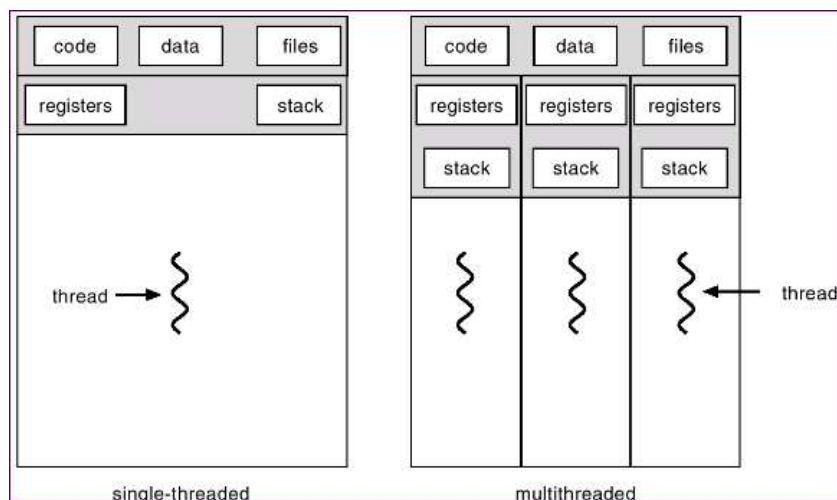
DEF

DEF



Obrázek 3.3: Přepnutí kontextu.

více úloh současně. Ilustrace jednovláknového a vícevláknových procesů je na obrázku 3.4.



Obrázek 3.4: Jednovláknové a vícevláknové procesy.

Vícevláknové procesy mají oproti jednovláknovému procesu několik vlastností, které jsou klíčové z hlediska jejich použitelnosti. Jsou to zejména:

- *Responsiveness*. Multivláknové procesy zvyšují rychlost odezvy aplikace vůči uživateli, pokud je část aplikace blokována nebo provádí náročnou

operaci. Např. webový prohlížeč umožňuje interakci s uživatelem, zatímco se v jiném vlákne nahrává obrázek.

- *Resource sharing*. Vlákna sdílejí paměť, zdroje a kód procesu. Sdílení kódu umožňuje mít více různých vláken aktivit ve stejném adresovém prostoru.
- *Economy*. Alokace paměti a zdrojů při vytváření procesu je drahá. Díky sdílení zdrojů je vytváření a přepínání kontextu vláken mnohem úspornější.

Vlákna mohou být v systémech podporována na dvou úrovních:

- *Uživatelská vlákna (user threads)* jsou spravována na uživatelské úrovni, bez účasti jádra operačního systému.
- *Vlákna jádra (kernel threads)* jsou podporována a spravována přímo operačním systémem.

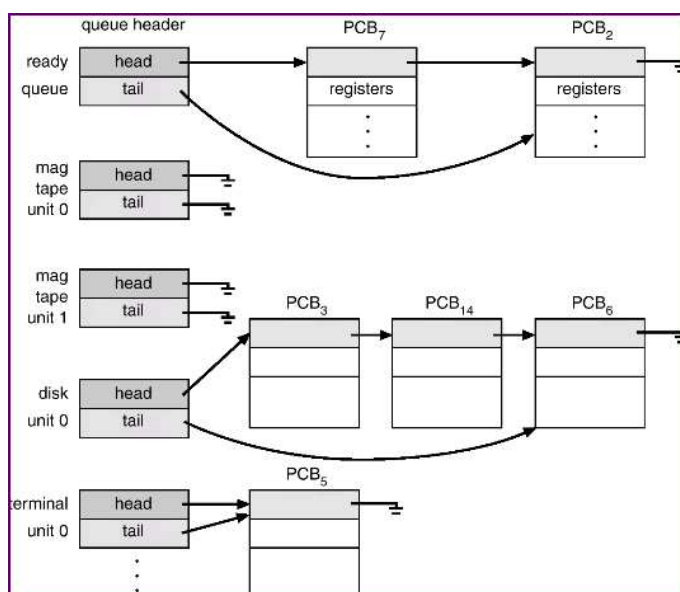
3.2 Plánování procesů

V operačním systému, který umožňuje souběžný běh více procesů, sledujeme dva základní cíle. Z hlediska multiprogramování je klíčová maximalizace využití procesoru. Z pohledu sdílení času (*time sharing*) je důležitá frekvence přepínání procesoru (CPU) mezi jednotlivé procesy takovým způsobem, aby byl uživatel schopen komunikovat s každou spuštěnou aplikací (tj. procesem). K zajištění těchto cílů existuje v operačních systémech *plánovač procesů (process scheduler)*, který vybírá nejvhodnější proces pro běh v daném okamžiku. V počítačových systémech, které mají pouze jeden procesor, může reálně běžet pouze jeden proces – pokud je jich spuštěno více, musí ostatní čekat, až plánovač procesů rozhodne o jejich přidělení procesoru.

3.2.1 Plánovací fronta

Když proces vstupuje do systému, je vložen do fronty úloh (*job queue*), která obsahuje všechny procesy v systému. Procesy, které jsou v hlavní paměti, jsou připravené k běhu a čekají na přidělení procesoru jsou umístěny ve *frontě připravených procesů (ready queue)*. Systém obsahuje i další fronty procesů. Každý proces běží většinou po nějak omezenou dobu a poté je mu odebrán procesor, např. protože vypršelo jemu přidělené časové kvantum, nebo provádí vstupně-výstupní operaci a čeká na její vyřízení apod. Proces tedy může čekat nejen na procesor, ale i na jiná zařízení jako je např. disk. Seznam procesů čekajících na příslušné vstupně-výstupní zařízení je nazýváno *fronta zařízení (device queue)*. Každé zařízení má svou frontu, viz obrázek 3.5. Prvky každé fronty jsou PCB jednotlivých procesů. Hlavička fronty obsahuje ukazatel na první PCB ve frontě a na poslední PCB ve frontě.

DEF



Obrázek 3.5: Fronty procesů.

3.2.2 Plánovač

V okamžiku, kdy se procesor stane nevyužitý, musí operační systém vybrat jeden z procesů, který je umístěn ve frontě připravených procesů. Výběr je prováděn *krátkodobým plánovačem (short-term scheduler)*, který vybírá procesy podle implementovaného algoritmu. Těchto algoritmů existuje více, my si některé z nich blíže popíšeme v kapitole 3.3.

Rozhodnutí o přepínání procesů, resp. o přepínání využití procesoru, může být provedeno za některých z uvedených okolností:

1. když se proces přepne ze stavu *běžící* do stavu *čekající* (např. při vstupně-výstupní operaci),
2. když se proces přepne ze stavu *běžící* do stavu *připravený* (např. při přerušení),
3. když se proces přepne ze stavu *čekající* do stavu *připravený* (např. při dokončení vstupně-výstupní operace) a
4. když se proces ukončí.

Pokud k plánovací akci dojde v situacích 1 a 4, říkáme, že plánovací schéma je *nepreemptivní (nonpreemptive)* nebo také *cooperative* – v ostatních případech se jedná o *preemptivní (preemptive)* plánování.

- *Nepreemptivní plánování.* Pokud je proces jednou přidělen procesoru, tento proces si drží procesor, dokud ho sám neuvolní buď svým ukončením nebo přepnutím do stavu čekání. Příkladem mohou být plánovače používané v operačních systémech Windows 3.x.

DEF



- *Preemptivní plánování*. K přepnutí kontextu dochází na základě vnějšího podnětu (přerušeni), typicky od časovače. K přepnutí kontextu tedy může dojít kdykoliv a proces nemá přímou kontrolu nad dobou svého běhu.

3.2.3 Plánovací kritéria

Různé plánovací algoritmy mají různé vlastnosti, které mohou favorizovat jednu třídu procesů nad ostatními. Pro porovnání různých algoritmů lze využít plánovací kritéria, na základě kterých provádějí tyto algoritmy plánování.



- *Využití procesoru (CPU utilization)*. Jedním z cílů plánovače je využití procesoru na maximum, tedy mít co možná nejkratší dobu, kdy není procesor vytížen. Hodnota využití procesoru může teoreticky nabývat hodnot 0–100 procent, v reálných systémech se tato hodnota pohybuje od 40 do 90 procent.
- *Propustnost (Throughput)*. Další míra, určující vhodnost plánovacího algoritmu, může být počet procesů, které jsou ukončeny za jednotku času. Pro dlouhé procesy to může být jeden proces za hodiny, pro krátké transakce např. 10 procesů za sekundu.
- *Doba běhu (Turnaround)*. Doba celkového běhu procesu je suma period strávených procesem čekáním na natažení do paměti, čekáním ve frontě připravených procesů, vlastním vykonáváním procesu (tj. přidělením procesoru) a vykonáváním vstupně-výstupních operací.
- *Doba čekání (Waiting time)*. Plánovací algoritmus neovlivňuje dobu, po kterou je proces vykonáván nebo provádí vstupně-výstupní operace. Algoritmus pouze ovlivňuje dobu, po kterou proces čeká ve frontě připravených procesů. Doba čekání je tedy suma period strávených procesem ve frontě čekajících procesů.
- *Doba odezvy (Response time)*. Důležité kritérium v interaktivních aplikacích. Doba odezvy je čas, který uběhne od odeslání požadavku do okamžiku první odezvy na tento požadavek.

Základním přístupem k plánovačům je maximalizace využití procesoru a propustnosti a naopak minimalizovat dobu běhu, dobu čekání a dobu odezvy. V mnoha případech se plánovače optimalizují na průměrné míry, např. aby průměrné využití procesoru bylo 70 procent. V některých případech je vhodné se zaměřit na optimalizaci minimálních nebo maximálních hodnot – např. chceme minimalizovat maximální hodnotu doby odezvy.

3.3 Plánovací algoritmy

Algoritmů pro plánování existuje celá řada, mezi nejznámější patří:

- First Come First Served (FCFS),

- Shortest Job First (SJF),
- Round-Robin (preemptivní obdoba FCFS),
- Prioritní plánování a
- Víceúrovňové prioritní plánování.

3.3.1 First-Come, First-Served

Nejjednodušší plánovací algoritmus je *First-Come, First-Served* (FCFS). Proces, který požaduje procesor jako první, dostane přidělen procesor jako první. FCFS politika je jednoduše implementovaná frontou typu FIFO (first-in, first-out). Když proces vstoupí do fronty připravených procesů, je zařazen na konec fronty. Když je procesor volný, je přidělen procesu ze začátku fronty. Běžící proces je pak z fronty vyřazen. FCFS algoritmus je *nonpreemptivní* a jeho nevýhodou je poměrně velká doba čekání.

Proces	Doba obsazení procesoru
P_1	24
P_2	3
P_3	3

Tabulka 3.2: FCFS: Příklad procesů a doby obsazení procesoru.

Tabulka 3.2 ukazuje tři procesy a dobu jednoho obsazení (přidělení) procesoru příslušnými procesy v milisekundách. Doba přidělení procesoru (v angl. terminologii *CPU burst-time*) je čas strávený procesem od přidělení procesoru po jeho odejmutí. Pokud budeme předpokládat, že procesy z našeho příkladu přišly do fronty připravených procesů v pořadí P_1 , P_2 a P_3 , a jsou obslouženy ve FCFS uspořádání, pak doba čekání pro proces P_1 bude 0 ms, pro proces P_2 24 ms a pro proces P_3 27 ms. Průměrná doba čekání je tedy $(0 + 24 + 27)/3 = 17$ milisekund. Pokud budeme uvažovat pořadí procesů P_2 , P_3 a P_1 , pak bude doba čekání pro proces P_2 0 ms, pro proces P_3 3 ms a pro proces P_1 6 ms. Průměrná doba čekání je tedy $(6 + 0 + 3)/3 = 3$ milisekundy.

Průměrná doba čekání v algoritmu FCFS tedy není minimální a značně kolísá v závislosti na době obsazení procesoru jednotlivými procesy a jejich pořadí.

3.3.2 Shortest-Job-First

Algoritmus *Shortest-Job-First* (SJF) rozhoduje o přidělení procesoru podle doby následného obsazení procesoru (*CPU burst-time*) daným procesem. V okamžiku, kdy je procesor dostupný, vybere plánovač proces s nejmenší hodnotou doby následného obsazení procesoru. Pokud je tato hodnota shodná u více procesů, vybere se na základě algoritmu FCFS.

Tabulka 3.3 ukazuje čtyři procesy a dobu následného obsazení procesoru. Procesor bude přidělován procesům v následujícím pořadí P_4 , P_1 , P_3 a P_2 . V uvedeném pořadí budou jednotlivé čekací doby 0, 3, 9 a 16 milisekund. Průměrná doba čekání bude $(3 + 16 + 9 + 0)/4 = 7$ milisekund.

$x + y$

$x + y$

Proces	Doba obsazení procesoru
P_1	6
P_2	8
P_3	7
P_4	3

Tabulka 3.3: SJF: Příklad procesů a doby obsazení procesoru.

Algoritmus SJF je optimální, ovšem problémem je znalost doby následného obsazení procesoru. Jedním z možných způsobů, jak tuto dobu zjistit, je predikce na základě znalosti předchozích hodnot. Doba příštího obsazení procesoru je obecně predikovaná jako exponenciální průměr odhadnutých dob předchozích obsazení procesoru:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n \quad (3.1)$$

Symbol t_n představuje délku n . periody obsazení procesoru, τ_n predikovanou dobu n . obsazení, τ_{n+1} je nově predikovaná doba následujícího obsazení a α vyjadřuje relativní váhu kladenou na poslední a minulou historii v predikci.

Algoritmus SJF může být jak nepreemptivní, tak preemptivní. Preemptivní SJF se vyznačuje tím, že provádí rozhodnutí o přepnutí kontextu také v okamžiku příchodu nového procesu do fronty připravených procesů. Pokud je doba následného obsazení procesoru nově příchozího procesu menší než zbývající predikovaná doba právě běžícího procesu, pak dojde k preempci (přerušeni) běžícího procesu a procesor se předá nově příchozímu procesu. Přerušený proces se zařadí do fronty připravených procesů.

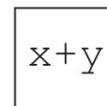
Proces	Doba příchodu	Doba obsazení procesoru
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Tabulka 3.4: SJF: Příklad preemptivního plánovače.

Preemptivní SJF algoritmus si ukážeme na příkladu procesů zobrazených v tabulce 3.4. Tabulka ukazuje kromě predikované doby následného obsazení procesoru také okamžik, kdy daný proces vstoupil do fronty. Jako první se spustí proces P_1 , protože je to jediný proces ve frontě. V čase 1 přijde proces P_2 . Zbývající doba procesu P_1 (7 ms) je větší než doba požadované procesem P_2 (4 ms). Proces P_1 bude tedy přerušen a proces P_2 bude naplánován. Sekvence period běhu jednotlivých procesů bude následující (v závorce je uveden čas přepnutí kontextu od začátku sekvence procesů):

$$P_1(1ms) \rightarrow P_2(5ms) \rightarrow P_4(10ms) \rightarrow P_1(17ms) \rightarrow P_3(26ms)$$

Průměrná doba čekání pro tento příklad bude $((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 = 6.5$ milisekund. V případě nepreemptivního algoritmu SJF by byla průměrná doba 7.75 milisekund.



3.3.3 Priority Scheduling

V tomto plánovacím algoritmu je každému procesu přiřazena priorita a procesor je vždy přidělen procesu s nejvyšší prioritou. Procesy se stejnou prioritou jsou plánovány podle algoritmu FCFS. Algoritmus SJF je speciálním případem prioritního plánování, kde priorita je inverzní k predikované době následujícího obsazení procesoru. Čím vyšší je predikovaná doba, tím menší má proces prioritu.

Priority jsou obecně indikovány celým číslem v pevně daném rozsahu, např. 0 až 1024. Nicméně neexistuje konsenzus, která hodnota označuje nejvyšší či nejnižší prioritu, tedy zda nejnižší hodnota rozsahu znamená nejnižší nebo nejvyšší prioritu. V tomto textu budeme předpokládat, že nejnižší hodnota reprezentuje nejvyšší prioritu.

Algoritmus si předvedeme na příkladu. Uvažujme procesy s dobou obsazení procesoru a prioritami, jak je uvedeno v tabulce 3.5.

Proces	Doba obsazení procesoru	Priorita
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Tabulka 3.5: Příklad prioritního plánovače.

Sekvence obsazení procesoru jednotlivými procesy bude následující (v závorce je uveden čas přepnutí kontextu od začátku sekvence procesů):

$$P_2(1ms) \rightarrow P_5(6ms) \rightarrow P_1(16ms) \rightarrow P_3(18ms) \rightarrow P_4(19ms)$$

Průměrná doba čekání je 8.2 milisekund.

Prioritní plánování může být preemptivní nebo nepreemptivní. V okamžiku příchodu nového procesu do fronty připravených procesů se jeho priorita porovná s prioritou běžícího procesu. Preemptivní plánovač přeruší běh aktuálního procesu a nahradí ho nově příchozím, pokud je priorita nově příchozího procesu větší. Nepreemptivní plánovač v tomto případě pouze vloží nově příchozí proces na začátek fronty.

3.3.4 Round-Robin

Plánovací algoritmus *Round-Robin* (RR) je preemptivní podoba algoritmu FCFS. V RR algoritmu je definováno časové kvantum obecně v rozsahu 10 až 100 milisekund. Fronta připravených procesů je implementovaná jako kruhová fronta. Plánovač postupně prochází frontou a přiděluje procesor jednotlivým procesům nejvýše po dobu jednoho kvanta. Nově příchozí procesy jsou zařazeny na konec fronty. Procesor se přiděluje procesu ze začátku fronty. Po přidělení procesoru se nastaví časovač tak, aby generoval přerušení po uběhnutí jednoho časového kvanta.

$$x + y$$



Při běhu procesu mohou nastat tyto dva případy:

- *Doba obsazení procesoru je menší než časové kvantum.* Proces se sám vzdává procesoru (např. při vstupně-výstupní operaci). Plánovač vybere následující proces ve frontě a přidělí mu procesor.
- *Doba obsazení procesoru je větší než časové kvantum.* Po uplynutí časového kvanta generuje časovač přerušení. Dojde k přepnutí kontextu a proces je vložen na konec fronty připravených procesů. Plánovač vybere následující proces ve frontě a přidělí mu procesor.

Proces	Doba obsazení procesoru
P_1	24
P_2	3
P_3	3

Tabulka 3.6: RR: Příklad procesů a doby obsazení procesoru.

Průměrná doba čekání v algoritmu RR je často dlouhá. Předpokládejme příklad procesů definovaných v tabulce 3.6, které jsou uloženy ve frontě v pořadí P_1 , P_2 a P_3 . Časové kvantum budeme předpokládat 4 milisekundy. Sekvence obsazení procesoru jednotlivými procesy bude následující (v závorce je uveden čas přepnutí kontextu od začátku sekvence procesů):

$$P_1(4ms) \rightarrow P_2(7ms) \rightarrow P_3(10ms) \rightarrow P_1(14ms) \rightarrow P_1(18ms) \rightarrow$$

$$P_1(22ms) \rightarrow P_1(26ms) \rightarrow P_1(30ms)$$

Průměrná doba čekání v uvedeném příkladu je $17/3 = 5.66$ milisekund.

$x + y$

3.3.5 Víceúrovňové prioritní plánování

Víceúrovňové plánování patří do speciální kategorie plánovacích algoritmů. Je určeno pro situace, kdy potřebujeme rozdělit procesy na různé kategorie, nad kterými je výhodnější používat různé plánovací algoritmy, optimalizované na jiná kritéria (např. u interaktivních procesů je důležitá doba odezvy).

Ve víceúrovňovém plánování existuje více front připravených procesů. Tyto fronty odpovídají jednotlivým kategoriím procesů. Každý proces je zařazen pouze do jedné z front. Každá fronta má svůj vlastní plánovací algoritmus. Kromě toho je potřeba mít plánovač ošetřující plánování mezi frontami. Tento plánovač je obvykle implementován jako prioritní plánování s fixními prioritami. Každá fronta má absolutní přednost před frontami s nižší prioritou. Pokud je ve frontě s vyšší prioritou alespoň jeden proces, nemohou být naplánovány procesy z fronty s nižší prioritou. Jiná varianta plánování mezi frontami spočívá v přidělení určité části času procesoru jednotlivým frontám. Např. fronta interaktivních procesů může dostat 60 procent času procesoru. Po vyčerpání přiděleného času v rámci definovaného časového kvanta se začnou plánovat procesy z jiné fronty.

3.3.6 Problémy spojené s prioritním plánováním

V plánovacích algoritmech spojených s prioritami může docházet k tomu, že některým procesům nebude nikdy přidělen procesor.



Problém vyhladovění. Proces, který je připraven k běhu, nedostane přidělen procesor, protože ten je neustále přidělován více prioritním procesům. Řešením tohoto problému může být přidělení určitého časového kvanta skupině procesů se stejnou prioritou. Po vyčerpání kvanta se začnou plánovat procesy s nižší prioritou, také po dobu přiděleného časového kvanta (viz např. víceúrovňové plánování, kap. 3.3.5). Jiným řešením může být explicitní kontrola, zda někdo nehladoví.

Problém inverze priorit. Nízko prioritní proces má alokovaný nějaký zdroj a je blokován více prioritními procesy. Nízko prioritní zdroj tedy nemůže dokončit operaci s daným zdrojem a blokuje ho. V okamžiku, kdy více prioritní procesy potřebují tento zdroj, jsou nutně zablokovány a musejí čekat na nízko prioritní proces.

Řešení systémů Unix/Linux. V systémech typu Unix/Linux se uplatňuje víceúrovňové prioritní plánování a *dynamická změna priorit*. Toto řešení vychází ze staticky definované prioritní úrovně *nice*, kterou může ovlivňovat uživatel při vytváření procesu. Pokud proces spotřeboval v určitém časovém úseku hodně procesorového času, jeho priorita se dynamicky snižuje (resp. zvyšuje, pokud má proces např. malé využití procesoru). Důsledkem je rychlejší reakce interaktivních procesů, protože ty většinou čekají na vstupní reakce a mají tedy malou spotřebu času procesoru. Také procesy s velmi malou počáteční prioritou mají šanci, že se jejich priorita časem zvýší a bude pro ně naplánován procesor.

Další výraznou komplikací plánování představují víceprocesorové systémy (nutnost vyvažovat výkon) či real-time systémy (nutnost zajistit garantovanou odezvu některých akcí).

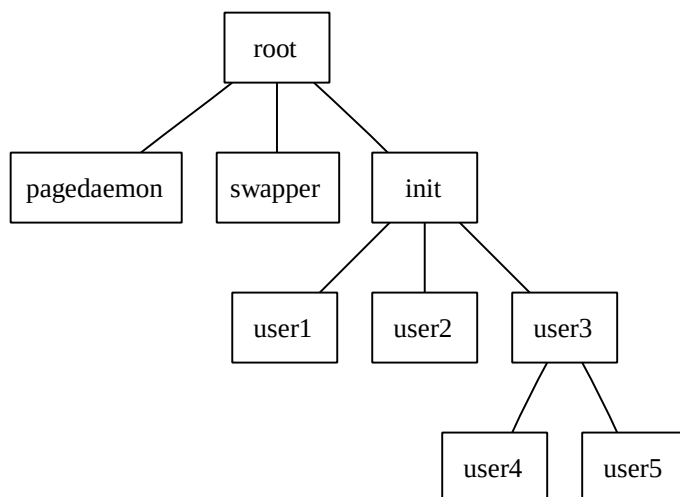
3.4 Procesy v systému Unix

3.4.1 Hierarchie procesů

Během své činnosti mohou procesy vytvářet nové procesy. Vytvářející proces je nazýván *rodič* (*parent*), nově vytvářený proces se nazývá *potomek* (*child*). Každý proces má unikátní nezáporné číslo – identifikátor (*PID*) a procesy jsou organizovány do stromové hierarchie, viz obrázek 3.6. Kromě běžných uživatelských procesů existují také procesy speciální:

- naprosto prvním procesem ($PID=0$) bývá *swapper*,
- předkem všech uživatelských procesů je *init* s $PID=1$,

- mezi další procesy jádra může patřit např. *pagedaemon*.



Obrázek 3.6: Strom procesů v typickém Unix systému.

Všechny uvedené speciální procesy, až na proces *init*, jsou součástí jádra, tzn. že jejich kód je součástí jádra (nemají svůj program uložený na disku). Proces *init* je normální proces, který je inicializován při startovací (bootstrap) proceduře a nikdy neumírá. Pokud procesu skončí předek, jeho předkem se automaticky stane *init*, který později převezme jeho návratový kód (proces nemůže definitivně skončit a jako *zombie* čeká, dokud neodevzdá návratový kód).

Pro získání základních informací o procesech slouží početná skupina funkcí, např. výpis stromu procesů `ps tree`, získání PID `getpid(void)`, získání PID rodičovského procesu (PPID) `getppid(void)` atd.

3.4.2 Vytvoření nového procesu

Jedinou cestou, jak v Unixu vytvořit nový proces, je použití systémového volání `fork()`. Služba `fork` vytvoří klon (duplikát) procesu, ze kterého je služba `fork` volána. Funkce se volá jednou, ale vrací dvě hodnoty – podle nich určí proces svou totožnost po provedení `fork`. Vrátil-li `fork` hodnotu `-1`, došlo k chybě (pravděpodobně byl překročen limit počtu souběžně běžících procesů. Návratová hodnota `0` indikuje, že proces testující tuto hodnotu je potomkem. Rodičovskému (parent) procesu vrátí `fork` identifikační číslo potomka. Po provedení služby `fork` provádějí rodič i potomek stejný kód a je čistě věcí programu, jak rozdvojení procesů ošetří (viz ukázka kódu na obrázku 3.7).

Služba `fork` slouží k vytvoření samotného procesu, naším cílem však je většinou vytvořit nový proces řízený jiným programem, než původní klonovaný proces. K tomuto účelu slouží skupina systémových volání `exec`:

- `execl("/bin/ls", "ls", "-l", NULL);`
První argument je cesta k programu, další argumenty reprezentují argumenty volaného programu `arg0` (`arg0` je vždy název programu), `arg1`

atd. Hodnota `NULL` identifikuje konec seznamu argumentů. Náš příklad odpovídá spuštění programu s jedním argumentem: `/bin/ls -l`.

- další varianty jsou `exec lp`, `exec le`, `exec v`, `exec vp`

Pokud je systémové volání `exec` úspěšné, nahradí obraz (kód, data atd.) aktuálního procesu novým obrazem vytvořeným na základě programu, který je zadán jako první argument funkce `exec`. Pokud dojde k chybě, vrací se řízení volajícímu procesu a služba `exec` končí s návratovou hodnotou identifikující chybu.

Následující kód reprezentuje příklad vytvoření nového procesu. Rodič čeká na dokončení svého potomka (viz volání `wait`), potomek nahradí svůj původní obraz procesu programem `ls`.

```
pid=fork();
if (pid==0) {
    // kód pro proces potomka
    // exec(...), exit(exitcode)
    // např.:
    execlp("/bin/ls", "ls", NULL);
} else if (pid==-1) {
    // kód pro rodiče, nastala chyba při fork()
    // errno obsahuje bližší informace
} else {
    // kód pro rodiče, pid = PID potomka
    pid2 = wait(&stav);
}
```

Obrázek 3.7: Ukázka vytvoření nového procesu v systému Unix.

V operačním systému *Windows* existuje služba `CreateProcess(...)` zahrnující funkčnost `fork` i `exec`.

3.5 Komunikace procesů

Procesy mohou mezi sebou komunikovat prostřednictvím prostředků, které se souhrnně označují jako meziprocesová komunikace (*Inter-Process Communication* – *IPC*). Mezi tyto prostředky patří zejména:

- signály (systémová volání `kill`, `signal`, ...)
- roury (systémová volání `pipe`, ...)
- zprávy (systémová volání `msgsnd`, `msgrcv`, ...)
- sdílená paměť (systémová volání `shmget`, `shmat`, ...)
- sockety (systémová volání `socket`, ...)
- RPC – Remote Procedure Call

 $x + y$


3.5.1 Signály

Signál je (v základní verzi) číslo (`int`) zaslané procesu prostřednictvím pro to zvláště definovaného rozhraní. Signály jsou generovány

- při chybách (např. aritmetická chyba, chyba práce s pamětí, ...),
- externích událostech (vypršení časovače, dostupnost I/O, ...),
- na žádost procesu – IPC (`kill`, ...).

Signály často vznikají *asynchronně* k činnosti programu, není tedy možné jednoznačně předpovědět, kdy daný signál bude doručen. Je nutné pečlivě zvažovat obsluhu, jinak mohou vzniknout “záhadné”, zřídka se objevující a velice špatně laditelné chyby (mj. také motivace pro *formální verifikaci*). Nejběžněji používané signály jsou shrnuty v tabulce 3.7. Další signály lze najít v manuálové stránce – viz příkaz `man -S7 signal`.



<code>SIGHUP</code>	odpojení, ukončení terminálu
<code>SIGINT</code>	přerušení z klávesnice (<code>Ctrl-C</code>)
<code>SIGKILL</code>	tvrdé ukončení
<code>SIGSEGV, SIGBUS</code>	chybný odkaz do paměti
<code>SIGPIPE</code>	zápis do roury bez čtenáře
<code>SIGALRM</code>	signál od časovače (<code>alarm</code>)
<code>SIGTERM</code>	měkké ukončení
<code>SIGUSR1, SIGUSR2</code>	uživatelské signály
<code>SIGCHLD</code>	pozastaven nebo ukončen potomek
<code>SIGCONT</code>	pokračuj, jsi-li pozastaven
<code>SIGSTOP, SIGTSTP</code>	tvrdé/měkké pozastavení

Tabulka 3.7: Přehled nejpoužívanějších signálů v systému Unix.

3.5.2 Obsluha signálů

S každým signálem, který je přijat procesem, je spojena jeho obsluha. Mezi implicitní reakce na signál patří *ukončení procesu*, *pozastavení procesu* a *ignorování signálu*. Implicitní obsluhu všech signálů lze uživatelsky předefinovat, kromě signálů `SIGKILL` a `SIGSTOP`. Obsluhu signálu `SIGCONT` je možné předefinovat, ale vždy dojde k odblokování procesu (toto implicitní chování tedy zrušit nelze, je možné je pouze doplnit).



Předefinování obsluhy signálu. K předefinování obsluhy signálů slouží funkce `signal` a `sigaction`:

- `sighandler_t signal(int signum, sighandler_t handler);`
 - `handler` je ukazatel na *obslužnou funkci*, příp. předdefinované hodnoty `SIG_DFL` (implicitní obsluha signálu) či `SIG_IGN` (ignorovat signál).

- `int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);`
 - možnost nastavení blokování během obsluhy,
 - nastavení režimu obsluhy (automatické obnovení implicitní obsluhy, ...),
 - nastavení obsluhy s příjmem dodatečných informací spolu se signálem.

Blokování signálů. Při nastavování obsluhy signálů je vhodné tyto signály blokovat. Pokud je procesu zaslán signál, který je blokován, uloží se tento signál do fronty, kde čeká na doručení. V okamžiku odblokování signálu je tento signál obslužen procesem. Smyslem blokování signálu je předejít situaci, kdy proces přijímá signál a dosud není korektně nastavena obsluha signálu – pokud by nebyl signál blokován, došlo by k jeho ztrátě (byl by obslužen implicitně a ne námi požadovanou rutinou). Jiná situace může nastat, pokud je obsluha nastavena a signál přijde v okamžiku, kdy se proces chystá čekat na signál, ale dosud se k čekání nedostal – signál je zpracován předčasně a může dojít k zablokování procesu, neboť proces začne čekat na signál, který již byl zpracován. K nastavení masky blokování signálů lze použít:



- `int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);`
- `how` je `SIG_BLOCK`, `SIG_UNBLOCK` či `SIG_SETMASK`.
- K sestavení masky signálů slouží funkce `sigemptyset`, `sigfillset`, `sigaddset`, `sigdelset` nebo `sigismember`.

Nelze blokovat signály `SIGKILL`, `SIGSTOP` a `SIGCONT`. Nastavení blokování se dědí do potomků. Dědí se také obslužné funkce signálů, při použití `exec` se ovšem nastaví implicitní obslužné funkce. Pokud je nějaký zablokovaný signál přijat vícekrát, zapamatuje se jen jedna instance! Toto neplatí pro tzv. real-time signály.

3.5.3 Zpracování signálů

Zasílání signálů. Proces může zaslat signál jinému procesu, skupině procesům nebo všem procesům (s výjimkou procesu `init`, pokud nemá pro příslušný signál definovanou obsluhu). Aby proces mohl zaslat signál jinému procesu, musí odpovídat jejich UID nebo EUID (`SIGCONT` lze zasílat všem procesům v sezení – session), případně se musí jednat o privilegovaného odesílatele (např. `EUID=0`).

Pro zasílání signálů existuje systémové volání `int kill(pid_t pid, int sig);`.

Čekání na signál Proces může na signál čekat (tím se dá programovat jednoduchá synchronizace procesů). Existují dvě základní varianty, jak čekat na signál:

- *Jednoduché čekání*
 - `int pause(void);`

- nelze spolehlivě nastavit signály, které mají být blokovány mimo dobu, kdy se čeká.
- *Zabezpečené čekání*
 - `int sigsuspend(const sigset_t *mask);`
 - lze spolehlivě přepínat mezi signály blokovánými mimo a po dobu čekání,
 - `mask` jsou blokovány po dobu čekání, po ukončení se nastaví původní blokování.

Příklad použití signálů Uvažujme proces, který vytvoří potomka a čeká na signál od něj. Nejprve definujeme proměnnou `interrupt`, jejíž hodnota se změní po příchodu signálu. Funkce `synch_signal` zpracovává přichozí signál. Funkce `child_function` představuje kód potomka. Důležitou částí z hlediska našeho příkladu je zaslání signálu `SIGUSR1` svému předkovi – viz použití příkazu `kill(getppid(), SIGUSR1)`.

 $x+y$

```
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

/* When a SIGUSR1 signal arrives, set this variable. */
volatile sig_atomic_t usr_interrupt = 0;

void synch_signal (int sig) {
    usr_interrupt = 1;
}

/* The child process executes this function. */
void child_function (void) {
    ...
    /* Let the parent know you're here. */
    kill (getppid (), SIGUSR1);
    ...
}
```

Ve funkci `main` nastavíme obsluhu signálu `SIGUSR1` pomocí `sigaction`. Poté vytvoříme potomka a v něm zavoláme funkci `child_function`, u které předpokládáme, že nikdy neskončí. Nyní zablokujeme signál `SIGUSR1`, tzn. že od tohoto okamžiku bude signál `SIGUSR1`, pokud by přišel, čekat na doručení. Čekání na signál je realizováno v cyklu `while` – testuje se proměnná `usr_interrupt`. Pokud není proměnná nastavena, dosud nedošlo ke zpracování signálu a můžeme začít čekat – viz funkce `sigsuspend`. Při volání funkce `sigsuspend` dojde k odblokování příslušných signálů. Volání funkce `sigsuspend` zablokuje proces, dokud proces nepřijme libovolný signál. Proto

je nutné čekání provádět v cyklu – pokud by byl přijat jiný signál, nedojde k nastavení proměnné a proces znovu čeká. Při příjmu signálu se vyvolá jeho obsluha, po ukončení obsluhy se opět zablokují příslušné signály a funkce `sigsuspend` skončí.

```
int main (void) {
    struct sigaction usr_action;
    sigset_t block_mask, old_mask;

    /* Establish the signal handler. */
    sigfillset (&block_mask);
    usr_action.sa_handler = synch_signal;
    usr_action.sa_mask = block_mask;
    usr_action.sa_flags = 0;
    sigaction (SIGUSR1, &usr_action, NULL);

    /* Create the child process. */
    if (fork() == 0) child_function ();

    /* Wait for the child to send a signal */
    sigemptyset (&block_mask);
    sigaddset (&block_mask, SIGUSR1);
    sigprocmask (SIG_BLOCK, &block_mask, &old_mask);
    while (!usr_interrupt)
        sigsuspend (&old_mask);
    sigprocmask (SIG_UNBLOCK, &block_mask, NULL);
    ...
}
```

Pokud by signál `SIGUSR1` nebyl blokován, mohlo by dojít k následující situaci. Proces otestuje proměnnou `usr_interrupt`, zjistí, že nebyla nastavena a pokračuje v čekání. V okamžiku po otestování a před voláním funkce `sigsuspend` přichází signál, je zpracován a proměnná `usr_interrupt` je nastavena. Poté začne proces čekat na signál, který již přišel a z logiky naší aplikace vyplývá, že již nikdy nepřijde – proces se zablokuje.



3.5.4 Roury

Roury (pipes) jsou nejstarším komunikačním prostředkem v Unixu. Jsou podporovány všemi typy operačních systémů založených na systému Unix za těchto omezujících podmínek:

- jedná se o poloviční duplex – data putují jen jedním směrem,
- mohou být použity pouze v případě, že procesy mají společného předchůdce.

Roura se vytvoří vyvoláním funkce `pipe(int filedes[2])`. Argumentem funkce je pole o dvou prvcích, kam se uloží deskriptory proudů určeným

pro čtení (`filedes[0]`) a zápis (`filedes[1]`). Číst z roury a zapisovat do roury lze např. prostřednictvím služeb `read` a `write` s využitím příslušných deskriptorů. Jestliže čteme z roury, jejíž vstup byl uzavřen, `read` vrátí 0. Jestliže zapisujeme do roury, kterou nikdo nečte, generuje se signál `SIGPIPE`.

Použití roury si ukážeme na jednoduchém příkladu. Proces nejprve vytvoří rouru a poté vytvoří svého potomka. Potomek dědí deskriptory roury. Rodičovský proces poté uzavře proud určený pro čtení (nepotřebuje ho), zapíše do roury řetězec „*hello world*“ a skončí. Potomek uzavře proud pro zápis, přečte z roury řetězec a zapíše ho na standardní výstup. Poté se také ukončí.

 $x + y$

```
int main(void)
{
    int      n, fd[2];
    pid_t    pid;
    char     line[MAXLINE];

    if (pipe(fd) < 0)
        err_sys("pipe error");

    if ( (pid = fork()) < 0)
        err_sys("fork error");

    else if (pid > 0) {          /* parent */
        close(fd[0]);
        write(fd[1], "hello world\\n", 12);

    } else {                    /* child */
        close(fd[1]);
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }

    exit(0);
}
```

3.6 Shrnutí

V této kapitole jsme se věnovali konceptům správy procesů v operačních systémech. Seznámili jsme se s pojmy proces a vlákno. Diskutovali jsme problematiku plánování procesů a přiblížili jsme si vybrané plánovací algoritmy. Zaměřili jsme se také na organizaci procesů a práci s procesy v operačním systému Unix. Na závěr jsme si přiblížili problematiku meziprocsovou komunikace, přičemž jsme si podrobněji popsali dva způsoby komunikace mezi procesy – signály a roury.

 Σ

3.7 Kontrolní otázky

1. Co je cílem plánovače procesů v operačních systémech?
2. Specifikujte pojmy preemptivní a nepreemptivní plánování procesů. Jaký je základní rozdíl mezi těmito způsoby plánování?
3. Vyjmenujte a charakterizujte základní plánovací algoritmy.
4. Jaké jsou základní vlastnosti plánovacích algoritmů?
5. Jaké jsou problémy spojené s prioritními plánovači?
6. Čím je konceptuálně reprezentován proces? Stručně charakterizujte.
7. Co je přepnutí kontextu?
8. Jak jsou organizovány procesy v operačním systému Unix? Vysvětlete způsob vytváření nových procesů.
9. Co jsou to signály? Vysvětlete jakým způsobem se signály zpracovávají.
10. Proč je nutné blokovat signály?



Kapitola 4

Virtuální paměť



4h

Cílem této kapitoly je vysvětlení principu virtuální paměti založené na stránkování na žádost (demand paging) včetně některých souvislostí s programováním aplikací pro tyto systémy.

Virtuální paměť je technika, která umožňuje provádění procesů, které nejsou celé zavedeny do paměti. Výhodou tohoto přístupu je to, že programy mohou používat více paměti než je v počítači nainstalováno. Další výhodou je abstrakce - paměť každého procesu je velmi velké souvislé pole bajtů nezávislé na množství fyzické paměti instalované v počítači. To dovoluje programátorům programovat bez znalosti velikosti fyzické paměti. Navíc virtuální paměť umožňuje snadné sdílení paměti mezi procesy. Implementace systému virtuální paměti není jednoduchá a vyžaduje speciální technické vybavení. Při nevhodném způsobu programování aplikací, nebo při přetížení může virtuální paměť způsobit značné zpomalení výpočtu.

Pro zvládnutí této kapitoly je třeba znát princip práce procesoru a základy hierarchie paměti v počítači. Je nutná alespoň základní znalost pojmů procesor, paměť, vyrovnávací paměť, instrukce, adresa, přerušení a příslušných souvislostí.

4.1 Adresové prostory

Virtualizace paměti je spojena s pojmy logický a fyzický adresový prostor. Adresový prostor je množina všech použitelných adres.

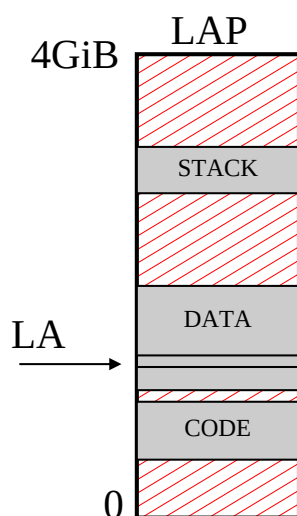
Logický adresový prostor (zkratka: LAP), někdy též *virtuální adresový prostor* je množina všech možných adres použitelných procesem. Každý proces (i jádro) má svůj vlastní logický adresový prostor, který je typicky zcela oddělen od LAP ostatních procesů v systému.

Velikost logického adresového prostoru je dána rozsahem adres v instrukcích a pro 32 bitovou architekturu je typicky 2^{32} bajtů, tj 4GiB. To ale neznamená, že proces může využít celý rozsah. Obvykle jsou v LAP „zakázané“ oblasti, které z různých důvodů nelze využít. Tyto oblasti jsou obvykle chráněny a při jakémkoli přístupu na tyto adresy dojde k přerušení které musí být obslouženo operačním systémem.

Příklad umístění různých oblastí v LAP je uveden na obrázku 4.1. Přesné rozložení sekcí pro kód, data a zásobník v logickém adresovém prostoru je dáno použitým operačním systémem¹.

¹Rozložení sekcí je obvykle komplikovanější, protože se používají sdílené knihovny, sdílená paměť atd. Informace o paměťovém prostoru procesu můžeme získat například v Linuxu ze souboru `/proc/<PIDprocesu>/maps`.





Obrázek 4.1: Příklad logického adresového prostoru procesu

Oblast kolem adresy 0 je typicky chráněna proti přístupu, aby bylo možné odhalit programátorské chyby při práci s ukazateli (například přístup přes NULL ukazatel). Oblast kódu je typicky celá chráněna proti zápisu, aby nemohlo dojít k jeho přepisování, což je zvláště důležité při sdílení kódu několika procesy.

Fyzický adresový prostor (FAP) je paměťový prostor daný použitou operační pamětí RAM v počítači. FAP je v běžném počítači jen jeden a je společně používán procesy i jádrem operačního systému. Jeho maximální velikost je omezena architekturou počítače. Počet bitů fyzické adresy nemusí být stejný jako u logické adresy. Například moderní 32-bitové procesory Pentium mají 36-bitovou adresovou sběrnici, 64-bitové procesory dnes obvykle vystačí se 40-48 bity fyzické adresy. Je důležité si uvědomit, že velikost fyzické adresy nemá vliv na programování aplikací, protože ty používají pouze logické adresy. Pouze jádro operačního systému (OS) a některé specializované programy vyžadují znalost fyzického adresového prostoru. Podrobnější informace o technickém vybavení naleznete v literatuře.

DEF

4.2 Princip virtualizace paměti

Systém virtuální paměti musí zajistit, aby při provádění procesu každý přístup na platnou adresu do LAP automaticky zajistil přítomnost požadovaných dat ve fyzické paměti, kde s nimi procesor může manipulovat. Zpřístupnění fyzické paměti se provádí tzv. mapováním logických adres na fyzické². Není nutné aby celý logický adresový prostor byl vždy celý mapován do fyzické paměti. Jednotlivé části LAP se do FAP mapují až podle potřeby (on demand). Tento přístup zajiš-

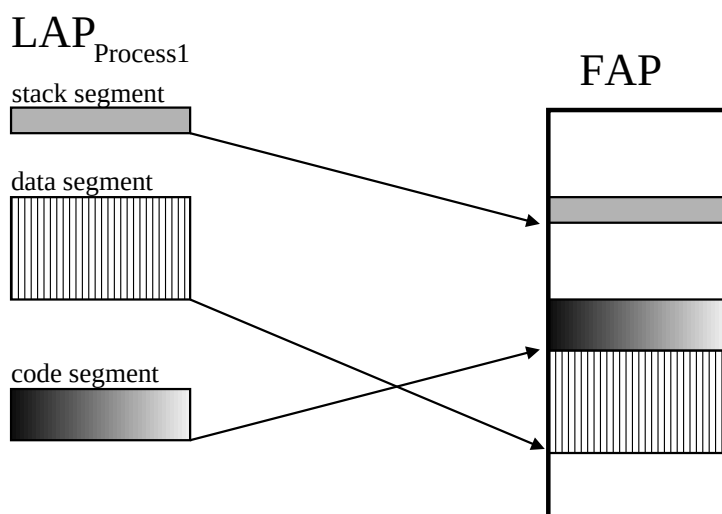
²Je důležité si uvědomit, že mapování neznamená kopírování obsahu paměti ale pouze transformaci adres.

ťuje lepší využití fyzické paměti.

Virtuální paměť dovoluje použít speciální odkládací prostor na disku ke zvětšení celkového rozsahu použitelné paměti. Do odkládacího prostoru se ukládají dočasně nepotřebné části fyzické paměti a v případě potřeby se opět zavedou zpět (většinou na jiné místo).

Způsobů jak implementovat systém virtuální paměti je několik:

- Segmentace (segmentation) – LAP je tvořen několika segmenty – oblastmi paměti obecně různé velikosti (až do limitu daného rozsahem adres – offsetů v segmentu). Program při tomto způsobu virtualizace musí obsahovat informace ve kterém segmentu leží požadovaná data nebo kód. Pozice segmentů ve FAP je dána tzv. segmentovými registry, kterých je omezený počet (například architektura x86 má 4 použitelné segmentové registry: DS, CS, SS, ES). Pokud proces potřebuje více paměti než je maximální velikost segmentu musí dynamicky přepínat mezi jednotlivými segmenty. Například 16bit adresace dovoluje segmenty o velikosti maximálně 64KiB, což je pro větší programy příliš málo. Příklad ilustrující použití segmentů je uveden na obrázku 4.2.



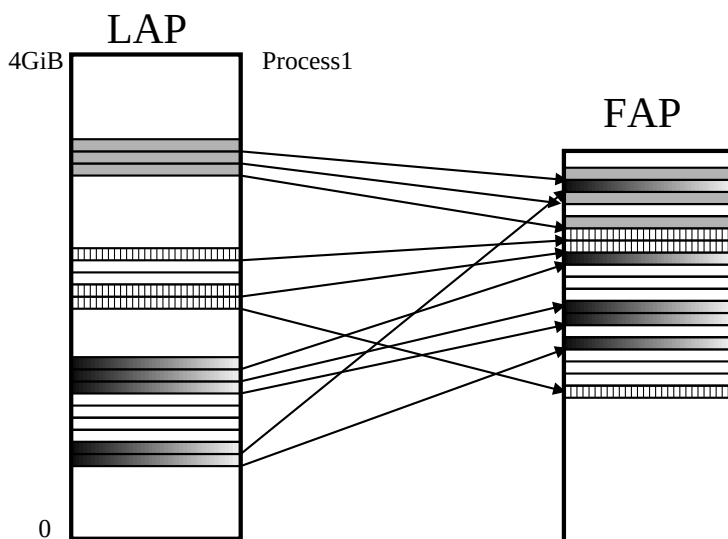
Obrázek 4.2: Překlad adres při segmentaci

Protože se musí v programu specifikovat přepínání segmentových registrů, je tento přístup náročnější na programátora než stránkování. Různá velikost segmentů může způsobovat fragmentaci ve fyzické paměti. Výhodou segmentace je jednoduchá technická realizace. V současné době se tento systém virtualizace paměti již prakticky nepoužívá.

- Stránkování na žádost (Demand paging) provádí mapování z LAP do FAP



po stránkách (souvislé oblasti paměti pevné velikosti – například 4KiB³). Stránka je nejmenší jednotkou přidělování paměti procesu a její velikost musí být mocninou 2. Potom si adresový prostor můžeme představit jako pole stránek, jehož indexy odpovídají číslu stránky, které se dá snadno „vypočítat“ operací výběru horních několika bitů adresy. LAP i FAP jsou rozděleny na stejně velké stránky. Stránce ve FAP obvykle říkáme rámce (frame). Stránka je také nejmenší jednotkou pro kterou lze nastavit ochranu paměti (právo číst, zapisovat, provádět kód) a sdílení paměti mezi procesy. Příklad ilustrující mapování stránek na rámce je na obrázku 4.3. Některé stránky nelze mapovat vůbec (nejsou zobrazeny), některé jsou aktuálně mapovány a některé mohou být mapovány později, nebo vůbec, pokud nebude přistupováno na adresy ležící v těchto stránkách. Každý proces má vlastní mapování a ve fyzické paměti jsou i rámce přidělené jiným procesům.



Obrázek 4.3: Překlad adres při stránkování

Důležitou vlastností tohoto systému virtuální paměti je to, že proces se nemusí starat o mapování LAP na FAP, protože toto mapování je transparentně (tj. pro proces neviditelně) zajišťováno kombinací technického vybavení (hardware, HW) a jádra OS. Nevýhodou je vyšší složitost implementace a větší režie. Tento způsob virtualizace paměti v současnosti používají všechny moderní víceúlohové operační systémy.

- Existují i jiné přístupy, například kombinace segmentace + stránkování, o kterých se zde nebudeme zmiňovat.

Všechny uvedené přístupy dovolují použít pomalejší ale levnější paměť (typicky disk) pro zvýšení množství dostupné paměti pro procesy. Velikost FAP je dána

³Na 64bitových architekturách mají stránky obvykle 8KiB, na některých systémech je možné používat i speciální stránky o velikosti 2 nebo 4MiB.

kapacitou instalované operační paměti (RAM) v počítači. Protože RAM je drahá, nebývá jí k dispozici mnoho (ve srovnání s kapacitou disků) a nároky všech současně spuštěných procesů bývají často vyšší. Proto využíváme levné diskové paměti k odkládání dočasně nepotřebných částí FAP, abychom uvolněnou paměť mohli použít pro další požadavky procesů. Tento systém dobře funguje díky typické vlastnosti většiny programů – dobré lokalitě odkazů. Program s dobrou lokalitou odkazů při své činnosti v určitém časovém intervalu potřebuje jen malý počet stránek, protože se odkazuje stále do stejných oblastí paměti (například provádění cyklu čekajícího na vstup/výstup znamená, že je stále čtena jen velmi malá část paměti z oblasti kódu).

Výhody virtuální paměti můžeme obecně shrnout do následujících bodů:

- Lze spustit programy, které zaberou více než je kapacita fyzické paměti RAM.
- Lze snadno zajistit sdílení částí paměti různými procesy.
- Oddělení adresových prostorů procesů automaticky zajišťuje ochranu paměti (procesy se navzájem neovlivňují). Je také možné nastavit ochranu individuálně pro jednotlivé segmenty/stránky v rámci LAP procesu.

4.3 Princip stránkování na žádost

Stránkování na žádost je jeden z možných způsobů virtualizace paměti. Dnes je využíván ve většině operačních systémů. Princip stránkování na žádost si znázorníme na příkladu systému s 32 bitovou logickou adresou (4GB virtuální adresový prostor), který provádí instrukci zápisu do paměti. Paměťové operace spojené se čtením instrukce z kódové oblasti záměrně vynecháme a zaměříme se pouze na provedení instrukce. Postup je ilustrován obrázky.



1. Přístup do paměti

Předpokládáme příkaz

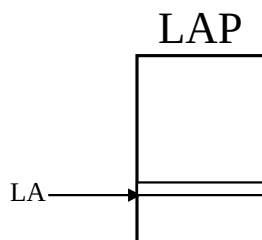
```
a=1;
```

který překladač přeloží například⁴ na instrukci

```
mov [LA], 1
```

kde LA je logická adresa globální neinicializované proměnné a typu `int`. Při provádění instrukce pošle CPU tuto logickou adresu LA na adresovou sběrnici.

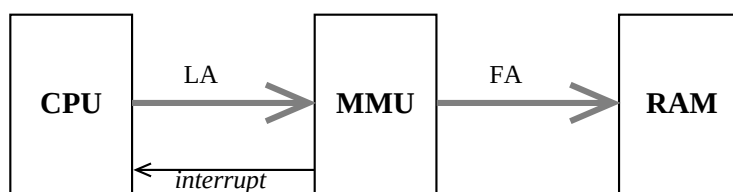
⁴Předpokládáme architekturu běžných osobních počítačů kategorie PC.



Obrázek 4.4: Přístup do paměti

2. Překlad adresy

Na adresové sběrnici je připojeno speciální technické vybavení – tzv *jednotka správy paměti* (Memory Management Unit, MMU), která transformuje logickou adresu LA na fyzickou adresu FA, která je dále předávána operační paměti (RAM). MMU je u moderních procesorů na jednom čipu spolu s CPU a vyrovnávacími paměťmi (cache).



Obrázek 4.5: Činnost MMU

Tato jednotka provádí transformaci adres, pro kterou se používá termín *překlad adres* (address translation). Tato činnost je prováděna podle informací uložených do speciální datové struktury nazvané *tabulka stránek* (Page Table). Při překladu logické adresy (LA) na fyzickou (FA) mohou nastat dvě možnosti:

- (a) Překlad proběhne úspěšně a instrukce může být dokončena. Tento případ nastává nejčastěji a je rychlý, protože nevyžaduje žádnou činnost operačního systému.
- (b) Uvedenou logickou adresu LA nelze převést na FA (to je typické například pro první přístup do příslušné stránky paměti). V takovém případě MMU signalizuje procesoru tento problém přerušením (interrupt). Toto přerušování se nazývá *výpadek stránky* (Page fault). Jde o speciální typ přerušování, protože prováděnou instrukci nelze dokončit. Procesor na toto přerušování reaguje vyvoláním podprogramu pro obsluhu výpadku stránky, který je součástí jádra OS.

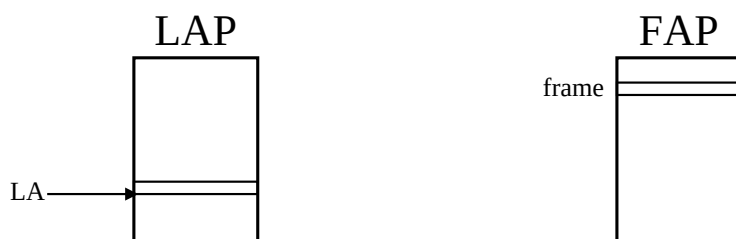
3. Obsluha výpadku stránky

V rámci obsluhy přerušení od MMU podsystém virtuální paměti v jádře OS zkontroluje zda odkaz do paměti je platný (tj vede do přístupné oblasti LAP) a pokud není, je proces ukončen příslušným signálem.

Pokud je LA platná, musí být vytvořeno mapování stránky do které odkazuje LA na rámec ve fyzické paměti. Pro jednoduchost budeme nyní uvažovat pouze jednu z možných situací: *první přístup* k neinicializovaným datům.

(a) Alokace rámce (fyzické stránky)

Výpadek stránky znamená, že ve fyzické paměti není požadovaná stránka s příslušným obsahem přítomna. Proto prvním krokem musí být získání volného místa pro novou stránku ve FAP. Podsystém správy paměti si udržuje datové struktury informující o volném místě ve FAP a s využitím vhodných algoritmů získá adresu volného rámce a přidělí jej přerušenému procesu.



Obrázek 4.6: Alokace rámce

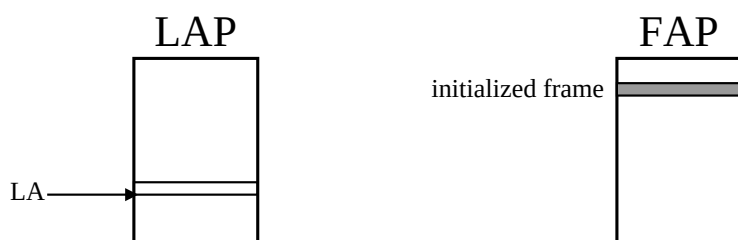
(b) Naplnění rámce daty

Po alokaci rámce ve FAP je nutné zajistit, aby v něm byla aktuální data (tj například hodnoty proměnných procesu). V našem případě jde o neinicializovaná data a první přístup k nim, proto je celá stránka vynulována. Neinicializované proměnné tím automaticky získají počáteční hodnotu 0. Zde je třeba si uvědomit, že každá kategorie proměnných je překladačem umístěna do jiné části logického adresového prostoru, proto tato stránka obsahuje pouze neinicializované proměnné. Jiné oblasti paměti vyžadují odlišný přístup, který bude uveden až v následujícím textu.

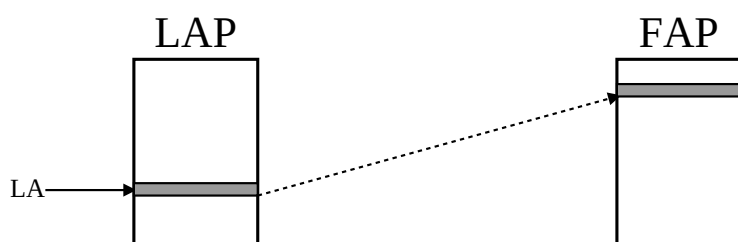
(c) Aktualizace tabulky stránek

Po alokaci rámce a jeho naplnění daty je ještě nutné aktualizovat odpovídající záznam v tabulce stránek příslušného procesu, aby bylo vytvořeno mapování logické stránky na rámec. Tím je zajištěno, že od tohoto okamžiku bude MMU schopna přeložit všechny logické adresy v rozsahu dané stránky.

4. Po dokončení obsluhy výpadku stránky je proces připraven a v okamžiku, kdy plánovač rozhodne o jeho pokračování je *znovu provedena stejná in-*

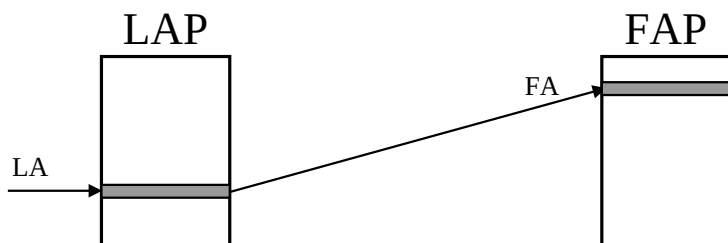


Obrázek 4.7: Inicializace rámce



Obrázek 4.8: Aktualizace tabulky stránek

strukce, která výpadek stránky způsobila. Nyní už MMU přeloží adresu bez problémů a výpočet může pokračovat prováděním další instrukce.



Obrázek 4.9: Přístup do paměti proveden

Z uvedeného příkladu (ve kterém chybí řada detailů) je zřejmé, že virtualizace paměti vyžaduje podporu operačního systému (protože proces nemůže přímo pracovat s FAP) a technického vybavení (protože překlad adres musí být rychlý). Nyní postupně upřesníme výše uvedený základní princip.

4.4 Tabulky stránek

Prvním problémem se kterým se setkáme při překladu adres je nutnost udržovat informace o mapování LA–FA v odpovídající datové struktuře. Pokud bychom udržovali tyto informace pro každý bajt, bylo by to velmi paměťově neefektivní. Proto se používá jako minimální jednotka jedna *stránka* (page), která je tvořena

souvislou posloupností N bajtů. Velikost stránky je vždy mocninou čísla 2. Typická velikost je na 32 bitových architekturách $2^{12} = 4096$ bajtů, což znamená cca 1 milion stránek v LAP.

Tabulka stránek je datová struktura která umožní rychlé vyhledání záznamu podle zadaného čísla logické stránky. Typicky je podobná poli, které je indexované číslem logické stránky. Každý záznam v tabulce stránek obsahuje tyto položky:



- Příznak přítomnosti stránky ve FAP – tento bit používá MMU při rozhodování zda vyvolat výpadek stránky.
- Číslo rámce ve FAP kde se nachází obsah stránky.
- Bity popisující přístupová práva do stránky (rwx⁵) slouží např. k ochraně kódových stránek proti přepsání (například při chybně napsaném programu).
- Další údaje nutné pro fungování virtuální paměti – obvykle příznaky že do stránky bylo přistupováno (access bit) a že byla modifikována (dirty bit).

Tabulky stránek jsou obvykle celé uloženy ve FAP⁶. MMU obsahuje speciální registr s ukazatelem na začátek tabulky stránek pro aktuální proces. Přepínání procesů vyžaduje změnu obsahu tohoto registru. Tabulky stránek můžeme implementovat různými způsoby:

- Jednoúrovňové tabulky stránek:

Způsob překladu logické adresy s využitím jednoduché tabulky stránek znázorňuje obrázek 4.10. Protože stránka má 2^{12} bajtů, je spodních 12 bitů LA beze změny převedeno do dolních 12 bitů FA. Zbytek LA se použije jako index do tabulky stránek a v ní uložené číslo rámce se použije jako horní bity výsledné FA.

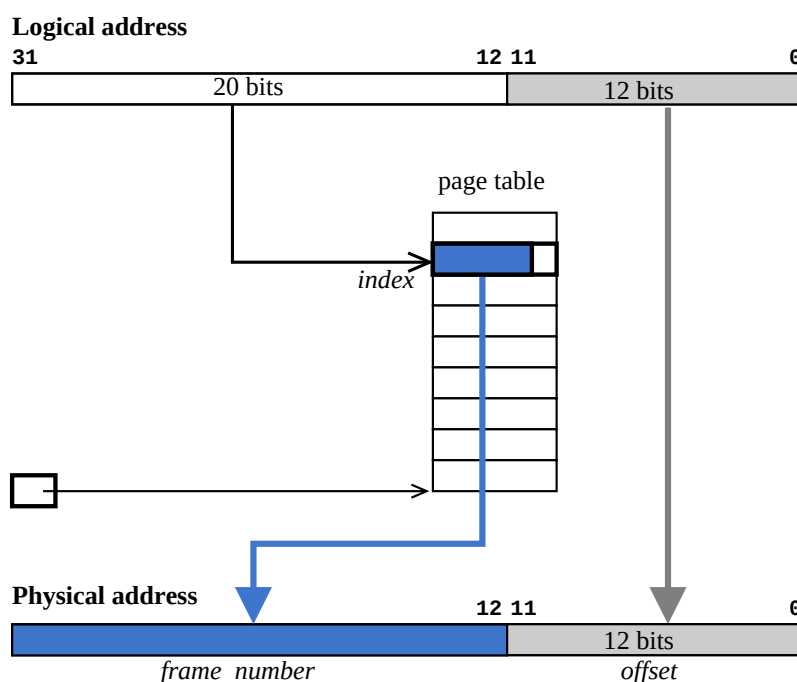
Uvažujeme-li jednoduchou tabulku stránek, znamená to, že pro každý proces bychom potřebovali tabulku s velkým počtem položek. Například pro 32 bitový systém se stránkami o velikosti 4KB (2^{12}) bude tabulka mít až milion položek ($2^{32}/2^{12}$). Má-li položka tabulky stránek 4B, dostaneme 4MB na tabulku stránek pro každý proces. Protože ale není LAP obsazen (kódem, daty, zásobníkem) souvisle, je využita pouze část tabulky stránek a mít tuto tabulku uloženu v paměti kompletní je velmi neefektivní. Proto se jednoduché tabulky stránek prakticky nepoužívají a místo nich jsou nejčastěji používány hierarchické tabulky stránek, které uspoří paměť, protože nemusí mít položky pro všechny stránky LAP.

- Hierarchické tabulky stránek:

Hierarchické tabulky stránek jsou rozděleny na menší části indexované menší tabulkou, která také může být rozdělena atd.

⁵Některé implementace nepoužívají všechna uvedená přístupová práva – například na i386 nelze chránit stránku proti provádění kódu v ní uloženého.

⁶Existují i operační systémy, které dovolují odkládat části tabulek stránek na disk.



Obrázek 4.10: Jednoúrovňová tabulka stránek

- Dvouúrovňové (běžné na 32bitových architekturách)

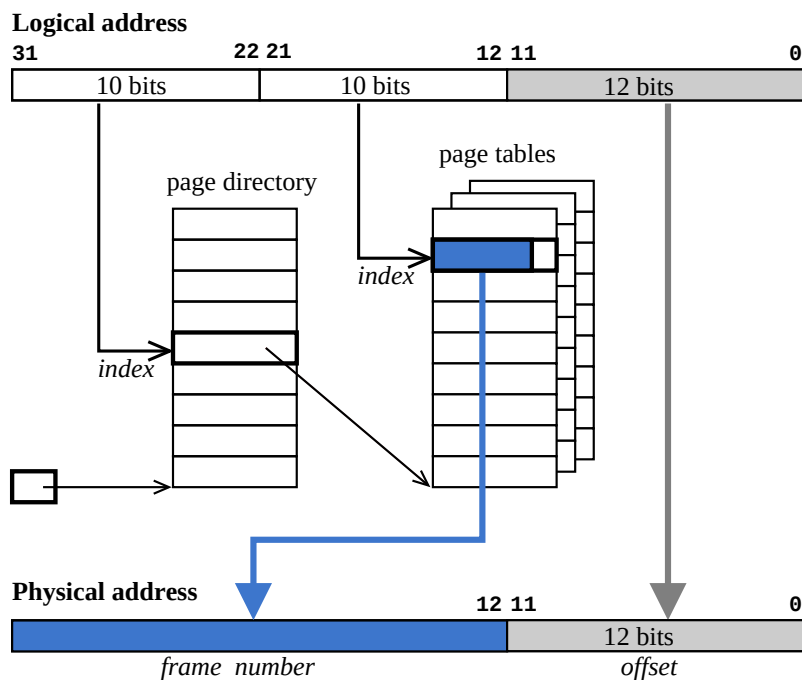
Tento typ tabulky stránek je používán na 32 bitových procesorech počínaje i386 – viz obrázek 4.11. Využívá se pomocná tabulka odkazů na menší části tabulky stránek (page directory) a je možné už v této pomocné tabulce poznačit, že daná část tabulky stránek nedefinuje žádné mapování. Protože většina programů potřebuje pouze několik procent celkového rozsahu adresového prostoru, jde o významnou úsporu paměti. Například pro malý program (kód, data i zásobník nemají více než 4MiB) potřebujeme jen 3 části tabulky po 1024 odkazech a pomocnou tabulku, celkem $4 \cdot 4\text{KiB} = 16\text{KiB}$ paměti.

- Víceúrovňové tabulky stránek jsou používány na 64bit architekturách, ale příliš velký počet úrovní není efektivní. Například architektura x86-64 (AMD64) používá maximálně čtyři úrovně.

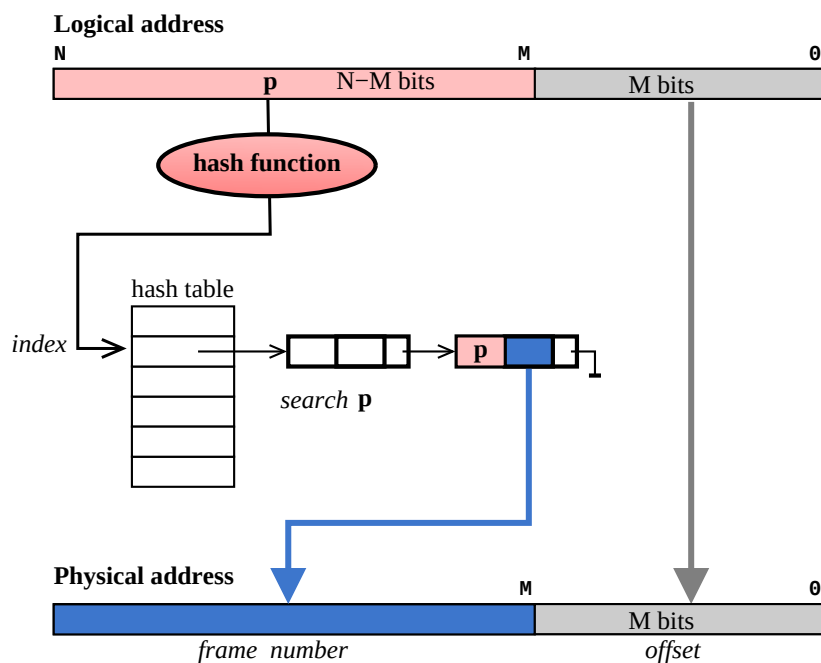
- Hashované tabulky stránek:

Tyto tabulky využívají pro zmenšení paměťové náročnosti pole (menší než odpovídající jednoúrovňová tabulka) indexované nikoli číslem stránky, ale výsledkem hashovací funkce která dostane číslo stránky a převede jej na index. Protože tato funkce může způsobit tzv. kolize (pro různé logické stránky vypočte stejný výsledek), musí být možné prohledáváním seznamu záznamů nalézt správný záznam. Proto je v položkách tohoto typu tabulek stránek uloženo i číslo logické stránky. Tyto tabulky stránek jsou používány na některých 64 bitových systémech.





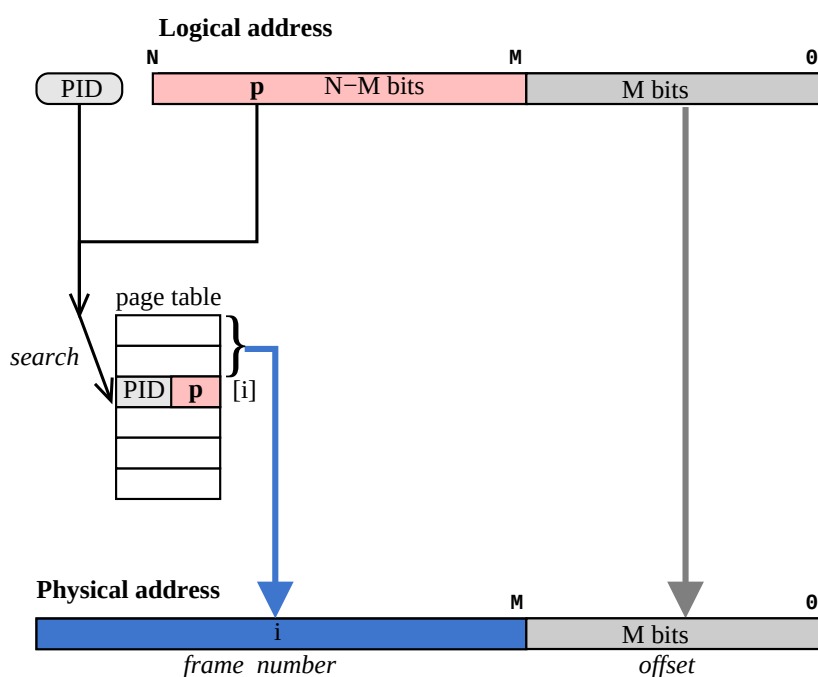
Obrázek 4.11: Dvouúrovňová tabulka stránek procesorů i386



Obrázek 4.12: Hashovaná tabulka stránek

- Invertované tabulky stránek:

Tyto tabulky jsou speciální případ – existuje pouze jedna tabulka pro FAP (pro každý rámec definuje, který proces do něj má namapovanu kterou stránku). Tento přístup vyžaduje další pomocné datové struktury pro každý proces a rychlé vyhledávání v tabulce. Výsledkem úspěšného vyhledání je index udávající pozici záznamu v tabulce – viz obrázek 4.13. Problematická je především implementace sdílení stránek. Podrobnosti naleznete v literatuře.



Obrázek 4.13: Invertovaná tabulka stránek

4.5 Překlad adres, MMU

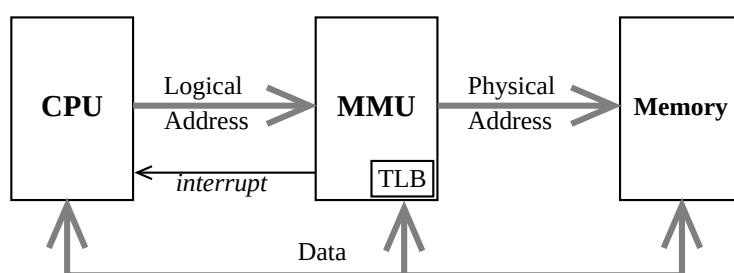
Nyní se zaměříme na činnost MMU a čas potřebný pro jeden přístup do paměti. Překlad adres způsobuje zpomalení přístupu, protože musí proběhnout nalezení a přečtení obsahu položky v tabulce stránek. To znamená, že MMU musí mít přístup do fyzické paměti (RAM), kde jsou uloženy tabulky stránek. Po přečtení příslušné položky tabulky stránek je použit její obsah v MMU. Podle obsahu bitu udávajícího přítomnost stránky ve FAP se provedou následující operace:

- Pokud není bit přítomnosti nastaven – MMU vyvolá přerušení (PageFault) a žádný přístup do paměti neproběhne.
- Pokud je bit přítomnosti nastaven a stránka má vyhovující přístupová práva, provede MMU překlad LA na FA. Překlad znamená kopii dolních N bitů



logické adresy (pro velikost stránky 2^N) a do zbývajících horních bitů výsledné FA je okopírováno číslo rámce z aktuální položky tabulky stránek. Výsledná FA je odeslána na adresovou sběrnici operační paměti (RAM) – viz obrázek 4.14. Podle dalších signálů řídicích přístup do paměti proběhne čtení nebo zápis dat po datové sběrnici.

Pokud jde například o operaci zápisu a stránka má nastaven bit ochrany proti zápisu, dojde k přerušení podobně jako při výpadku stránky, které musí obsloužit jádro OS a proces ukončit (poslat procesu příslušný signál).



Obrázek 4.14: Překlad adres

Nalezení položky v jednoduché tabulce stránek je stejně náročné jako indexování pole a čtení jedné položky – to znamená jeden přístup do paměti navíc. Vlastní činnost MMU také vyžaduje určitý čas. Z toho by vyplývalo více než dvojnásobné zpomalení každého přístupu do paměti v porovnání se systémem bez překladu adres. Pro víceúrovňové tabulky stránek je situace ještě horší. Protože takové zpomalení je nepříjemné, využívají jednotky MMU speciální vyrovnávací paměť (TLB) pro zrychlení přístupu k tabulce stránek.

4.5.1 TLB

TLB (Translation Look-aside Buffer) je asociativní paměť⁷ do které jsou ukládány dvojice (klíč, hodnota), kde klíčem je číslo logické stránky a hodnotou je odpovídající položka tabulky stránek. Asociativní paměť je schopna po zadání klíče velmi rychle nalézt odpovídající hodnotu, nebo signalizovat její nepřítomnost.

MMU s TLB funguje tak, že po obdržení logické adresy provede prohledání TLB (klíčem je číslo logické stránky). Podle výsledku vyhledávání provede různé akce:

- nalezeno (TLB hit): MMU ihned provede překlad (velmi rychlé)
- nenalezeno (TLB miss):

1. čte z tabulky stránek příslušnou položku,

⁷ Asociativní paměť je technické vybavení, které přistupuje k datům podle obsahu části dat (tzv klíče) – nezadáme adresu.

2. přepíše položku v TLB. Pokud není volné místo, musí být implementována vhodná strategie výběru položky, která bude přepsána novou položkou. Tato činnost je běžně prováděna přímo technickým vybavením v MMU⁸.
3. provede překlad adres

Maximální počet položek v TLB je typicky omezen na několik desítek až stovek. Důvodem tohoto omezení je vysoká HW náročnost a tím i cena asociativní paměti. Obsah TLB je nutné zrušit při každém přepnutí kontextu (jen pokud se do TLB položek neukládá také PID procesu).

Důležitou charakteristikou práce MMU je kolik procent paměťových odkazů nalezne mapování v TLB a kolik ne. Typická úspěšnost TLB je více než 98 procent. Takto vysoká je proto, že obvykle se do paměti přistupuje postupně na následující adresy a vždy celá řada přístupů vede přes jednu položku tabulky stránek. Potom je pomalý pouze první přístup a všechny další už využijí rychlý přístup s využitím TLB. Například postupné procházení velkého pole 32 bitových čísel typu `int` vede na 1024 přístupů na stejnou 4KiB stránku a jen první z nich způsobí čtení z tabulky stránek (TLB-miss).

4.5.2 Efektivita překladu adres

Rychlost překladu adres velmi silně závisí na úspěšnosti TLB. Průměrná přístupová doba může být vyjádřena výrazem:

$$t = \alpha(\tau + \varepsilon) + (1 - \alpha)((N + 1)\tau + \varepsilon)$$

kde

- τ je vybavovací doba RAM,
- ε je vybavovací doba TLB,
- α je pravděpodobnost úspěšného vyhledání v TLB (*TLB hit ratio*)⁹,
- N je počet úrovní tabulky stránek ($N + 1$ přístupů odpovídá nutnosti překladu adresy a poté ještě načtení dat).

Příklad

Příklad výpočtu průměrného zpomalení způsobeného překladem adres v MMU s dvouúrovňovou ($N = 2$) tabulkou stránek:

Pokud trvá jeden přístup do paměti $\tau = 100ns$ a vyhledání v TLB $\varepsilon = 10ns$, je doba přístupu do paměti při nalezení položky v TLB 110ns a při nenalezení 310ns (musíme přičíst další dva přístupy do paměti nutné pro získání položky

⁸V některých procesorových architekturách je plnění TLB realizováno programem, což ještě více zpomalí případ neúspěšného hledání v TLB.

⁹Úspěšnost vyhledávání v TLB významně závisí na *lokalitě odkazů programu*. Lokalita odkazů je vlastnost programu vyjadřující míru toho, kolik různých stránek bude proces potřebovat v krátkém časovém úseku.

z tabulky stránek). Průměrná přístupová doba do paměti je potom při 98% úspěšnosti ($\alpha = 0.98$) TLB:

$$t = 0.98 * 110ns + (1 - 0.98) * 310ns = 114ns$$

Výsledné zpomalení proti systému bez MMU je tedy pouze 14 procent.

4.6 Odkládání stránek

V typickém víceúlohovém OS běží desítky procesů a postupně zabírají FAP – systém virtuální paměti musí řešit problém s jeho přeplněním. Když procesy vyžadují více paměti než máme ve FAP, musí systém virtuální paměti odkládat některé stránky do speciální odkládací paměti – obvykle na disk. Odkládáním se uvolní místo pro nové požadavky.

4.6.1 Lokalita odkazů

Typický program většinu času provádí jen malou část svého kódu (například řadicí program `sort` většinu času tráví v jedné funkci implementující řazení) a odkazy do paměti jsou často omezeny podobným způsobem. Této vlastnosti programu se říká *lokalita odkazů*. Dobře napsané programy se chovají tak, že pro jejich běh stačí v určitém časovém intervalu jen malý počet stránek současně mapovaných do FAP – říkáme že mají dobrou lokalitu odkazů.

Stránky, které proces potřebuje v určitém časovém intervalu tvoří tzv. *pracovní množinu* stránek. Počet stránek v pracovní množině bývá podstatně menší než celkový počet stránek používaných procesem za celou dobu jeho běhu. To dovoluje udržovat ve fyzické paměti jen potřebné stránky a dynamicky přidávat/odebírat další stránky podle potřeb procesu.

Příklad

Při ukládání matic v paměti po řádcích (typické pro jazyk C) je zpracovávání matice po sloupcích:

```
for (j = 0; j < MAX; j++)
  for (i = 0; i < MAX; i++)
    A[i, j] = 0;
```

mnohem méně výhodné než zpracovávání po řádcích:

```
for (i = 0; i < MAX; i++)
  for (j = 0; j < MAX; j++)
    A[i, j] = 0;
```

Rozdíl se projeví nejvýrazněji když je matice větší než fyzická paměť. Poznamenejme, že podobně na lokalitu odkazů reagují i vyrovnávací paměti procesorů, proto je třeba při programování na výkon náročných aplikací přihlížet k parametřům paměťového systému.

x+y

4.6.2 Výběr stránky pro uvolnění

Není-li při alokaci nového rámce ve FAP volné místo, vybereme některou stránku a její obsah odložíme na disk (jen pokud je to nutné), zrušíme její mapování v odpovídající tabulce stránek a máme místo pro novou stránku. Problémem je vhodný výběr takové stránky – ideální by byla ta, která už nebude nikdy třeba. Obvykle se využívá informací o předchozím používání stránek. Typicky jsou používány varianty algoritmu LRU (Least Recently Used) - nejdéle nepoužitá stránka je vybrána k odstranění z FAP. Problém těchto algoritmů je, že potřebují informace o používání stránek v čase. Typické implementace využívají jeden bit (access bit) v položce tabulky stránek udávající, že do stránky bylo přistupováno. Vhodným algoritmem je tato informace průběžně zpracovávána a slouží jako vstup pro přibližné varianty LRU algoritmu.

4.6.3 Swap

Odkládání na disk vyžaduje speciální paměť – obvykle diskovou oblast nebo soubor pro ukládání obsahu stránek. Tato oblast je z historických důvodů obvykle nazývána „swap“. Podsystem virtuální paměti si musí udržovat přehled o volném místě a místě uložení obsahu stránek ve vhodných datových strukturách.

Velikost této datové oblasti musí být dostatečná pro běh všech aplikací, které potřebujeme provozovat. Některé implementace stránkování na žádost vyžadují, aby swap byl větší než fyzická paměť (RAM). Někdy je možné vypnout odkládání a pracovat i bez swapu (v případě že máme dostatek paměti).

4.6.4 Souvislosti a problémy

Je třeba si uvědomit několik souvislostí:

- Při výpadku stránky se musí zkontrolovat, zda byl její obsah v minulosti odložen na disk a v takovém případě obnovit její obsah ze swapu.
- Ne každá stránka se musí odkládat. Například kódové stránky se nemění (jsou-li chráněny proti zápisu) a lze je vždy znovu načíst ze spustitelného souboru.

Dále je třeba zdůraznit některé problémy způsobené odkládáním stránek na relativně pomalý, ale levný disk:

- Operace s diskem trvají velmi dlouho ve srovnání s přístupem do paměti. Typická přístupová doba disků je řádově v milisekundách (10^{-3} s), přístup do paměti trvá obvykle několik nanosekund (10^{-9} s). Vidíme, že diskové operace jsou přibližně milionkrát pomalejší (z hlediska doby přístupu).
- Příliš velká pracovní množina stránek vede na časté odkládání/čtení stránek, což silně zpomaluje výpočet.
- Při celkovém přetížení systému virtuální paměti nastává tzv. „trashing“ – systém neustále odkládá a načítá stránky a většinu času se věnuje režii s tím

spojené. Tato situace je velmi nežádoucí, protože celkový výkon počítače velmi klesá.

4.7 Obsluha výpadku stránky

Nyní již máme dostatek informací, abychom mohli probrat obsluhu výpadku stránky¹⁰ podrobně se všemi detaily:



1. Kontrola, zda se neodkazujeme mimo přidělený adresový prostor. Pokud ano, dostane proces signál (typicky SIGSEGV), který jej implicitně ukončí. Je ale možné v rámci procesu obsloužit uvedený signál a pokračovat v činnosti procesu (například na jiném místě kódu, nebo se změněnou adresou).

2. Alokace rámce

Pro přidělování rámců existuje řada algoritmů. Všechny musí používat pomocné datové struktury (tabulku volných rámců, tabulku stránek) a provádět následující činnost:

- Použijeme volný rámec, pokud nějaký volný je. Tato varianta je velmi rychlá a proto se používají speciální postupy, aby vždy bylo v systému alespoň několik volných rámců k dispozici. Například pokud počet volných rámců klesne pod určitou mez, aktivuje se speciální proces *page daemon* (zloděj stránek), který běží tak dlouho, dokud neuvolní dostatečný počet rámců.

Udržovat určitý volný prostor ve fyzické paměti je zvláště výhodné v situacích, kdy je zapotřebí rychle přidělit menší počet stránek – například u interaktivních procesů nedochází ke zpoždění a uživatelské rozhraní reaguje rychleji na akce uživatele.

- pokud není k dispozici volný rámec, musíme některý uvolnit:
 - (a) Vybereme vhodný již obsazený rámec – tzv. „oběť“ (victim page), kterou odložíme na disk. Existuje několik používaných algoritmů výběru odkládaných stránek:
 - Algoritmus FIFO:

Výhodou tohoto algoritmu je velmi jednoduchá implementace. Stránky jsou uspořádány podle pořadí jejich alokace a vybrána je ta, která byla alokována jako první.

Problémem je to, že tento algoritmus může odstranit „starou“, ale stále často používanou stránku. Algoritmus trpí tzv. *Beladyho anomálií* – s rostoucím počtem rámců (zvětšením paměti RAM) může růst počet výpadků stránek, což je nepřirozené.

¹⁰V rámci obsluhy přerušení je obvykle provedena jen část následujících operací, protože nelze blokovat ostatní přerušení příliš dlouho. Zbytek zpracování výpadku je obvykle řešen jinými prostředky – viz literatura.

Používá se v kombinaci s přemístěním uvolněného rámce do množiny volných rámců, přidělením jiného volného rámce a možností ihned získat zpět právě uvolněný rámec při následném výpadku signalizujícím, že byla zvolena nesprávná „oběť“.

– Algoritmus LRU (Least Recently Used):

Algoritmus LRU vybere nejdéle nepoužitou stránku. Tento algoritmus je velmi dobrou aproximací ideálního algoritmu (tj. algoritmu se znalostí budoucnosti).

Implementace vyžaduje výraznou HW podporu. Je nutné zajistit označování stránek časovým razítkem posledního přístupu a udržování zásobníku stránek, jehož vrcholem je naposledy použitá stránka.

V praxi je tento algoritmus jen obtížně implementovatelný, proto se používají různé aproximace LRU.

– Aproximace LRU pomocí referenčního bitu ve stránkách:

Tento algoritmus používá speciální bit (access bit) v polích tabulky stránek, který je nastaven MMU při každém přístupu do stránky.

Referenční bit stránky je operačním systémem periodicky zaznamenáván do pomocných datových struktur a nulován, oběť je vybrána na základě uložené historie referenčního bitu. Ukládáme-li například 8 bitů historie tak, že přidáme nejvyšší bit a ostatní posuneme doprava a máme stránky s historií 11000100 a 01110111, odstraníme druhou z nich. Kterou stránku vybrat jako oběť poznáme podle menší hodnoty tohoto 8 bitového čísla.

– Existují i další algoritmy: 2nd chance, LFU, MFU, ...

- (b) Obsah vybrané stránky odložíme na swap je-li to nutné. Odložit se musí obecně všechny stránky, které byly procesem modifikovány. Pro detekci změny obsahu stránky MMU nastavuje při každém zápisu do stránky v příslušné položce tabulky stránek jeden bit – *příznak modifikace* („dirty bit“).

Například kódové stránky se nemusí odkládat, protože jejich obsah je vždy k dispozici v programu. Datové stránky je obvykle nutné uložit, protože obsahují procesem vypočtená data, která pravděpodobně budou ještě potřeba.

- (c) Uvolněný rámec můžeme použít.



3. Naplnění rámce

Inicializace rámce po alokaci je závislá na předchozím stavu stránky, kterou budeme do rámce mapovat. Pro některé oblasti LAP musíme rozlišit první výpadek stránky od ostatních:

- Při prvním výpadku dané stránky musíme vždy provést počáteční inicializaci obsahu stránky podle oblasti, ve které se stránka v LAP nachází:

Kód načteme z programu (spustitelného souboru) kde je uložen od známé pozice,

Konstantní data načteme také z programu podobně jako kód,

Inicializovaná data načteme z programu, protože inicializovaná data (například globální proměnné s inicializací) jsou při překladu uložena do programu,

Vše ostatní jsou neinicializovaná data (data, heap, zásobník) a taková stránka musí být vynulována, protože v ní nelze ponechat původní obsah z důvodu bezpečnosti (na stejném místě v paměti například mohla být stránka jiného procesu s tajnými daty, hesly či klíči a jejich ponechání by je promítlo do počátečních hodnot neinicializovaných proměnných jiného procesu – potenciálního útočníka na bezpečnost systému). Navíc některé programovací jazyky (například C) požadují aby neinicializované globální proměnné měly počáteční hodnotu nula a tento postup to automaticky zajistí.

- Pokud byla stránka v minulosti uvolněna z FAP máme následující možnosti:

Kód znovu načteme z programu protože je stejný jako při prvním výpadku. To samozřejmě platí jen pokud nelze přepisovat kódové stránky. (Přepisování kódu je z programátorského hlediska velmi nevhodné, ale pokud je to nutné, je možné zrušit ochranu proti zápisu a takové stránky se potom chovají jako inicializovaná data.)

Konstantní data jsou také neměnná (chráněna proti zápisu), proto se s jejich stránkami pracuje jako s kódovými,

Vše ostatní (inicializovaná i neinicializovaná data) musíme v případě že byla modifikována (a stránka musela být odložena do swapu) načíst ze swapu zpět do paměti. Přitom se uložená stránka ve swapu neuvolňuje, protože je možné, že tato stránka nebude modifikována a potom ji nebude nutné zapisovat znovu do swapu při jejím dalším odstraňování z FAP (ušetříme tím velmi pomalou diskovou operaci).

4. Úprava tabulky stránek. Zřízení mapování logické stránky na rámec znamená nastavení bitu přítomnosti, čísla rámce a inicializaci dalších příznaků v odpovídající položce tabulky stránek daného procesu.
5. Nakonec je proces nastaven do stavu *připravený*.

4.8 Výkonnost stránkování na žádost

Hlavním kritériem výkonnosti paměťového pod systému je přístupová doba do paměti (tj. za jak dlouho od vyslání adresy jsou k dispozici požadovaná data).

Průměrnou dobu přístupu do paměti můžeme vypočítat podle vzorce:

$$t = (1 - p)T + pD$$

kde

- p : pravděpodobnost výpadku stránky (*page fault rate*),
- T : doba přístupu bez výpadku,
- D : doba přístupu s výpadkem.

Vzhledem k tomu, že doba přístupu bez výpadku je o několik řádů menší než doba s výpadkem $T \ll D$, musíme zajistit, aby pravděpodobnost výpadku p byla co nejmenší. Toho dosáhneme splněním několika podmínek (nejlépe všech současně):

- Dostatek paměti – rozsah paměti musí být přiměřený počtu procesů a jejich náročnosti na paměť.
- Dobrá lokalita odkazů v procesech.
- Vhodné algoritmy výběru odkládaných stránek (dáno implementací OS).

4.8.1 Thrashing

Při nedostatku paměti může i v případě použití dobrých algoritmů pro výběr odstraňovaných stránek nastat tzv. *thrashing* při kterém proces (v horším případě celý systém) stráví více času čekáním na uvolnění rámců a načtení obsahu stránek než užitečným výpočtem.

Pokud je zvolen rozumný algoritmus výběru odstraňovaných stránek, je příčinou thrashingu nedostatek rámců přidělených procesům, což je obvykle důsledkem práce s příliš velkým počtem procesů.

Tento stav je nežádoucí a nelze se mu zcela vyhnout, protože nelze zakázat spouštění dalších procesů, které mají další požadavky na paměťový prostor.

4.8.2 Poznámky

Pro zvýšení efektivity stránkování na žádost existuje několik postupů. Jeden z nich je tzv. *prepaging* – snaha zavádět do systému více stránek současně (např. při startu procesu). Vychází se při tom z předpokladu, že budou v blízké budoucnosti pravděpodobně požadovány následující stránky. Navíc je práce s diskem efektivnější při přenosu několika po sobě jdoucích stránek současně v rámci jedné diskové operace.

Některé systémy řeší problém přetížení systému virtuální paměti násilným ukončováním vybraných paměťově náročných procesů. Například na OS Linux existuje v jádře tzv. „Out of Memory Killer“, který ruší jiné procesy pokud dojde k úplnému vyčerpání paměti. Problémem ale je které procesy v takovém případě rušit – na některých totiž závisí další funkce systému (init) případně řada jiných procesů (X-server). Proto toto řešení není univerzální a je možné jej vypnout.

4.9 Další vlastnosti systému stránkování na žádost

Systém virtuální paměti umožňuje řadu zajímavých doplňků, které umožní dále optimalizovat využití paměti a zrychlit některé operace. Zde se stručně zmíníme o sdílení stránek příbuznými procesy, sdílené paměti, paměťovém mapování souborů, sdílených knihovnách a zamykání stránek v paměti.

4.9.1 Sdílení stránek

Nyní se zaměříme na některé optimalizace související se stránkováním. Běžnou optimalizací je sdílení kódových stránek procesů řízených stejným programem. To je možné, protože kód nelze přepisovat a je tedy ve všech LAP takových procesů stejný a je i na stejných adresách. Toto sdílení kódu může být provedeno tak, že procesy mají stejný obsah části tabulky stránek, nebo sdílí příslušnou část hierarchické tabulky stránek. Podobně lze sdílet konstantní data.

Zajímavější je situace při sdílení datových stránek. Procesy mohou sdílet paměť, ale musí o to explicitně požádat jádro – tato varianta je náplní další podkapitoly. Zde budeme uvažovat příbuzné procesy vzniklé voláním `fork`.

Každý proces má svůj vlastní adresový prostor a sdílení jeho částí s jinými procesy je možné jen pokud tyto části mají stejný obsah pro všechny zúčastněné procesy. Toto implicitně vzniklé sdílení stránek musí být zrušeno v okamžiku, kdy jeden proces zapíše do takto sdílené datové stránky. Zapisující proces tak nepozná že dočasně sdílel tuto stránku s jiným procesem. Provádí se to mechanismem nazvaným *Copy-on-Write* (COW, kopie při zápisu). Tento mechanismus využívá vlastnosti systému stránkování – sdílená stránka má nastavenou ochranu proti zápisu a dokud se z ní pouze čte, zůstává sdílená více procesy. Pokud dojde k pokusu některého procesu o zápis, je vyvoláno přerušení a podsystém virtuální paměti kontroluje co se stalo. Pokud jde o sdílenou datovou stránku, je standardním způsobem alokovan rámec a do něj je okopírován původní obsah sdílené stránky. Tento nový rámec je pak mapován do LAP procesu, který provedl zápis ale již bez ochrany proti zápisu, protože tato nová stránka již není sdílena. Proces potom může pokračovat a přepsat si svoji kopii původně sdílené stránky. Výhodou tohoto přístupu je značná úspora paměti, pokud jsou v adresovém prostoru velké datové struktury do kterých se málo zapisuje a zůstávají tak sdílené.

Implicitně sdílené stránky vznikají při startu procesu:

- Při spuštění procesu pomocí `fork` se nevytvoří kopie veškeré paměti procesu ale vytvoří se pouze tabulky stránek a datové stránky se poznačí jako *copy-on-write*. To znamená značnou úsporu času i paměti. (Pokud ale proces následně provede `exec`, bude nutné tabulky stránek opět přepsat podle nově spouštěného programu.)
- K vytvoření fyzické kopie stránky dojde až při pokusu o zápis některého z procesů do takto označené stránky – viz výše uvedený mechanismus *Copy-On-Write*.

- Existuje také speciální služba `vfork`, která zajistí skutečné sdílení datových stránek více procesy (na rozdíl od `fork` s `copy-on-write`, který sdílí stránky jen dokud mají stejný obsah pro všechny procesy).

4.9.2 Sdílená paměť

Sdílená paměť (*shared memory*) je jedna z nejdůležitějších forem meziprocesové komunikace (IPC, InterProcess Communication). Sdílení paměti je velmi efektivní – co jeden proces zapíše do sdílené oblasti paměti, ostatní procesy podílející se na sdílení okamžitě mají k dispozici¹¹.

Implementace sdílení stránek zajišťuje mapování části LAP několika procesů na stejné rámce ve FAP – jde tedy o jednoduché operace s tabulkami stránek. Jádro systému implementuje pro práci se sdílenou pamětí několik služeb. Například norma POSIX definuje tyto služby:

- `shmget` vytvoří oblast sdílené paměti o zadané velikosti a vrátí její identifikaci (klíč),
- `shmat` připojí (mapuje) zadanou oblast do LAP procesu buď na zadanou adresu, nebo na nějaké vhodné místo do ještě neobsazeného prostoru v LAP,
- `shmctl` nastavuje nebo čte parametry sdílení, ochranu paměti atd.,
- `shmdt` odpojí zadanou oblast z LAP procesu.

Sdílená paměť je často používána například pro implementaci plug-in rozšíření programů (například GIMP) a pro rychlou komunikaci procesů (X-server, xlib).

4.9.3 Paměťově mapované soubory

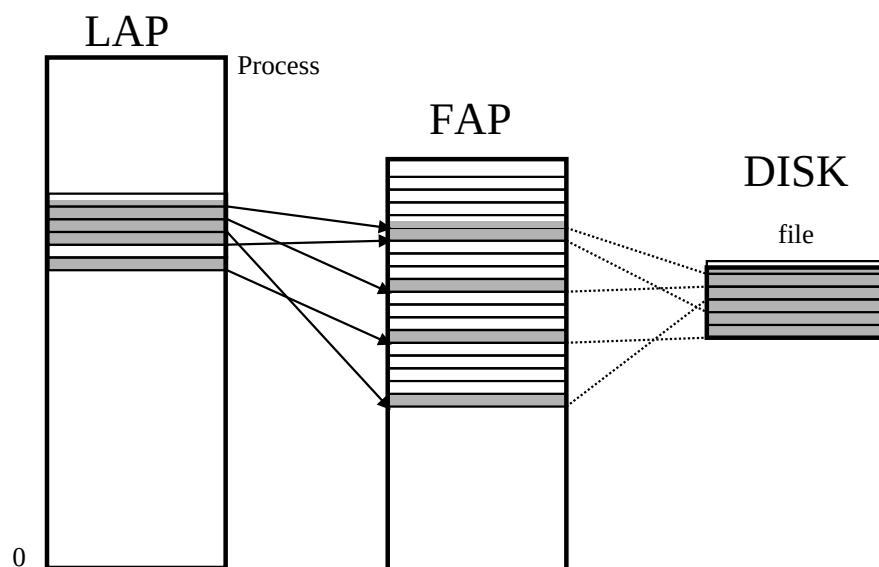
Podsystem virtuální paměti lze také využít k rychlé práci se soubory. Systémové volání v POSIX systémech nazvané `mmap` mapuje obsah otevřeného souboru (nebo jeho části) do volného místa v LAP procesu a volání `munmap` toto mapování ruší.

Jednotlivé bloky souboru jsou mechanismem stránkování mapovány do stránek v LAP procesu. Pro zjednodušení budeme předpokládat, že jsou alokační bloky souborového systému stejně velké jako stránky (ale obecně to platit nemusí). Soubory jsou stránkováním na žádost načítány podle potřeby po stránkách do fyzické paměti a veškeré operace s daty souboru jsou realizovány standardním přístupem do paměti namísto použití `read()`/`write()`. Při zápisu do takto přečtených stránek jsou automaticky tyto změny zaznamenány v tabulce stránek (*dirty bit*) a takto označené stránky jsou ukládány zpět do souboru stejným mechanismem jako jsou běžné datové stránky odkládány do swapu.

¹¹Při práci se sdílenými datovými strukturami je nutné zajistit synchronizaci procesů aby nedocházelo k souběžnému zápisu na stejné místo a podobným problémům. Také je nutné si uvědomit, že ukazatele obecně nejsou platné v rámci všech procesů sdílejících stejnou oblast paměti, protože sdílená paměť nemusí být na stejné adrese ve všech procesech.

Výhodou tohoto přístupu je významná úspora času: odpadá režie na systémové volání (které je relativně pomalé) a kopírování z vyrovnávací paměti v jádru do paměti procesu. Proto je mapování souborů do LAP často využíváno například databázovými systémy.

Obrázek 4.15 zobrazuje malý soubor mapovaný do LAP procesu. Zobrazená situace mohla nastat například po přečtení celého souboru (odpovídá čtení pole bajtů v paměti) s tím, že potom byla druhá stránka souboru vybrána jako oběť a byla uvolněna z FAP, proto nemá zobrazeno žádné mapování.



Obrázek 4.15: Příklad paměťově mapovaného souboru

Podobně jako při sdílení paměti je možné provádět také sdílený přístup k souboru několika procesy.

Při mapování souborů do LAP nastává u 32 bitových systémů problém s místem. Velké soubory mohou překročit velikost volného prostoru v LAP a někdy i velikost celého LAP (4GiB). Toto je jeden z důvodů proč je v řadě případů nutné používat 64 bitové systémy, které mají v LAP dostatečně velký prostor (přibližně 10^{19} B) pro mapování jakýchkoli dnes používaných souborů.

4.9.4 Sdílené knihovny

Sdílené knihovny (soubory s příponami `.dll`, `.so`) jsou knihovny podprogramů, které jsou využívány při programování aplikací. Výhodou těchto knihoven je, že nejsou součástí programů a proto jsou takové programy relativně malé. Navíc je možné zajistit na rozdíl od běžného sdílení kódu i sdílení kódu těchto knihoven procesy, řízenými různými programy. Výsledkem je, že kód sdílené knihovny je ve FAP maximálně jednou i když je sdílena mnoha procesy. Toto sdílení se pochopitelně netýká datových oblastí knihovny – ty musí mít každý proces.

Použití sdílených knihoven má některé další důsledky:

- Výhodná je možnost nezávisle na programech aktualizovat knihovny. To je zvláště významné při bezpečnostních aktualizacích – výměna jedné sdílené knihovny „opraví“ všechny programy danou knihovnu používající. U staticky sestavených programů by to znamenalo znovu přeložit a přeinstalovat všechny tyto programy.
- Nevýhodou je závislost programů na dalších souborech – sdílených knihovnách a jejich verzích. To může někdy komplikovat instalaci programů¹².
- Protože je nutné dynamické sestavování sdílených knihoven provádět až po spuštění programu, je start procesu pomalejší než u staticky sestavených programů.

4.9.5 Zamykání stránek ve FAP

Stránky je možné ve fyzické paměti uzamknout a tím zabránit jejich odložení. Zamykání je nutné především u stránek, do nichž probíhá vstup/výstup, protože periferie a DMA řadič obvykle mohou pracovat jen s fyzickou pamětí (nepoužívají MMU). Jádro také vyžaduje uzamčení stránek především z důvodů efektivity (je malé a je často potřeba).

Procesy mohou také jádro požádat (voláním `mlock()`) o uzamčení některých svých stránek v paměti. To je nutné například pro ukládání citlivých dat, která se nesmí za žádných okolností objevit ve swapu (kde by je mohl někdo přečíst po vypnutí počítače a vyjmutí disku). Další použití uzamčených stránek je v real-time systémech z důvodu rychlé reakce procesu na události.

4.10 Shrnutí

V této kapitole jsme se seznámili s principy virtualizace paměti obecně a s jednou konkrétní implementací – stránkováním na žádost. Tuto implementaci dnes používají téměř všechny víceúlohové operační systémy. Proto je znalost principu stránkování velmi důležitá a to nejen pro pochopení funkce operačního systému, ale především pro programování efektivních aplikací pro tyto systémy.

4.11 Otázky

Všechny otázky předpokládají stránkování na žádost.

- Popište činnost systému virtuální paměti při výpadku kódové stránky
- Popište činnost systému virtuální paměti při výpadku datové stránky (případ inicializovaných dat, všechny možnosti).
- Za předpokladu, že máme 32 bitovou logickou adresu, položku tabulky stránek velkou 32 bitů a velikost stránky je 4KiB vypočítejte maximální

¹²Tyto komplikace jsou na některých systémech známy pod pojmem „DLL hell“.

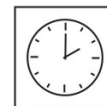


velikost dvouúrovňové tabulky stránek v KiB, pokud by byly mapovány všechny logické stránky. Jak se situace v tomto případě liší od použití jednoduché tabulky stránek? Proč je obvykle dvouúrovňová tabulka menší?

- Za předpokladu, že přístup do paměti trvá 50 ns, hledání v TLB 10 ns a používáme jednoúrovňové tabulky stránek vypočtete minimální a maximální přístupovou dobu do paměti (stránka je ve fyzické paměti).
- Popište hlavní výhody a nevýhody systému stránkování na žádost.

Kapitola 5

Synchronizace procesů



4:00

Cílem této kapitoly je seznámit čtenáře s problémy, které jsou spojeny se synchronizací paralelně běžících procesů (či dalších výpočetních aktivit: vláken, úloh apod.) a se základními možnostmi jejich řešení.



5.1 Časově závislé chyby

Zajištění správné *synchronizace* paralelně prováděných procesů, vláken či jiných typů výpočtů (jako je např. obsluha přerušení apod.) patří k nejnáročnějším programátorským úlohám. Zabýváme-li se operačními systémy je přitom obtížné (ne-li nemožné) se paralelismu vyhnout. Uvědomme si ale, že nutnost řešit problémy spojené s paralelismem může nastat i tvůrcům „jednoduchých“ jednovláknových aplikačních programů např. při obsluze signálů.

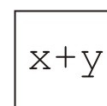
Chyby vznikající špatnou synchronizací paralelních procesů mohou být velmi zákeřné. Často vznikají jen zřídka při souběhu různých okolností a může pak být velmi obtížné je reprodukovat a identifikovat jejich příčinu. Pro zmíněnou nutnost souběhu různých okolností se tyto chyby také obtížně detekují klasickým testováním.

V souvislosti s chybami v synchronizaci procesů se často používá označení *časově závislá chyba* (neboli „*race condition*“). Jedná se o chyby vznikající při přístupu ke sdíleným zdrojům (sdílená data, sdílená I/O zařízení) kvůli různému pořadí provádění jednotlivých výpočtů v systému, tj. kvůli jejich různé relativní rychlosti. Označení „*race condition*“ vychází z představy chování systému závislého na výsledku závodu (či dostihu) mezi jednotlivými paralelními procesy.

Jako jednoduchý příklad *race condition* uveďme chování systému dvou paralelních procesů, z nichž jeden (producent) po zápisu dat do sdílené vyrovnávací paměti (bufferu) zvýší hodnotu čítače N počtu položek v této paměti a druhý (konzument) po načtení dat z vyrovnávací paměti hodnotu čítače N sníží. Soustředíme se pouze na paralelní manipulace se sdíleným čítačem N :

konzument: $N - -$ || producent: $N++$

Na první pohled by měla být uvedená manipulace s čítačem N bezproblémová a čítač by měl vždy uvádět správný počet položek v bufferu. Je ovšem zapotřebí dát pozor na to, že operace $N - -$ a $N++$ použité ve vyšším programovacím jazyce typu C/C++ mohou být přeloženy do strojového kódu např. jako *neatomická posloupnost* instrukcí typu: (1) načti hodnotu N z paměti do registru procesoru, (2) inkrementuj registr a (3) ulož hodnotu registru do paměti. Při přepínání kontextu pak může dojít k následujícímu výpočtu (předpokládejme, že počáteční hodnota N je 5):



```

producer: reg1 = N (reg1 == 5)
producer: reg1 = reg1 + 1 (reg1 == 6)
consumer: reg2 = N (reg2 == 5)
consumer: reg2 = reg2 - 1 (reg2 == 4)
producer: N = reg1 (counter == 6)
consumer: N = reg2 (counter == 4  !!!!!)

```

Vidíme, že výsledkem není očekávaná hodnota 5 nýbrž 4 a při jiném pořadí přepínání kontextu pak můžeme kromě správné hodnoty 5 získat také hodnotu 6! Výsledek tedy skutečně závisí na výsledku „závodu“ mezi producentem a konzumentem.

Zákeřnost výše uvedené chyby může přitom spočívat mj. v tom, že pokud bude systém testován, nebo provozován, v podobě, ve které jsou uvedené operace přeloženy na atomickou operaci s registrem procesoru, nemusí se tato chyba dlouho projevit. Problém pak může překvapivě vyplout na hladinu až se značným zpožděním po překompilování na jinou architekturu, při použití jiné optimalizace apod.

V následujících odstavcích nyní probereme základní možnosti a principy synchronizace procesů zajišťující správné pořadí jejich provádění a zabráňující výše uvedenému nežádoucímu chování.¹

5.2 Problém kritické sekce

Kritickou sekcí nazveme úsek kódu, jehož korektní provádění vyžaduje vzájemné vyloučení vůči jiným úsekům kódu, tzv. *sdíleným kritickým sekcím*. Skupina sdílených kritických sekcí tedy představuje skupinu úseků kódu takovou, že v daném okamžiku může existovat nanejvýš jeden proces, jehož řízení se nachází v rámci některého z těchto úseků kódu. Může samozřejmě existovat několik různých skupin sdílených kritických sekcí, které se navzájem neovlivňují – jedna může např. souviset s manipulací sdílené proměnné N a jiná s manipulací jiné sdílené proměnné M apod.

Při práci se sdílenými kritickými sekcemi je zapotřebí zajistit:

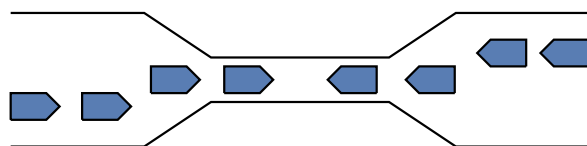
1. *Vzájemné vyloučení (mutual exclusion)*, kdy nanejvýš jeden proces může být v daném okamžiku v dané skupině sdílených kritických sekcí. V některých případech se také řeší obecnější problém, kdy je v dané skupině sdílených kritických sekcí přípustný nanejvýš určitý počet procesů.
2. Mimo to je ovšem zapotřebí zajistit také *dostupnost kritické sekce* – tj. každý proces by měl mít možnost vstoupit do kritické sekce v konečném čase, má-li o to zájem. Jinak by triviálním řešením bylo nepřipustit do kritické sekce nikdy nikoho, což by ovšem – jak čtenář zajisté uzná – nebylo příliš praktické.

¹Podrobnější informace najde zájemce o hlubší pochopení této problematiky v obecně dostupné literatuře (např. viz [21, 22]), na Internetu, příp. také v rámci kursu *Pokročilé operační systémy* v magisterském studiu na FIT.

DEF

V rámci zajištění dostupnosti kritické sekce je zapotřebí zabezpečit, aby nedocházelo k tzv. uváznutí, blokování a pokud možno ani stárnutí. *Uváznutím (deadlockem)* rozumíme situaci, kdy každý ze skupiny procesů čeká na stav, který by mohl nastat pouze tehdy, kdyby někdo z dané skupiny procesů mohl pokračovat (vzniká cyklická závislost na sebe vzájemně čekajících procesů) – viz ilustrace na obr. 5.1.

DEF



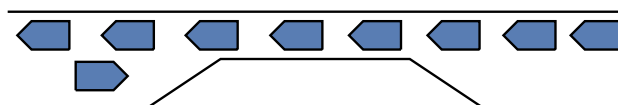
Obrázek 5.1: Ilustrace problému uváznutí na dopravní situaci

Blokováním (blocking) rozumíme situaci, kdy proces, jenž žádá o vstup do kritické sekce, musí čekat, přestože je kritická sekce volná (tj. žádný proces se nenachází v ní ani v žádné sdílené kritické sekci) a ani o žádnou z dané množiny sdílených kritických sekcí žádný další proces nežádá. K takové situaci může dojít např. tehdy, budeme-li žádat, aby se procesy v kritické sekci pravidelně střídaly, ovšem jeden proces bude žádat o kritickou sekci s vyšší frekvencí – ten pak musí pravidelně čekat, přestože druhý proces kritickou sekci momentálně nepotřebuje. Pokud pak proces, na který je řada pro vstup do kritické sekce (případně neočekávaně) skončí, druhý se již do kritické sekce nikdy nedostane.

DEF

Konečně *stárnutím (starvation)* – viz ilustrace na obr. 5.2 – rozumíme situaci, kdy proces čeká na podmínku, která vždy může, ale nemusí nastat. Pokud by se např. mezi procesy žádajícími o kritickou sekci vybíralo čistě náhodně, je teoreticky možné, že bude vyhrávat vždy jen jeden proces. Na rozdíl od možnosti uváznutí a blokování bývá možnost stárnutí někdy v praxi tolerována.

DEF



Obrázek 5.2: Ilustrace problému vyhladovění na dopravní situaci

Při řešení problému kritické sekce budeme typicky předpokládat, že *mimo kritickou sekci může proces setrvat libovolně dlouho* (příp. i neomezeně dlouho – může skončit aniž by se do kritické sekce dostal nebo aniž by se do ní dostal opakovaně). Na druhou stranu budeme předpokládat, že v rámci kritické sekce proces setrvá pouze konečně dlouho (nemůže v ní skončit) a také, že plánování procesů je *spravedlivé* (tzn. že proces připravený k běhu nebude neomezeně čekat na přidělení procesoru).

Než se podíváme na několik různých způsobů řešení přístupu do kritické sekce, předešleme, že i když použijeme korektní řešení přístupu do jednoduché

kritické sekce, mohou se výše zmíněné problémy s dostupností kritické sekce projevit při konstrukci složitějších, zanořených kritických sekcí. V podkapitole 5.4 se proto znovu a podrobněji podíváme na řešení problému uvážnutí, který v praxi patří z uvedených problémů k nejčastějším.

5.2.1 Specializované algoritmy pro vstup do kritické sekce

Jedním z možných řešení problému kritické sekce jsou specializované algoritmy nevyžadující žádnou výraznou podporu hardware. Mezi tyto algoritmy patří např. *Petersonův algoritmus* uvedený na obrázku 5.3 ve verzi pro dva paralelní procesy identifikované jako 0 a 1.

Princip činnosti tohoto algoritmu je takový, že proces i při vstupu do kritické sekce vyjádří svůj zájem o kritickou sekci nastavením příznaku `flag[i]`, ale současně dá přednost svému „protivníkovi“ nastavením `turn` na $1-i$. Následně proces aktivně čeká, až buď jeho protivník nemá o kritickou sekci zájem, nebo mu vrátí přednost ke vstupu do kritické sekce.

Pro uvedený algoritmus lze dokázat, že zajišťuje jak vzájemné vyloučení tak dostupnost kritické sekce. Doporučujeme čtenáři, aby se nad tímto zamyslel – při prvním shlédnutí algoritmu tato vlastnost nemusí být zřejmá. Obtížnost podobných úvah můžeme přitom vzít jako výraznou motivaci pro vývoj automatických metod formální verifikace, o kterých se krátce zmíníme na závěr této kapitoly.

```
bool flag[2] = { false }; // a shared variable
int turn = 0;             // a shared variable

// process i (i==0 or i==1):
while {1} {
    // ... before the critical section
    flag[i] = true;
    turn = 1-i;
    while (flag[1-i] && turn != i) ;
                                // busy waiting
    // the critical section
    flag[i] = false;
    // the remainder section
}
```

Obrázek 5.3: Petersonův algoritmus pro dva paralelní procesy identifikované jako 0 a 1

Pro vzájemné vyloučení N procesů pak lze mimo zobecnění Petersonova algoritmu [18] (a dalších podobných algoritmů – viz např. [21, 22]) použít také velmi elegantní tzv. *bakery algoritmus* L. Lamporta [11].² Pro zájemce je tento



²Slovo bakery je zde skutečně použito ve významu pekárna. Název vychází z představy čekání zákazníků u pekaře na základě utržených lístků s pořadovými čísly.

```

bool flag[N] = {false};
int ticket[N] = { 0 };
int j, max;

// process i
while (1) {
    // ... before the critical section
    flag[i] = true; // finding the max ticket
    for (j = 0; j < N; j++) {
        if (ticket[j] > max) max = ticket[j];
    }
    ticket[i] = max + 1; // take a new ticket
    flag[i] = false;
    for (j = 0; j < N; j++) {
        while (flag[j]); // wait
        if (ticket[j] > 0 &&
            (ticket[j] < ticket[i] ||
             (ticket[j] == ticket[i] && j < i))) {
            while (ticket[j] > 0);
        }
    }
    // the critical section
    ticket[i] = 0;
    // the remainder section
}

```

Obrázek 5.4: Bakery algoritmus vzájemného vyloučení pro N procesů

algoritmus v plné verzi uveden na obrázku 5.4. Nám postačí ale pochopit jeho základní princip:

- Před vstupem do kritické sekce proces získá „lístek“, jehož číselná hodnota je větší než čísla přidělená již čekajícím procesům. (S ohledem na to, že nelze atomicky otestovat hodnotu lístků všech procesů, provádí se to iterativně a hodnota některých lístků může být nakonec i stejná – pak rozhoduje velikost identifikačního čísla procesu.)
- Držitel nejmenšího čísla lístku s nejmenším identifikačním číslem procesu může vstoupit do kritické sekce.
- Čísla lístků přidělovaných procesům mohou teoreticky neomezeně růst, v praxi algoritmus ovšem počítá modulo rozsah datového typu použitého pro lístky. Dá se však ukázat, že i v takové situaci zajišťuje vzájemné vyloučení i dostupnost kritické sekce.

5.2.2 Využití hardware pro synchronizaci

Výše uvedené algoritmy lze užít i na platformě, která nenabízí žádnou zvláštní podporu hardware pro řešení vzájemného vyloučení (mimo sdílenou paměť³). V praxi se ale v současné době výrazně častěji používají řešení založená na různých sofistikovanějších podpoře ze strany hardware, která bývá dostupná na většině pokročilých procesorů.

Jednou z možných podpor nabízených hardware pro řešení problému vzájemného vyloučení je *atomická instrukce* typu *TestAndSet*, jejíž sémantiku můžeme popsat v jazyce C takto:

```
int TestAndSet(int *target) {
    int rv = *target;
    *target = 1;
    return rv;
}
```

Uvědomme si ovšem, že se jedná pouze o vysvětlení principu této instrukce. Aby ji bylo možné skutečně využít, musí se jednat o atomickou instrukci procesoru, nikoliv o výše uvedenou funkci. Na platformě Intel 386 a vyšší se může například jednat o instrukci `LOCK BTS`.

Po provedení instrukce *TestAndSet* bude vždy sdílená proměnná, na níž je *TestAndSet* aplikována, nastavena na 1 (jež zde má význam „zamknuto“), přičemž návratová hodnota bude odpovídat původní hodnotě této proměnné. Pro synchronizaci na kritické sekci je pak možné instrukci typu *TestAndSet* využít následovně:

```
int lock = 0; // a shared variable

// ...
while (TestAndSet(&lock)) ;
// critical section
lock = 0;
// ...
```

Uvedený kód pracuje na následujícím principu: (1) Kritická sekce je po provedení *TestAndSet* vždy uzamčena (buď byla a zůstává uzamčena, nebo původně nebyla, ale nyní již je uzamčena). (2) Proces se dozví, zda kritická sekce byla před provedením *TestAndSet* uzamčena (pak musí čekat), nebo zda byla volná (pak díky atomicitě *TestAndSet* ví, že ji právě uzamkl a může tedy pokračovat do kritické sekce).

Jinou *atomickou instrukcí* často užívanou pro synchronizaci je instrukce typu *Swap* (upozorníme, že níže uvedený kód opět pouze ilustruje funkci této instrukce – nelze ho užít pro synchronizaci):

³Navíc existují i algoritmy pro distribuované prostředí, např. [19].



```
void Swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Například na platformě Intel 386 a vyšší *Swap* odpovídá instrukci **LOCK XCHG**. Pro synchronizaci na kritické sekci lze tuto instrukci využít následovně:

```
int lock = 0; // a shared variable

// ...
int key = 1;
while (key)
    Swap(&lock, &key);
// critical section
lock = 0;
// ...
```

Výrazným nedostatkem výše uvedených řešení založených na atomických instrukcích ovšem je, že zahrnují tzv. *aktivní čekání*: Proces, který čeká na vstup do kritické sekce neustále cyklicky provádí některé instrukce a zbytečně zatěžuje procesor.

5.2.3 Semaforey

Jedním z nejčastěji používaných synchronizačních prostředků nevyžadujících aktivní čekání je tzv. semafor. *Semafor*, navržený původně E.W. Dijkstrou v [4], je v principu celočíselná sdílená proměnná přístupná dvěma atomickými (až na možnost pozastavení, je-li semafor obsazen) operacemi:

- *lock* (v některé literatuře též označovaná jako *P* či *wait*) a
- *unlock* (také *V* či *signal*).

Pozor! Někdy používané označení *wait* a *signal* *neznámá*, že semaforey mají cokoliv společného s operacemi téhož jména používanými při práci se signály v Unixu!

Semafor může vpouštět procesy do kritické sekce po jednom (tzv. *binární semafor*, též označovaný jako tzv. *mutex*), případně i ve větších předdefinovaných počtech. Při obsazování kritické sekce se hodnota semaforu snižuje, při uvolňování naopak zvyšuje. Je-li hodnota semaforu kladná, udává počet procesů, který je ještě možné vpustit do kritické sekce. Je-li tato hodnota nulová znamená to, že kritická sekce je obsazená, ale nikdo na kritickou sekci nečeká. Klesne-li semafor do záporných hodnot znamená to, že kritická sekce je obsazená a navíc příslušný počet procesů na semaforu čeká. Tyto procesy typicky nečekají aktivně, ale jsou pozastaveny a odkaz na ně je uložen ve frontě spojené se semaforem.

DEF



Semafor tedy můžeme deklarovat například jako následující strukturu (upozorníme, že se jedná o konceptuální implementaci – reálná implementace v praxi může být složitější díky nutnosti co nejvyšší optimalizace):

```
typedef struct {
    int value;
    processQueue q; // a queue of waiting processes
} semaphore;
```

Fronta čekajících procesů může být implementována jako zřetězený seznam záznamů o procesech (*process control block*), které v sobě typicky mají mj. ukazatel na následující záznam o procesu. Tyto záznamy je pak možné snadno řetězit do různých seznamů a příslušnou frontu deklarovat např. jako `struct PCB *q`.

Konceptuální implementace operace *lock* na semaforu *S* může nyní vypadat takto:

```
lock(S) {
    S.value--;
    if (S.value < 0) {
        // add the calling process to S.q;
        block(); // suspend the calling process
    }
}
```

Všimněme si, že v případě, kdy kritická sekce je již obsazená, proces, který zavolá *lock*, v konečném důsledku sám sebe vloží do čekací fronty a pozastaví se (resp. to na jeho žádost spočívající ve volání *lock* provede operační systém).

Konceptuální implementace operace *unlock* na semaforu *S* pak může vypadat takto:

```
unlock(S):
    S.value++;
    if (S.value <= 0) {
        // remove a process P from S.q;
        wakeup(P); // resumes execution of P
    }
}
```

Proces, který pomocí operace *unlock* uvolňuje kritickou sekci, zjistí zda někdo nečeká ve frontě spojené se semaforem. Pokud ano, odstraní první čekající proces z fronty a aktivuje ho.

Pro *synchronizaci na kritické sekci* můžeme nyní semafor užít velmi snadno:

```
semaphore mutex; // initially, mutex = 1
// ...
lock(mutex);
// critical section
unlock(mutex);
// ...
```

Příklad: Předpokládejme, že máme dva procesy řízené výše uvedeným kódem (jehož řádky si můžeme pro názornost očíslovat). Zapiště několik různých možností posloupností provádění jednotlivých příkazů (včetně kódu operací *lock* a *unlock*), příslušných hodnot semaforu a obsahu fronty s ním spojené podobně jako v ilustrativním příkladu časově závislé chyby v úvodu kapitoly.



S výše uvedenou konceptuální implementací semaforu ovšem souvisí jeden kritický problém, který jsme doposud ještě neřešili: Operace *lock* i *unlock* musí být *atomické*! Jinými slovy jejich tělo představuje rovněž kritickou sekci. K řešení této kritické sekce samozřejmě nemůže použít semaforey. Jako alternativa se nabízí *zákaz přerušení*, ten lze ovšem užít jen na jednoprocessorových systémech, případně použití specializovaných instrukcí a aktivního čekání (tzv. *spinlock*), jak jsme ho popsali v podkapitole 5.2.2.



Přirozenou otázkou nyní může být, jaký smysl má vyhýbat se aktivnímu čekání použitím semaforů, pro jejichž implementaci použijeme aktivní čekání. Řešení tohoto zdánlivého paradoxu spočívá v tom, že zatímco sdružené kritické sekce tvořené tělem operací *lock* a *unlock* jsou kratičké (a obvykle maximálně optimalizované vývojáři operačního systému až na několik málo instrukcí), kritické sekce chráněné semaforem jsou v režii aplikačního programátora a mohou být libovolně dlouhé (např. čtení z disku, tisk na tiskárně apod.).

Diskusi semaforů uzavřeme odkazem na systémová volání, které je možno pro práci se semaforey užít. Pro synchronizaci procesů lze v SVr4 užít *semget* (alokace semaforu) a *semop* (na základě parametrů lze užít jako *lock* i *unlock*, případně i pro neblokující pokus o uzamčení apod.). Pro synchronizaci vláken lze dle POSIX užít *pthread_mutex_lock/unlock*.

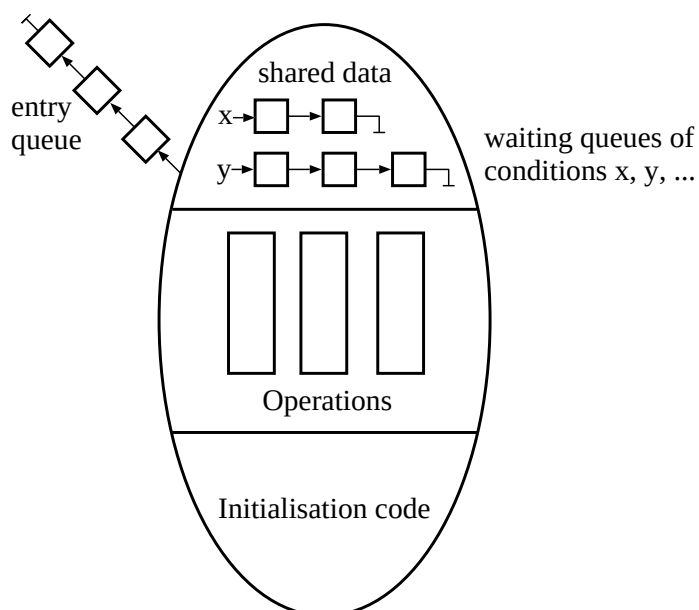
5.2.4 Monitory

Semaforey jsou velmi *obecným* a *nízkoúrovňovým* synchronizačním mechanismem. Umožňují jak vzájemné vyloučení tak i čekání na událost (jeden proces semafor zamkne, jiný odemkne) apod. Důsledkem toho je, že použití semaforu je relativně komplikované. Navíc i při nejjednodušším použití musí programátor neustále dávat pozor na to, aby nezapomněl v určitém okamžiku semafor zamknout a poté ho opět odemknout. U demonstračních příkladů se toto může zdát snadné, ale praxe ukazuje, že u velkých projektů, na kterých pracují týmy vývojářů (a současně je u těchto systémů kladen důraz na optimalizaci výkonu a není tedy možné mít dlouhodobě vše zamknuto), je situace jiná.

Pro programátorsky komfortnější implementaci synchronizace byly proto navrženy různé *vysokoúrovňové synchronizační mechanismy*, často vystavěné nad semaforey. Mezi ně patří např. monitory, podmíněné kritické sekce, bariéry apod. My si v této podkapitole krátce představíme alespoň jeden z těchto mechanismů, a to konkrétně monitory.

Monitor v podobě, v jaké byl původně zaveden, je *abstraktní datový typ*, který v sobě zapouzdřuje sdílená data, nad kterými je možno v rámci monitoru implementovat určité operace, přičemž je zaručeno, že v daném okamžiku bude vždy rozpracována pouze jedna z těchto operací. Navíc monitor zahrnuje i příslušný





Obrázek 5.5: Ilustrace koncepce monitoru

inicializační kód pro sdílená data – viz ilustrace na obr. 5.2.4. Na vzniku konceptu monitoru se podíleli P. Brinch-Hansena, O.J. Dahl, E.W. Dijkstra a Tony Hoare [6, 8].

Vzájemné vyloučení mezi operacemi nad monitorem může zajistit automaticky a bezpečně překladač např. vložení příslušných operací nad semaforey na vstupu a výstupu do/z operací monitoru. Za běhu pak na vstupu do monitoru může vzniknout fronta čekajících procesů.

Zavedení monitoru vyžaduje rozšíření syntaxe programovacího jazyka např. o následující konstrukci:

```
monitor monitor-name {
    shared variable declarations

    procedure P1 (...) {
        ...
    }

    procedure P2 (...) {
        ...
    }

    {
        initialization code
    }
}
```

Monitor tak, jak jsme ho doposud popsali, poskytuje velmi komfortní řešení problému vzájemného vyloučení. Aby ho ovšem bylo možno využít i pro čekání na určitou událost, rozšiřuje se dále o koncept tzv. *podmínek* (*conditions*). Nad podmínkami deklarovanými v rámci monitoru (například syntaktickou konstrukcí `condition c1, c2, ...;`) lze užít operace:

- *wait* a
- *signal*, resp. *notify*.⁴

Operace *wait* umožňuje pozastavení procesu v rámci monitoru, automatické uvolnění monitoru a čekání na určitou událost. Operace *signal* či *notify* (podle toho, která je podporována) upozorňují procesy čekající na určité podmínce na vznik události, na kterou čekají.

Po použití *signal* či *notify* samozřejmě nemohou pokračovat jak odesílatel informace o vzniku události tak její příjemce, neboť v tom případě by se do monitoru současně dostalo více procesů. V případě použití operace typu *signal* proto pokračuje příjemce a teprve až opustí monitor, pokračuje odesílatel (a pak případně další procesy čekající na vstup do monitoru). U operace typu *notify* pokračuje odesílatel (kdežto příjemce je pouze odblokován a znovu soutěží o vstup do monitoru).

Mechanismus podmínek a operací *wait* a *signal* či *notify* je možné opět automaticky implementovat pomocí vhodného použití semaforů.

Monitory se syntaktickou podporou zmíněnou výše do běžných současných jazyků přímo nepronikly. V POSIXu je ovšem v knihovně pro práci s vlákny podpora pro použití podmínek v podobě datového typu `pthread_cond_t` a s ním souvisejících funkcí `pthread_cond_wait/signal/broadcast`. Ty je ovšem zapotřebí kombinovat s použitím semaforů. Kromě toho je koncept monitoru (s jednou implicitní podmínkou) zapracován do jazyka *Java* v podobě tříd či metod chráněných konstrukcí *synchronized*.

5.3 Některé klasické synchronizační problémy

V této podkapitole si nyní všimneme několika typických synchronizačních problémů, které se (případně v různých variantách) často objevují v paralelních systémech.

5.3.1 Producent a konzument

Systém *producenta a konzumenta* sestává ze dvou procesů (obecně jich samozřejmě může být více), z nichž jeden generuje data a zasílá je druhému k dalšímu zpracování přes vyrovnávací paměť (buffer) o kapacitě omezené na *N* položek. V situaci, kdy se vyrovnávací paměť zaplní, musí čekat producent; na druhou stranu, je-li buffer prázdný, musí čekat konzument.

⁴Opět upozorňujeme na shodu jmen. Nejedná se zde o operace nad semaforey ani o operace užívané při práci se signály v Unixu.

DEF

DEF

Konceptuální řešení tohoto synchronizačního problému, ve kterém abstrahuje vlastní práci s vyrovnávací pamětí, generování dat a jejich zpracování, může při použití semaforů vypadat následovně.

```
semaphore full, empty, mutex;

// Initially:
//
// full = 0, empty = N, mutex = 1

// Producer                                // Consumer
do {                                        do {
    ...                                    lock(full)
    // produce an item I                  lock(mutex);
    ...                                    ...
    lock(empty);                          // get I~from buffer
    lock(mutex);                          ...
    ...                                    unlock(mutex);
    // add I~to buffer                    unlock(empty);
    ...                                    ...
    unlock(mutex);                        // consume I
    unlock(full);                          ...
} while (1);                             } while (1);
```

Binární semafor *mutex* zde slouží k vzájemnému vyloučení při práci s vyrovnávací pamětí. Obecné semaforey *full* a *empty* slouží k předávání informace o počtu obsazených, resp. volných, položek ve vyrovnávací paměti. V případě jejího přeplnění či vyprázdnění pozastaví producenta, resp. konzumenta, dokud není možné pokračovat.

5.3.2 Čtenáři a písáři

Jiným typickým synchronizačním problémem je systém *čtenářů a písářů*. V tomto systému máme libovolný počet procesů, z nichž některé čtou sdílená data (např. z databáze), zatímco jiné tato data modifikují. Platí, že v daném okamžiku může současně číst libovolný počet čtenářů. Pokud ovšem nějaký písář píše, nemůže žádný čtenář číst, ani žádný další písář psát.

V konceptuální podobě, ve které abstrahujeme vlastní čtení a zápis sdílených dat, by řešení pomocí semaforů (které ale ještě není dokonalé, jak se zmíníme níže) mohlo vypadat následovně:

```
int readcount;
semaphore mutex, wrt;

// Initially:
//
// mutex = 1, wrt = 1, readcount = 0
```

DEF

```

// Writer
do {
    ...
    lock(wrt);
    // writing
    ...
    unlock(wrt);
    ...
} while (1);

// Reader
do {
    lock(mutex);
    readcount++;
    if (readcount == 1)
        lock(wrt);
    unlock(mutex);
    ...
    // reading
    ...
    lock(mutex);
    readcount--;
    if (readcount == 0)
        unlock(wrt);
    unlock(mutex);
    ...
} while (1);

```

Oba použité semafore jsou binární. Semafor *wrt* slouží pro vzájemné vyloučení píšícího pásaře vůči všem ostatním pásařům i čtenářům. Semafor *mutex* slouží pro vzájemné vyloučení čtenářů při manipulaci se sdílenou proměnnou *readcount*, v níž se udržuje aktuální počet čtoucích čtenářů. Všimněme si, že semafor *wrt* zamyká pouze první ze čtenářů a odemyká ho poslední z nich.

Nedostatkem uvedeného řešení je možnost *vyhladovění pásařů*. Ti mohou být donekonečna předbíháni čtenáři, kteří se mohou střídát v kritické sekci – než jeden odejde, druhý přijde, než ten odejde, přijde třetí, ... Pokud by toto řešení nebylo v praxi dostatečné, je možné dosáhnout řešení bez stárnutí po přidání dalšího semaforu. Toto řešení ponecháme čtenáři jako vhodné cvičení.

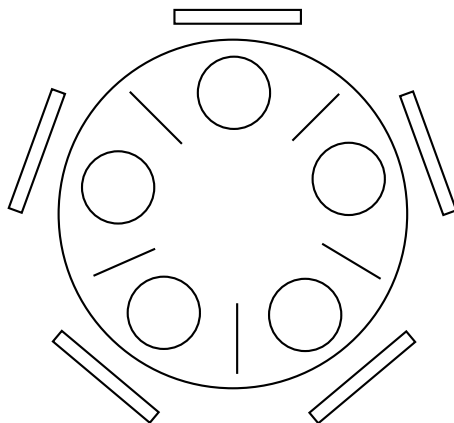
5.3.3 Večeřící filozofové

Posledním typickým synchronizačním problémem, který zde rozebereme, je problém *večeřících filozofů*. Jedná se o abstrakci synchronizačního problému s cyklickými závislostmi mezi procesy a použitými zdroji.

V systému večeřících filozofů máme pět filozofů, kteří sedí kolem kulatého stolu – viz obr. 5.3.3. Každý z nich má svůj talíř a k jídlu používá čínské hůlky. Poněkud netypicky ovšem nemá každý filozof svou dvojici hůlek, nýbrž mezi každými dvěma filozofy je umístěna jediná hůlka. Aby filozof mohl jíst musí získat obě hůlky, které se nacházejí po stranách jeho talíře. Pak se může najíst, hůlky vrátit na stůl a přemýšlet. Celý proces se následně může opakovat.

Možné řešení problému večeřících filozofů s využitím semaforů je uvedeno níže. Pro každou hůlku je deklarován binární semafor. V uvedeném řešení filozofové získávají hůlky v pořadí levá-pravá, nebo pravá-levá, podle toho, zda sedí na sudém či lichém místě.





Obrázek 5.6: Problém večeřících filozofů

```
semaphore chopstick[5];

// Philosopher i
do {
    if (i % 2) { // odd numbered philosophers
        wait(chopstick[i])
        wait(chopstick[(i+1) % 5])
    } else {    // even numbered philosophers
        wait(chopstick[(i+1) % 5])
        wait(chopstick[i])
    }

    ...
    // eat
    ...
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    ...
    // think
    ...
} while (1);
```

Přirozenou otázkou nyní může být, zda je nutné hůlky obsazovat v *uvedeném pořadí*. Čtenáři doporučujeme se zamyslet, k čemu by v systému mohlo dojít, kdyby hůlky obsazovali všichni filozofové v pořadí levá-pravá (či pravá-levá), a dále co musí obecně platit pro pořadí obsazování hůlek, aby k tomuto problému nedocházelo.



5.4 Uváznutí (deadlock)

V podkapitole 5.2 jsme probírali pojem kritické sekce a různé možnosti synchronizace na ní. Uváděli jsme synchronizační prostředky, které při správném použití umožní řešení splňující jak vzájemné vyloučení na kritické sekci, tak její dostupnost. Správné použití zmíněných prostředků na jednoduchých kritických sekcích je relativně jednoduché (i když i tam mohou vzniknout chyby – např. opomenutí zamknutí či odemknutí semaforu). Ovšem situace se dále výrazně komplikuje při řešení složitějších synchronizačních problémů zahrnujících zanořené kritické sekce. V takovém případě i použití synchronizačních prostředků, které by pro každou jednotlivou kritickou sekci samostatně bylo korektní, může vést k chybám (viz i některé z výše uvedených typických synchronizačních problémů). Jednou z nejčastějších z těchto chyb je přitom uváznutí (deadlock), který si proto nyní blíže rozebereme.

Definici uváznutí (deadlocku) jsme již uváděli výše, nyní si ji ale připomeneme. *Uváznutí (deadlock)* je situace, kdy ve skupině procesů každý čeká na uvolnění prostředku s výlučným přístupem přiděleného jinému procesu z dané množiny (vzniká cyklická závislost na sebe čekajících procesů) a ani jeden proces z této množiny proto nemůže pokračovat.

Aby došlo k uváznutí (deadlocku) musí být splněny *čtyři nutné podmínky*:

1. Musí být užito vzájemného vyloučení při přístupu ke sdíleným prostředkům.
2. Ve skupině procesů, které uvázly, jsou procesy, které vlastní alespoň jeden zdroj a čekají na další.
3. Sdílené zdroje jsou uvolňovány až po úplném dokončení jejich použití.
4. Vzniká cyklická závislost na sebe čekajících procesů, tj. proces 1 vlastní nějaké zdroje a čeká dále na zdroj používaný procesem 2, proces 2 rovněž vlastní nějaké zdroje a čeká dále na zdroj používaný procesem 3 a tak dále až nějaký proces N rovněž vlastní jisté zdroje a čeká na zdroje vlastněné procesem 1.

Pozor! Zmíněná cyklická závislost čekajících procesů tedy nijak nesouvisí s cyklením zmíněném při aktivním čekání na vstup do kritické sekce.

Jako jednoduchý příklad systému, ve kterém hrozí uváznutí, můžeme uvést následující jednoduchý systém se dvěma procesy a dvěma semaforů – můžeme si např. představit, že jeden z těchto semaforů zajišťuje výlučný přístup k disku, jiný k paměti:

```
semaphore mutexDisk, mutexMem;
```

```
// Initially:
```

```
//
```

```
// mutexDisk = 1, mutexMem = 1
```

DEF



$x + y$

```

// Proces 1
do {
    ...
    lock(mutexDisk);
    ...
    lock(mutexMem);
    ...
    // a critical section
    ...
    unlock(mutexMem);
    unlock(mutexDisk);
    ...
} while (1);

// Proces 2
do {
    ...
    lock(mutexMem);
    ...
    lock(mutexDisk);
    ...
    // a critical section
    ...
    unlock(mutexDisk);
    unlock(mutexMem);
    ...
} while (1);

```

Pokud bude provádění uvedených procesů naplánováno tak, že nejprve proces 1 získá přístup k disku, pak proces 2 získá přístup k paměti a následně proces 1 požádá o přístup k paměti a proces 2 o přístup k disku, dojde k uváznutí. Lze si snadno ověřit, že všechny výše uvedené nutné podmínky uváznutí budou v takovém případě splněny.

V následujících podkapitolách se podíváme na tři možnosti řešení problému uváznutí: *prevence uváznutí*, *vyhýbání se uváznutí* a *detekce a zotavení se po uváznutí*. Každý z těchto přístupů má své výhody a nevýhody a je vhodné ho použít v určitých situacích. V praxi se v běžných operačních systémech dokonce často neužívá žádný z nich a předpokládá se, že programy budou psány tak, aby k uváznutí prostě nedocházelo – což se ovšem obtížně realizuje, a proto je pak kladen důraz na různé techniky odhalování možnosti uváznutí, mj. i na *formální verifikaci* zmíněnou na závěr této kapitoly.

5.4.1 Prevence uváznutí

Principem prevence uváznutí je *zrušení platnosti některé z nutných podmínek uváznutí*. S ohledem na čtyři nutné podmínky uváznutí může být prevence uváznutí konkrétně založena např. na následujících opatřeních:

1. U prostředků, které umožňují (skutečně současný) sdílený přístup, nejsou zámký zapotřebí a nebudeme je tedy zbytečně používat.
2. Proces může žádat o prostředky pouze tehdy, pokud žádné nevlastní.
3. Pokud proces požádá o prostředky, které nemůže momentálně získat, je pozastaven, všechny prostředky jsou mu odebrány a čeká se, až mu mohou být všechny potřebné prostředky přiděleny.
4. Prostředky jsou očíslovány a je možné je získávat pouze od nejnižších čísel k vyšším (nebo v jiném, pevně daném pořadí).

Ne vždy samozřejmě bude možné (nebo efektivní) použít každou z uvedených technik. Např. přidělování prostředků pouze v určitém pořadí, nebo všech

DEF

najednou, nemusí být efektivní; odebrání prostředků může být problematické, jsou-li nad nimi rozpracovány, ale nedokončeny, určité operace apod.

5.4.2 Vyhýbání se uváznutí

Princip vyhýbání se uváznutí je založen na tom, že každý proces před spuštěním deklaruje určité informace o způsobu, jakým bude využívat sdílené zdroje – v nejjednodušším případě se jedná o *maximální počet* současně požadovaných zdrojů jednotlivých typů (může se ale jednat i složitější informace o tom, kdy a jaké zdroje bude proces používat). Předem známé informace o možných požadavcích jednotlivých procesů a o aktuálním stavu přidělování se pak využijí k rozhodování o tom, které požadavky mohou být uspokojeny (a které musí počkat) tak, aby s ohledem na to, jaké ještě mohou přijít požadavky, nehrozil vznik cyklického čekání procesů na sebe navzájem.

Existuje řada různých *algoritmů pro vyhýbání se uváznutí*. Mezi ně patří mimo jiné algoritmus založený na *grafu alokace zdrojů* pro systémy s jednou instancí každého zdroje – viz obr. 5.7. Graf alokace zdrojů obsahuje dva typy uzlů – procesy (značeny kolečky) a sdílené zdroje (značeny čtverečky). Mezi zdrojem R_i a procesem P_j mohou být tři typy hran:

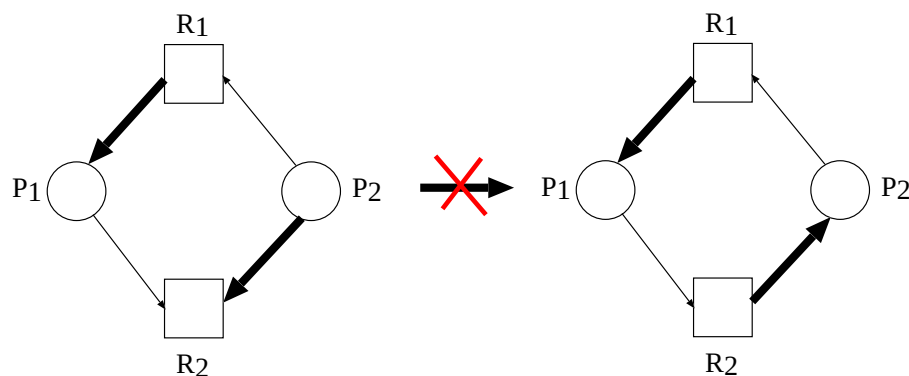
1. $P_j \rightarrow R_i$ vyjadřující, že proces P_j může požádat o zdroj R_i . Tato hrana se v grafu vytváří při spuštění procesu, kdy tento deklaruje informace o tom, které zdroje může používat.
2. $P_j \Rightarrow R_i$ (v obrázku značeno tlustou čarou) vyjadřující, že proces P_j aktuálně žádá o zdroj R_i . Tato hrana v grafu nahrazuje výše uvedenou hranu v okamžiku, kdy proces o příslušný zdroj požádá.
3. $R_i \Rightarrow P_j$ (v obrázku značeno tlustou čarou) vyjadřující, že zdroj R_i je momentálně používán procesem P_j . Tato hrana v grafu nahrazuje hranu předchozího typu v situaci, kdy systém přidělí zdroj R_i procesu P_j . (V okamžiku, kdy je zdroj uvolněn, se v grafu vracíme u dvojice P_j, R_i k prvnímu typu hrany.)

Algoritmus pro vyhýbání se uváznutí založený na grafu alokace zdrojů přitom *přidělí sdílený zdroj R_i procesu P_j pouze tehdy*, pokud při obrácení příslušné hrany nevznikne v grafu alokace zdrojů cyklus (viz obr. 5.7). Pokud by tomu totiž tak bylo, pak hrozí, že pokud procesy určitým způsobem zrealizují požadavky na zdroje, které deklarovaly, může dojít k uváznutí. Na druhou stranu pokud cyklus v grafu nevzniká, nemůže dále dojít k uváznutí při libovolných dalších požadavcích procesů (v mezích, které tyto procesy deklarovaly).

Zobecněním uvedeného algoritmu na zdroje s obecnou kapacitou je pak tzv. *bankéřův algoritmus* (viz např. [21]). Název tohoto algoritmu vychází z představy analogické situace v bance, kdy bankéř při půjčování prostředků musí vždy zajistit, že bude schopen uspokojit možné požadavky na výběr uložených vkladů.

Algoritmy pro vyhýbání se uváznutí sice zamezí vzniku uváznutí, ale opět se jedná o kompromisní řešení, které bude vyhovovat jen v některých situacích:





Obrázek 5.7: Graf alokace zdrojů a jeho využití pro vyhýbání se uváznutí

(1) u procesů je nutno znát a předem deklarovat požadavky, (2) rozhodnutí odložit přidělení zdroje může být značně konzervativní a může snižovat efektivitu systému.

5.4.3 Detekce uváznutí a zotavení

Jiným řešením, které vychází spíše z optimistické představy, že uváznutí není příliš pravděpodobné, je mechanismus detekce *uváznutí a zotavení po něm*. V jeho případě systém přiděluje zdroje tak, jak o ně procesy žádají (pokud jsou právě volné). Vytváří si ovšem graf zachycující vztahy mezi procesy a zdroji, o které procesy žádají, resp. které vlastní. Periodicky (nebo při každé žádosti o zdroje) se pak kontroluje, zda nedošlo k uváznutí. Pokud ano, pak systém uváznutí odstraní např. tak, že některému procesu odebere některé zdroje a proces pozastaví (případně ho zruší či restartuje). Problémem pak samozřejmě může být odebrání zdrojů v situaci, kdy jsou nad nimi rozpracovány, ale nedokončeny, určité operace.

DEF

5.5 Formální verifikace

Pokud není použit mechanismus prevence, obcházení či detekce a zotavení se z uváznutí (či jiné podobné mechanismy pro ostatní nežádoucí problémy spojené se synchronizací jako jsou blokování, stárnutí apod.), je vhodné ověřit, že systém je navržen tak, že žádné nežádoucí chování nehrozí.

Mezi možnosti odhalování nežádoucího chování systémů obecně patří inspekce systému, simulace a testování (resp. tzv. dynamická analýza) a také formální verifikace. *Inspekce systému* znamená, že jiná osoba nežli autor kódu musí příslušný kód projít, pochopit a uznat za správný. V případě *simulace* (na modelu systému) nebo *testování* (na reálném systému) se jedná o opakované provádění (experimenty) se systémem (či jeho modelem) ve snaze odhalit jeho možné nežádoucí chování.

Odhalování chyb v synchronizaci paralelních systémů testováním (či simulací) je ovšem značně problematická záležitost, protože ke vzniku takové chyby může být zapotřebí značná souhra okolností a její výskyt během testování (i v situaci, kdy se např. snažíme ovlivnit uměle plánovač tak, aby vznik chyby byl pravděpodobnější) může být velmi málo pravděpodobný. S ohledem na to je v současné době věnována značná pozornost i vývoji a aplikacím metod *formální verifikace*, jež na rozdíl od simulace a testování umožňují nejen vyhledávat chyby, ale také dokázat správnost systému s ohledem na zadaná kritéria (což znamená, že žádné chyby již nezůstaly bez povšimnutí).

Existující přístupy k *automatizované formální verifikaci* zahrnují:

- *dokazování teorémů (theorem proving)*,
- *model checking (ověřování modelu* – název vychází z historických kořenů, technika ale nemusí nutně pracovat s modely, nýbrž i přímo s ověřovaným systémem) a
- *statická analýza*.

Metoda *dokazování teorémů (theorem proving)* se velmi podobá dokazování teorémů v matematice, ovšem s poloautomatickou podporou tzv. *theorem proverem* (PVS [15], Isabelle [14], ...), který dokáže vést přesnou evidenci obecně známých fakt, již odvozených fakt o daném systému, korektních odvozovacích pravidel, dokáže nabídnout, která odvozovací pravidla se dají aktuálně použít apod. Dokazování teorémů je velmi mocná a obecná technika, ale je současně velmi náročná v tom, že obvykle vyžaduje velmi zkušeného experta, který určí, jak se má důkaz vést.

Model checking [3] je přístup, který využívá (explicitní či implicitní) generování a prohledávání *stavového prostoru* zkoumaného systému. Je to přístup, který je obvykle (takřka) zcela automatický. Je podpořen nástroji, tzv. *model checkery*, jako jsou např. Spin [12], SMV [9], Slam [1], Blast [7], JPF [2] a další. (Slam a Blast pracují nad jazykem C, JPF – neboli Java PathFinder – nad Javou). Hlavní nevýhodou model checkingu je *problém stavové exploze* [23], kdy velikost stavového prostoru roste exponenciálně s velikostí modelu, případně práce s neomezeným počtem stavů.

Konečně *statická analýza* [5, 20] se snaží o ověření příslušných vlastností na základě zdrojových kódů modelu či systému, aniž by se tento prováděl a procházel se jeho stavový prostor. Mezi statické analyzátory patří např. nástroje vyvíjené společnostmi Coverity, Polyspace, Grammatech, Microsoft, ..., případně nástroj L. Torvaldse Sparse. Výhodou statické analýzy je schopnost pracovat s ohromnými zdrojovými kódy (např. celý Linux či Windows), na druhou stranu nemusí být statická analýza schopna detekovat určité typy chyb a může být méně přesná (tj. může generovat řadu nepodložených podezření na neexistující chyby, případně se může jednat i o techniky jen přibližné, které mohou některé chyby pominout).

S ohledem na to, že žádný z výše uvedených přístupů není dokonalý, je pak možné konstatovat, že nejvýhodnější je samozřejmě kombinace co nejvíce různých přístupů k odhalování chyb. Smysl přitom mají i ty techniky formální verifi-



kace, jejichž běh nakonec není úspěšně dokončen (např. pro nedostatek paměti). Tyto techniky totiž prochází stavovými prostory zkoumaných systémů jiným způsobem než např. testování a je naděje, že i tehdy, nepodaří-li se jimi dokázat korektnost systému, mohou alespoň odhalit některé chyby.

5.6 Shrnutí

V této kapitole jsme diskutovali problematiku synchronizace paralelních procesů (či vláken, úloh a jiných výpočetních aktivit). Seznámili jsme se s pojmem časově závislých chyb (a náročností jejich odhalování) a problémem kritické sekce. Probrali jsme možnosti řešení problému kritické sekce pomocí specializovaných algoritmů, atomických instrukcí, semaforů a monitorů. Diskutovali jsme řešení několika typických synchronizačních problémů, konkrétně systémů producent – konzument, čtenáři a písaři a večeřející filozofové. Značnou pozornost jsme také věnovali problému uváznutí a možnostem jeho řešení. Na závěr jsme krátce zmínili oblast formální verifikace a jejích výhod při odhalování chyb v synchronizaci.



5.7 Kontrolní otázky

1. Co jsou to časově závislé chyby? Uveďte příklad. Proč je jejich odhalování náročné?
2. Co je to kritická sekce, co je při jejím řešení zapotřebí zajistit?
3. Jaké specializované algoritmy pro řešení problému kritické sekce znáte? Jaký je jejich princip?
4. Jaká podpora ze strany hardware se typicky využívá pro řešení problému kritické sekce a jak? (Popište konceptuální řešení pomocí vhodného programovacího jazyka či pseudokódu.)
5. Co je to semafor, jaký je základní princip jeho implementace (popište konceptuální řešení pomocí vhodného programovacího jazyka či pseudokódu) a jeho využití pro řešení problému kritické sekce? Co musí platit pro operace nad semaforem a jak toho lze docílit?
6. Co je to monitor a podmínky nad monitorem?
7. Popište konceptuální řešení problémů producent – konzument, čtenáři a písaři a večeřející filozofové pomocí semaforů.
8. Modifikujte konceptuální řešení problému čtenářů a písařů uvedené v textu tak, aby nehrozila možnost vyhladovění písařů.
9. Jaké nebezpečí hrozí u řešení problémů večeřejících filozofů, pokud bychom přidělovali hůlky vždy ve stejném pořadí? Co musí platit pro přidělování hůlek, aby tento problém nehrozil?
10. Může dojít k uváznutí v systému s jediným procesem?
11. Pomocí vhodného systému semaforů implementujte koncept monitorů a podmínek nad nimi.
12. Podívejte se do zdrojových kódů Linuxu či FreeBSD, jak jsou semafore (či další synchronizační mechanismy) skutečně implementovány.
13. Co je to uváznutí, blokování a stárnutí?
14. Jaké jsou nutné podmínky uváznutí? Co je to prevence uváznutí, vyhýbání se uváznutí (a jak lze k němu využít graf alokace zdrojů) a detekce uváznutí a zotavení se z něho? Jaké jsou výhody a nevýhody těchto přístupů?
15. Co je to formální verifikace, jaké má výhody oproti testování, jakou podobu může mít automatizovaná formální verifikace?



5.8 Řešení vybraných kontrolních otázek

8. Postačí např. přidat další semafor *wrtWaiting*, který písař zamkne na začátku čekání a odemkne při získání přístupu do kritické sekce a čtenáři ho zamknou a ihned opět odemknou ještě před tím, než začnou jakékoliv další synchronizační akce.
9. Hrozí uvážnutí. Obecně je možné se uvážnutí vyhnout při asymetrickém přidělování zdrojů (hůlek). Jinou možností je přidělovat vždy obě hůlky najednou, což bychom mohli implementovat např. dalším sdíleným semaforem, který by každý filozof zamknul před začátkem alokace hůlek. Tímto způsobem by ale filozof čekající na určitou hůlku mohl blokovat filozofa, který chce jinou hůlku, která je právě k dispozici.
10. Ano, stačí provést dvakrát *lock* nad stejným semaforem, aniž by byl mezitím uvolněn. Některé implementace semaforů s tímto dokonce počítají a umožní témuž procesu semafor uzamknout vícenásobně (použito např. v jádru FreeBSD). Otázka je, zda je tento přístup koncepční.



Literatura

- [1] T. Ball and S. K. Rajamani. The SLAM Toolkit. In *Proc. of CAV'01*, LNCS. Springer, 2001.
- [2] G. Brat, K. Havelund, S. Park, and W. Visser. Java PathFinder – Second Generation of a Java Model Checker. In *Workshop on Advances in Verification*, 2000.
- [3] E.M. Clarke, O. Grumberg, and D. Peled. *Model Checking*. MIT Press, 1999.
- [4] E.W. Dijkstra. Cooperating Sequential Processes. Technical report, Technological University, Eindhoven, The Netherlands, 1965.
- [5] D. Engler and M. Musuvathi. Static Analysis versus Software Model Checking for Bug Finding. In *Proc. of VMCAI'04*, volume 2937 of LNCS. Springer, 2004.
- [6] P. Brinch Hansen. Monitors and Concurrent Pascal: A Personal History. *ACM SIGPLAN Notices*, 28(3), 1993.
- [7] T. A. Henzinger, R. Jhala, R. Majumdar, and G. Sutre. Software Verification with Blast. In *Proc. of 10th SPIN Workshop*, volume 2648 of LNCS. Springer, 2003.
- [8] C.A.R. Hoare. Monitors: An Operating System Structuring Concept. *Communications of the ACM*, 17(10), 1974.
- [9] G.J. Holzmann. The Model Checker Spin. *IEEE Transactions on Software Engineering*, 23(5), May 1997.
- [10] T. Kašpárek. Co před námi tají /proc. www.root.cz, červen 2001.
- [11] L. Lamport. A New Solution of Dijkstra's Concurrent Programming Problem. *Communications of the ACM*, 17:453–455, 1974.
- [12] K.L. McMillan. The SMV System. Technical report, Carnegie-Mellon University, 1992.
- [13] Sun Microsystems. *Solaris ZFS Administration Guide*. Sun Microsystems, Santa Clara, CA, 2006. URL: <http://www.opensolaris.org/os/community/zfs/docs/zfsadmin.pdf>.

- [14] T. Nipkow, L.C. Paulson, and M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2005.
- [15] S. Owre, N. Shankar, J.M. Rushby, and D.W.J. Stringer-Calvert. *PVS System Guide*. Computer Science Laboratory, SRI International, Menlo Park, California, USA, 2001. URL: <http://pvs.csl.sri.com/>.
- [16] S.D. Pate. *Distributed Operating Systems & Algorithms*. Addison Wesley Longman, 1997.
- [17] S.D. Pate. *UNIX Filesystems*. John Wiley & Sons, 2003.
- [18] G.L. Peterson. Myths about the Mutual Exclusion Problem. *Information Processing Letters*, 12:115–116, 1981.
- [19] G. Ricart and A.K. Agrawala. An Optimal Algorithm for Mutual Exclusion in Computer Networks. *Communications of the ACM*, 24:9–17, 1981.
- [20] M.I. Schwartzbach. *Lecture Notes on Static Analysis*, 2006. BRICS, Department of Computer Science, University of Aarhus, Denmark.
- [21] A. Silberschatz, P.B. Galvin, and G. Gagne. *Operating Systems Concepts*, 7th ed. John Wiley & Sons, 2004.
- [22] A. Tanenbaum. *Modern Operating Systems*, 2nd ed. Prentice Hall, 2001.
- [23] A. Valmari. The State Explosion Problem. In *Lectures on Petri Nets I: Basic Models*, volume 1491 of *LNCS*. Springer, 1998.